

PD-LMI package User Manual

Version v1.1.2

July 27, 2026

Abstract

The PD-LMI package is a MATLAB/YALMIP package for modelling and certifying parameter-dependent linear matrix inequalities (PD-LMIs). It represents coefficient-backed known matrices and decision matrices as cell-local tensor-product Bernstein polynomials on a physical parameter grid, with continuous shared-face storage for decision functions. This representation provides a finite, inspectable bridge from a continuous parameter domain to semidefinite constraints.

The manual develops that bridge through a reproducible differentiable PD-LMI workflow: define the parameter grid and known data, construct continuous decision matrices, differentiate them cell by cell when scheduling-rate terms are present, form a residual inequality, choose a direct or certificate-based positivity condition, export the resulting constraints to YALMIP, and solve them with an available SDP solver. It also makes explicit the distinction between shared-face continuity of the represented function and the generally discontinuous cellwise derivative data.

Documentation snapshot v1.1.2 presents the implemented and tested public surface as one supported workflow: installation and verification; known-data, decision-variable, and finite-certificate objects; rate-aware `pdmatrix` construction, differentiation, algebra, inspection, plotting, comparison, and known-data inequalities; transient fast or strict validation diagnostics; MATLAB-style matrix operations and indexing implemented centrally by `pdbase` and inherited by `pdmatrix`/`pdvar`; six shared helper utilities; and Direct, Pólya, Putinar, SparseFullBox, and FullBox certificate assembly. The appendices provide the Bernstein-polynomial and positivity-certificate background needed to interpret those backends and their documented boundaries. The latest tagged GitHub Release remains v1.1.0.

Contents

Version History	vi
1 From LPV Models to Finite Solver Constraints	1
1.1 Model an LPV Induced- L_2 -Gain Problem	1
1.2 Follow the Package Path	2
1.3 Version 1.1.2 Documentation Scope	2
1.4 Assembly Optimizations and Validation Diagnostics	3
1.5 The Chain Rule in a DPD-LMI	4
1.6 Grid the Scheduling-Parameter Space	5
1.7 Desired Behavior Across Cell Boundaries	6
1.8 Bernstein Basis and Cell-Local Storage	7
1.8.1 Physical nodes, local labels, and continuity by storage	7
1.8.2 A two-dimensional storage example	8
1.9 From Cellwise Algebra to Solver Constraints	8
1.9.1 Store Known Matrices and Decision Matrices	9
1.9.2 Multiply in Bernstein Coefficient Space	9
1.9.3 Differentiate and Enumerate Rate Vertices	10
1.9.4 Form a Residual on Every Hypercube	11
1.9.5 Choose a Finite Certificate	12
1.9.6 Export to YALMIP and Recover the Solution	12
2 Install and Verify the Package	13
2.1 One-Call Installation	13
2.2 Validate the Installation	14
2.3 Run a Minimal Solver Example	14
2.4 Navigate the Reference	15
3 pdbase: Grid and Storage Backend	16
3.1 API Map	16
3.2 Construct <code>pdbase</code> Objects	16
3.3 Inspect Grid and Storage Properties	19
3.4 Inspect <code>cells</code> , <code>lbls</code> , and <code>coeffs</code>	20
3.5 <code>evaluate</code> Stored Coefficients	21
3.6 Differentiate Stored Coefficients with <code>rhodiff</code>	22
3.7 Inspect Counts and Shape	23
3.8 Apply Shared Matrix Operations	25

3.8.1	Unary Signs, Transpose, Vectorization, Diagonals, And Reshape	25
3.8.2	Triangular Parts And Reductions	26
3.8.3	Reordering, Rotation, And Repetition	28
3.8.4	Concatenation Dispatch	29
3.9	Transform Bernstein Storage	30
3.9.1	<code>elevate</code>	30
3.9.2	<code>elevVals</code>	31
3.9.3	<code>elevLocalValues</code>	32
3.9.4	<code>bernElev</code>	33
3.9.5	<code>bernProd</code>	34
3.9.6	<code>bernTbl</code> , <code>mapVals</code> , and <code>matSubs</code>	34
3.9.7	<code>mergeGrid</code>	35
3.10	Boundaries and Related APIs	37
4	<code>pdmat</code>: Known Parameter-Dependent Matrices	38
4.1	API Map	38
4.2	Construct <code>pdmat</code> Objects	38
4.3	Inspect Object Properties	42
4.4	Inspect And Elevate Inherited Bernstein Storage	42
4.5	<code>evaluate</code>	44
4.6	Differentiate Known Data with <code>rhodiff</code>	45
4.7	Inspect Coefficients and Plot Values	46
4.7.1	<code>bernsteinTable</code>	46
4.7.2	<code>disp</code> And <code>display</code>	48
4.7.3	<code>plot</code>	49
4.8	Combine Known Matrices	52
4.8.1	Signs, Addition, And Subtraction	52
4.8.2	Ordered Matrix Multiplication	54
4.9	Compare and Certify Known Data	55
4.10	Reshape and Transform Known Matrices	56
4.10.1	Shape, Equality, And Display Protocols	56
4.10.2	Transpose And Conjugate Transpose	57
4.10.3	Vectorization, Reshape, And Squeeze	58
4.10.4	Diagonals And Trace	59
4.10.5	Triangular Parts And Reductions	60
4.10.6	Reordering And Rotation	61
4.10.7	Concatenation And Block Assembly	62
4.10.8	Repetition	63
4.11	Index and Assign <code>pdmat</code> Entries	64

4.12	Boundaries and Related APIs	65
5	pdvar: Decision Matrix Functions	66
5.1	API Map	66
5.2	Construct <code>pdvar</code> Decisions	66
5.3	Inspect Object Properties	70
5.4	Inspect, Evaluate, And Elevate Inherited Storage	70
5.5	<code>rhodiff</code>	72
5.6	<code>bernsteinTable</code>	74
5.7	Build Affine Algebra with Known Data	75
5.7.1	Signs, Addition, And Subtraction	76
5.7.2	Multiplication By Known Or Numeric Data	77
5.8	Reshape and Transform Decision Matrices	79
5.8.1	Shape, Equality, And MATLAB Protocols	79
5.8.2	Transpose And Conjugate Transpose	80
5.8.3	Vectorization, Reshape, And Squeeze	80
5.8.4	Diagonals And Trace	81
5.8.5	Triangular Parts And Reductions	82
5.8.6	Reordering And Rotation	83
5.8.7	Concatenation And Block Assembly	84
5.8.8	Repetition	85
5.9	Index and Assign <code>pdvar</code> Entries	86
5.10	<code>value</code>	87
5.11	Comparisons To <code>pdlmi</code>	88
5.12	Boundaries and Related APIs	90
6	pdlmi: Finite Certificate Assembly	91
6.1	API Map	91
6.2	Inspect Certificate State	92
6.3	Construct <code>pdlmi</code> Comparisons	93
6.4	Assemble Direct Coefficient Constraints	96
6.5	Select Pólya Assembly with <code>applyPolya</code>	98
6.6	Select Putinar Assembly with <code>applyPutinar</code>	99
6.7	Select SparseFullBox with <code>applySparseFullBoxPreorder</code>	101
6.8	Select FullBox with <code>applyFullBoxPreorder</code>	103
6.9	Export with <code>toYalmip</code>	105
6.10	Boundaries and Related APIs	107
6.10.1	Solve with YALMIP	107

7	Shared Helper Utilities	108
7.1	<code>helper.cellGet</code>	108
7.2	<code>helper.chk</code>	108
7.3	<code>helper.combRows</code>	109
7.4	<code>helper.isZero</code>	110
7.5	<code>helper.mkGrid</code>	111
7.6	<code>helper.mkNest</code>	111
8	Worked Solver Examples	113
8.1	Deterministic Putinar, SparseFullBox, And FullBox Selection	113
8.2	Solve for a Parameter-Dependent Lyapunov Matrix	115
8.3	Solve a Rate-Dependent Block LMI	116
A	Bernstein Representation and Exact Operations	118
A.1	Convex Combinations Before Polynomials	118
A.2	Degree-One Interpolation On One Physical Cell	119
A.3	Higher Degree And The Partition Of Unity	119
A.4	Worked Conversion: $1 + 2\alpha + 3\alpha^2$	120
A.5	Exact Power–Bernstein Conversion	121
A.6	Lossless Degree Elevation	121
A.7	Physical Tensor Grids And Local Tensor Labels	122
A.8	Products As Ordered Weighted Convolution	123
A.9	Differentiation And Rate Vertices	125
A.9.1	Rate-Vertex Row Storage	126
A.10	de Casteljau Evaluation And Exact Subdivision	126
A.11	What Changes, And What Does Not	127
A.12	Further Reading	127
B	Polynomial-Matrix Certificates on Hypercubes	128
B.1	Positive Semidefinite Matrices And LMIs	128
B.2	SOS And Gram Matrices	129
B.3	Full-Space SOS	129
B.4	Direct Bernstein Coefficients	129
B.5	Pólya As Lossless Elevation	130
B.6	Exact Interval Markov–Lukács Forms	131
B.7	Putinar Modules And Schmüdgen Preorderings	132
B.8	The Implemented Multivariate FullBox State	133
B.9	The Implemented SparseFullBox Band-Limited State	135
B.10	Six Different Modeling Choices	136

B.11 Implemented Boundary	137
B.12 Further Reading	137
References	138
Index	140

Version History

The entries below distinguish the current documentation snapshot from tagged GitHub Releases and retain the latest release from each completed minor line.

v1.1.2 documentation — July 27, 2026

Consolidates the v1.1 documentation line: it retains the reorganized LPV induced- L_2 -gain spine and standardized notation, documents the operation-local Bernstein and Gram assembly optimizations, adds the transient `ValidationMode` diagnostic option, and covers rate-aware `pdmatrix` construction, differentiation, algebra, inspection, plotting, logical equality, and known-data inequalities. It also corrects `SparseFullBox` terminology to `FullBox` exact coefficient identities with band-limited Gram support. This documentation snapshot creates no tag or GitHub Release; v1.1.0 remains the latest tagged GitHub Release.

v1.1.0 — July 26, 2026 — latest GitHub Release

Documents the centralized `pdbase` ownership of shared matrix protocols and transformations while retaining complete `pdmatrix`/`pdvar` discoverability, reclassifies three object-aware backend utilities out of the six-function `+helper` namespace, and adds complete `SparseFullBox` constructor, state, certificate, endpoint, validation, and mathematical coverage. It adds no solver wrapper, hidden margin, or runtime API beyond the current source.

v1.0.0 — July 19, 2026

Marks the first stable public release of the documented package workflow. It consolidates installation and test-suite verification, the `pdbase`/`pdmatrix`/`pdvar`/`pdlmi` object model, continuous arbitrary-degree decision storage and rate-vertex differentiation, MATLAB-style matrix operations and indexing, and direct, Pólya, Putinar, and full-box finite certificate assembly. The 1.0 documentation also makes the affine-expression, tensor-box, certificate-sufficiency, and solver-ownership boundaries explicit; this milestone does not add new runtime behavior beyond the implementation covered by the current code and tests.

v0.4.7 — July 18, 2026

Completed the v0.4 manual and Web explorer refresh around the LPV-to-cellwise-Bernstein-to-solver workflow, including installer guidance, the public API reference, mathematical cross-references, and storage-transformation examples. Released by commit `c6a2e25`.

v0.3.6 — July 16, 2026

Completed the public rename to `pdbase`/`pdmatrix`/`pdvar`/`pdlmi`, corrected forward Bernstein coordinates, added `FullBox` and Putinar certificates with independent SOS validation, hardened API boundaries, and defined entry-wise inequality behavior. Released by commit `fe74a65`.

v0.2.12 — July 13, 2026

Completed the `pdmatrix`/`pdvar`/`pdlmi` layers, rate-aware differentiation, display, plotting, structural algebra, direct finite assembly, Pólya selection, and deterministic reference examples and tests. Released by commit `98b63e5`.

v0.1.0 — July 3, 2026

Introduced the original grid-and-storage foundation and the first package manual, including Bernstein traversal, validation, and helper contracts. Released by commit `63c06ee`.

Chapter 1

From LPV Models to Finite Solver Constraints

This chapter develops one complete modeling spine: an LPV plant with an induced- L_2 -gain objective, a bounded-real DPD-LMI, exact reduction of the scheduling-rate box, cell-local Bernstein representation on a tensor grid, a finite positivity certificate, and explicit YALMIP solution recovery. Each section answers one decision that a first-time user must make; Chapters A and B retain the detailed coefficient and Gram derivations.

1.1 Model an LPV Induced- L_2 -Gain Problem

Many systems are usefully linear only after their operating condition is made explicit. An LPV model retains the linear state-space structure while allowing its known matrices to depend on a measurable scheduling vector $\boldsymbol{\rho}(t) = (\rho_1(t), \dots, \rho_\ell(t))$ in a parameter box $\mathcal{P}$. The canonical analysis model in this manual is

$$\begin{aligned}\dot{x}(t) &= A(\boldsymbol{\rho}(t))x(t) + B_w(\boldsymbol{\rho}(t))w(t), \\ z(t) &= C_z(\boldsymbol{\rho}(t))x(t) + D_{zw}(\boldsymbol{\rho}(t))w(t),\end{aligned}\tag{1.1}$$

where $x(t)$ is the plant state, $w(t)$ is the disturbance, and $z(t)$ is the performance output. The objective is to certify zero-state induced- L_2 gain below γ : $\|z\|_2 < \gamma\|w\|_2$ for every nonzero admissible disturbance and every admissible scheduling trajectory.

A differentiable parameter-dependent Lyapunov matrix $P(\boldsymbol{\rho}) = P(\boldsymbol{\rho})^\top$ yields the bounded-real DPD-LMI [1]. Write $\boldsymbol{\nu} = \dot{\boldsymbol{\rho}}$, let $\mathcal{R}$ be the admissible rate box, and require

$$P(\boldsymbol{\rho}) \succ 0,\tag{1.2}$$

$$\mathcal{F}_{\text{BR}}(\boldsymbol{\rho}, \boldsymbol{\nu}, \gamma) = \begin{bmatrix} \dot{P}(\boldsymbol{\rho}, \boldsymbol{\nu}) + \text{He}(P(\boldsymbol{\rho})A(\boldsymbol{\rho})) & P(\boldsymbol{\rho})B_w(\boldsymbol{\rho}) & C_z(\boldsymbol{\rho})^\top \\ B_w(\boldsymbol{\rho})^\top P(\boldsymbol{\rho}) & -\gamma I & D_{zw}(\boldsymbol{\rho})^\top \\ C_z(\boldsymbol{\rho}) & D_{zw}(\boldsymbol{\rho}) & -\gamma I \end{bmatrix} \prec 0\tag{1.3}$$

for every $(\boldsymbol{\rho}, \boldsymbol{\nu}) \in \mathcal{P} \times \mathcal{R}$. Here $\text{He}(X) = X + X^\top$ and $\dot{P} = \sum_s \nu_s \partial P / \partial \rho_s$. The first inequality makes $V(t) = x(t)^\top P(\boldsymbol{\rho}(t))x(t)$ positive definite; the second supplies the dissipation inequality for the induced- L_2 bound. The strict signs shown here are analysis notation: the package inserts no hidden margin, so a numerical model must choose and encode conditions such as $P(\boldsymbol{\rho}) \succeq \varepsilon I$ explicitly. Setting $\boldsymbol{\nu} = 0$ removes the scheduling derivative and recovers an ordinary PD-LMI.

1.2 Follow the Package Path

The package starts after the user has chosen an LPV model, a parameter box, any scheduling-rate bounds, and the matrix inequality to certify. Its role is to represent the known and decision matrices, convert the parameter continuum into finite certificate constraints, and hand those constraints to YALMIP. It does not generate the plant or controller structure, choose an objective or SDP solver, or interpret solver diagnostics on the user’s behalf.

The main objects follow the same path used in the rest of the manual:

1. `pdmat` stores known parameter-dependent matrices, while `pdvar` creates continuous parameter-dependent decisions such as $P(\rho)$ and γ -dependent affine expressions. Both inherit grid, degree, label, cell-local storage, matrix-shape protocols, and shared structural operations from the backend class `pdbase`; ordinary users begin with `pdmat` and `pdvar`, not `pdbase`.
2. Matrix operations assemble the original residual $\mathcal{F}$. When scheduling-rate terms are present, `rhodiff` differentiates a `pdvar` on each cell and stores one derivative row for every vertex of $\mathcal{R}$.
3. A comparison such as `residual <= 0` creates a `pdlmi`, orients the comparison as a positive target S , and selects Direct by default. `applyPolya`, `applyPutinar`, `applySparseFullBoxPreorder`, and `applyFullBoxPreorder` rebuild alternative finite certificates from the same original residual.
4. `toYalmip` exports the stored constraints. The user calls YALMIP `optimize`, accepts an optimized γ only when `sol.problem == 0`, and calls `value` to recover numeric Bernstein data.

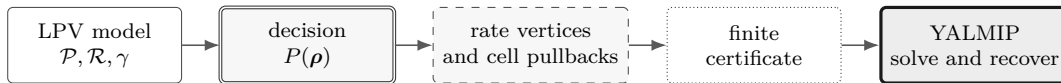


Figure 1.1: The end-to-end documentation spine. Rate-vertex traversal is an exact reduction for affine rate dependence; the parameter-domain certificate is sufficient and is followed by an explicit solver-status check.

Two finite steps in this path have different logical status. Traversing all vertices of $\mathcal{R}$ is exact when $\mathcal{F}$ is affine in ν . By contrast, a Bernstein or Bernstein–Gram construction is a sufficient certificate over the remaining ρ -continuum; failure at a fixed grid, degree, or certificate order does not prove that the original continuous inequality is infeasible. Sections B.10 and B.11 separates certificate selection from changes to the underlying model, and section 1.9 executes the path in MATLAB.

1.3 Version 1.1.2 Documentation Scope

Version 1.1.2 is a documentation snapshot of the implemented workflow. The supported surface comprises the `pdbase` storage backend and its shared matrix-operation implementation; rate-aware `pdmat` known data and `rhodiff` derivatives; continuous arbitrary-degree `pdvar` decisions and their cellwise `rhodiff` derivatives; `pdlmi` Direct, Pólya, Putinar, SparseFullBox, and FullBox assembly for decision expressions and coefficient-backed known inequalities; the documented MATLAB matrix-operation and indexing overloads; and the six public `helper` utilities. The source-indexed reference

chapters state the accepted forms, returned shapes, validation rules, and error boundaries for that surface.

Centralizing shared matrix operations in `pdbase` does not remove those operations from `pdmat` or `pdvar`: both modeling classes retain complete public, class-specific reference entries and behavior. The package remains restricted to tensor-product box grids, affine decision expressions, and the finite certificate constructions described here. Products nonlinear in decision or rate-dependent data are rejected, certificate failure remains inconclusive for the original continuum inequality, and YALMIP retains ownership of objectives, solver selection, and diagnostics. Version 1.1.0 remains the latest tagged GitHub Release; v1.1.2 is a documentation milestone and creates no tag or release. For release verification, run the root installer and `tests.run_all()` as described in Chapter 2. Examples involving `pdvar`, `rhodiff`, `pdlmi`, or solver state begin from a cleared YALMIP model so that their decision variables and constraints are reproducible in a fresh MATLAB session.

1.4 Assembly Optimizations and Validation Diagnostics

The v1.1.2 implementation substantially rewrites internal assembly while leaving the ordinary modeling path largely compatible. Bernstein elevation and multiplication build numeric operation-local plans once and reuse them across compatible cells and rate rows; Direct and Pólya assembly preallocate in the established cell, rate-row, coefficient, and column-major entry order; Gram families reuse numeric coefficient maps while allocating fresh independent Gram variables for every required block; and `toYalmip` concatenates retained constraints once. These changes preserve the represented function, decision-variable identities, certificate family, constraint and cone order, and exact coefficient identities, up to scale-appropriate floating-point summation differences.

The public constructors `pdbase`, `pdmat`, `pdvar`, and `pdlmi` accept the transient name-value option `ValidationMode="fast"` or `ValidationMode="strict"`, case-insensitively; the default is `"fast"`. Each `applyPolya`, `applyPutinar`, `applySparseFullBoxPreorder`, and `applyFullBoxPreorder` call accepts the same trailing option. The selection is local to that call, is not stored as an object property, and is not inherited by later algebra or certificate-selection calls. The developer-facing raw `pdbase` constructor is a deliberate exception to broad fast-mode claims: it checks repeated coefficient structure only in the first supplied physical cell, so a later supplied cell can escape coefficient-count, payload, or rate-row validation; strict mode audits every supplied cell. The `pdmat` and `pdvar` constructors first normalize their public sources and retain their documented class-specific guarantees. In the normalized algebra and certificate paths, fast mode samples repeated post-normalization structural checks, while strict mode checks every generated cell, rate row, coefficient payload, elevation/product application, and Gram-plan application. Both modes retain option, compatibility, continuity, affine-safety, inequality-classification, and actual constraint-generation checks; strict is a diagnostic mode, not a different mathematical model.

```
yalmip('clear')
A = pdmat([0 1], {1, 2}, Degree=1, ValidationMode="STRICT");
P = pdvar(1, [0 1], ValidationMode="fast");
C = pdlmi(P, ">=", ValidationMode="strict");
C = C.applyPolya(0, ValidationMode="FAST");
hasStoredMode = isprop(A, "ValidationMode")
```

```
hasStoredMode = logical
0
```

The three reproducible non-solver benchmarks warm every timed function, construct setup objects outside the timed bodies, verify mathematical equivalence first, and report the median of three independent `timeit` estimates. `tests.benchmark_stage1b_performance` uses two parameters on $\{0:0.25:1, -1:0.5:1\}$, 16 physical cells, degree three, and 3×3 payloads for numeric multiplication, known-by-affine multiplication, and elevation by two. `tests.benchmark_stage1c_performance` uses a symmetric 3×3 , two-parameter, 16-cell, degree-three residual with 256 Direct constraints and 400 Pólya-+1 constraints. `tests.benchmark_stage2_performance` uses one two-parameter cell, degree two, a symmetric 2×2 FullBox/Putinar residual, a scalar SparseFullBox residual, and a one-parameter symmetric 2×2 Putinar residual. Timing results are machine- and load-specific and are not performance guarantees.

Table 1.1: Contention-free MATLAB MCP measurements from one run on the current machine. Medians are machine- and load-specific observations, not performance guarantees. The final column reports a captured-baseline speedup only where that benchmark supplied one; $0.91\times$ means the measured Direct export was slower than its captured baseline.

Stage	Workload	Median (s)	Captured-baseline speedup
1B	Numeric multiplication	0.045856	$2.01\times$
1B	Known $\times$ affine	0.744692	$1.90\times$
1B	Numeric elevate +2	0.004389	$3.19\times$
1B	Symbolic elevate +2	0.117991	—
1C	Direct, fast	0.090959	$2.51\times$
1C	Direct, strict	0.153322	—
1C	Pólya +1, fast	0.242357	—
1C	Pólya +1, strict	0.376200	—
1C	Direct <code>toYalmip</code>	0.155284	$0.91\times$ (slower)
1C	Pólya <code>toYalmip</code>	0.271730	—
2	FullBox 2D, fast	0.053643	$5.61\times$
2	FullBox 2D, strict	0.051391	—
2	Putinar 2D	0.046707	—
2	Putinar 1D	0.015255	—
2	Sparse intermediate	0.104958	—
2	Sparse saturated	0.098002	—
2	FullBox <code>toYalmip</code>	0.006123	—
2	Sparse <code>toYalmip</code>	0.019741	—

1.5 The Chain Rule in a DPD-LMI

The dot on P is a time derivative along the scheduling trajectory, not a new independent matrix. Applying the multivariable chain rule gives

$$\dot{P}(\boldsymbol{\rho}(t)) = \frac{dP}{dt} = \sum_{s=1}^{\ell} \frac{\partial P}{\partial \rho_s}(\boldsymbol{\rho}(t)) \nu_s(t).$$

Give every scheduling direction independent finite rate bounds,

$$\mathcal{R} = \prod_{s=1}^{\ell} [\underline{\nu}_s, \bar{\nu}_s].$$

For a residual affine in $\boldsymbol{\nu}$, the rate box is handled exactly by traversing its 2^ℓ lower/upper vertices $\boldsymbol{\nu}^{(q)}$. The temporary index q is needed only while constructing the vertex blocks or explaining stored rate rows. After stacking those blocks, the rate index is suppressed in all parameter-domain certificate formulas. The coefficient-level differentiation and rate-row construction are derived in section A.9. The rate-dependent inequality becomes

$$\text{diag}_{q=1}^{2^\ell} \mathcal{F}(\boldsymbol{\rho}, \boldsymbol{\nu}^{(q)}) \prec 0, \quad \boldsymbol{\rho} \in \mathcal{P}. \quad (1.4)$$

Each diagonal block is the residual at one rate vertex. The remaining continuum is only the scheduling parameter $\boldsymbol{\rho}$, which the package treats cellwise with its Bernstein certificates.

We therefore focus on the following general DPD-LMI.

General form of the differentiable parameter-dependent LMI

$$\mathcal{F}(\boldsymbol{\rho}, \boldsymbol{\nu}) = \sum_{k=1}^N R_k(\boldsymbol{\rho}) \sum_{j=1}^{\ell} \frac{\partial y_k}{\partial \rho_j}(\boldsymbol{\rho}) \nu_j + F_0(\boldsymbol{\rho}) + \sum_{k=1}^N F_k(\boldsymbol{\rho}) y_k(\boldsymbol{\rho}) \prec 0, \quad \forall (\boldsymbol{\rho}, \boldsymbol{\nu}) \in \mathcal{P} \times \mathcal{R}. \quad (1.5)$$

Here R_k , F_0 , and F_k are known matrix functions, while $y_k(\boldsymbol{\rho})$ are general parameter-dependent decision functions; $P(\boldsymbol{\rho})$ in (1.3) is the canonical matrix-valued instance. Affine dependence on $\boldsymbol{\nu}$ reduces exactly to (1.4), but $\boldsymbol{\rho} \in \mathcal{P}$ remains a continuum. The package represents each y_k as a continuous piecewise polynomial with cell-local tensor-product Bernstein coefficients.

Setting $R_k \equiv 0$ for every k removes the derivative term and recovers an ordinary PD-LMI.

1.6 Grid the Scheduling-Parameter Space

The package currently considers only an axis-aligned box parameter domain,

$$\mathcal{P} = \prod_{s=1}^{\ell} [\underline{\rho}_s, \bar{\rho}_s].$$

This restriction is deliberate, as a compact box is easy to partition into hypercubes for cellwise modelling. General polytopic or curved parameter domains are outside the implemented model. Gridding starts independently in each parameter direction. For each direction, choose the following physical grid

$$\mathcal{G}_s = \{\underline{\rho}_s = \rho_s^{(1)}, \dots, \rho_s^{(k_s)} = \bar{\rho}_s\}, \quad s = 1, \dots, \ell.$$

The full tensor grid is $\mathcal{G} = \mathcal{G}_1 \times \dots \times \mathcal{G}_\ell$. It contains $\prod_{s=1}^{\ell} k_s$ physical nodes and partitions $\mathcal{P}$ into $\prod_{s=1}^{\ell} (k_s - 1)$ physical hypercubes. A hypercube is identified by the ℓ -dimensional multi-index $\mathbf{c} = (c_1, \dots, c_\ell)$; section A.7 gives the exact physical-grid and local-label conventions:

$$\mathcal{H}_{\mathbf{c}} = \prod_{s=1}^{\ell} [\rho_s^{(c_s)}, \rho_s^{(c_s+1)}]. \quad (1.6)$$

1.7 Desired Behavior Across Cell Boundaries

The package represents coefficient-backed known data and decision matrices by local polynomials, so products, degree elevation, and differentiation can be performed one physical cell at a time. A Lyapunov matrix or comparable decision function must not jump when the scheduling trajectory crosses a shared face, so `pdvar` guarantees global C^0 continuity. It does not require C^1 continuity, that is, one-sided derivatives may differ, and `rhodiff` retains the corresponding cell-local derivative data. Known `pdmat` data are different: the object records whether adjacent numeric faces match, but it preserves explicitly supplied discontinuous local data rather than adding decision constraints to repair them.

A monomial parameterization makes the additional continuity condition explicit. For two adjacent one-dimensional cells, write

$$A^{(c)}(\alpha) = \sum_{i=0}^m \alpha^i A_i^{(c)}, \quad \alpha \in [0, 1].$$

At their shared physical node, $\alpha_1 = 1$ in the left cell and $\alpha_2 = 0$ in the right cell. Enforcing C^0 continuity therefore requires the extra matrix equality

$$\underbrace{\sum_{i=0}^m A_i^{(1)}}_{A^{(1)}(1)} = \underbrace{A_0^{(2)}}_{A^{(2)}(0)}.$$

When $\ell > 1$, equality along a shared face is a polynomial identity in the tangential coordinates and produces a corresponding family of linear coefficient equalities. The number of such conditions grows with the polynomial degree and the number of shared faces.

The right side of Fig 1.2 shows the intended behavior: the two polynomial pieces share their boundary value, while their one-sided slopes may differ. A globally C^0 , piecewise-polynomial matrix function on compact $\mathcal{P}$ is locally Lipschitz. At a shared face, its Clarke generalized derivative is the convex hull of the limiting derivatives from the adjacent cells [2]. Because (1.5) is affine in those derivatives, imposing the cell-side conditions covers that convex hull; this interface argument does not claim that ∇P is continuous. Discontinuous coefficient-backed `pdmat` data remain a supported implementation choice, but they fall outside this continuity argument and must be analyzed cellwise.

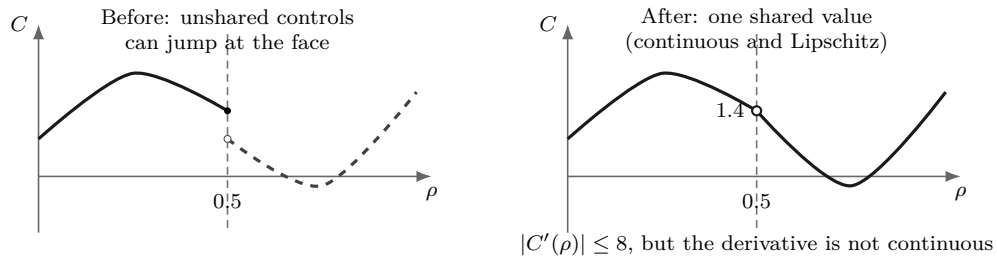


Figure 1.2: The required cell-interface behaviour. Enforcing one shared boundary value removes the jump while allowing unequal one-sided derivatives.

1.8 Bernstein Basis and Cell-Local Storage

For its cell-local polynomial representation, the package uses the Bernstein basis rather than the monomial basis in the preceding comparison. Bernstein endpoint identities expose each face polynomial directly through its face coefficients; shared decision storage can therefore impose C^0 continuity without adding separate cross-cell equalities. Inside $\mathcal{H}_{\mathbf{c}}$, define its width and forward pullback by

$$h_s^{(\mathbf{c})} = \rho_s^{(c_s+1)} - \rho_s^{(c_s)}, \quad \phi_{\mathbf{c}}(\boldsymbol{\rho}) = \boldsymbol{\alpha}, \quad \alpha_s = \frac{\rho_s - \rho_s^{(c_s)}}{h_s^{(\mathbf{c})}} \in [0, 1].$$

On one local coordinate $\alpha \in [0, 1]$, the normalized degree- m Bernstein basis is

$$B_i^m(\alpha) = \binom{m}{i} (1 - \alpha)^{m-i} \alpha^i, \quad i = 0, \dots, m.$$

The basis functions are nonnegative and sum to one. The first four nonnegative degrees are

$$\begin{aligned} m = 0 : & \quad B_0^0 = 1 \\ m = 1 : & \quad B_0^1 = 1 - \alpha, \quad B_1^1 = \alpha, \\ m = 2 : & \quad B_0^2 = (1 - \alpha)^2, \quad B_1^2 = 2(1 - \alpha)\alpha, \quad B_2^2 = \alpha^2, \\ m = 3 : & \quad B_0^3 = (1 - \alpha)^3, \quad B_1^3 = 3(1 - \alpha)^2\alpha, \quad B_2^3 = 3(1 - \alpha)\alpha^2, \quad B_3^3 = \alpha^3. \end{aligned}$$

In ℓ parameter directions, the package uses the tensor-product expansion

$$C^{(\mathbf{c})}(\phi_{\mathbf{c}}(\boldsymbol{\rho})) = \sum_{\mathbf{i} \in \{0, \dots, m\}^\ell} \left(\prod_{s=1}^{\ell} B_{i_s}^m(\alpha_s) \right) C^{(\mathbf{c})}[\mathbf{i}].$$

The matrix value is a convex combination of the local coefficient matrices. Exact degree elevation, multiplication, and differentiation operate on those coefficients without changing the identity of the physical cell. The detailed formulas appear in sections [A.3](#), [A.6](#), [A.8](#) and [A.9](#).

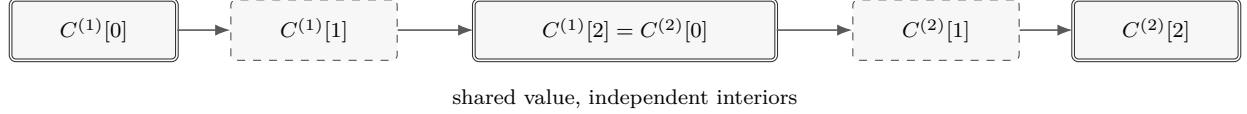
1.8.1 Physical nodes, local labels, and continuity by storage

The two index systems have different meanings:

- the physical node $\rho_s^{(j)}$ and the cell component c_s are one-based and identify geometry;
- the local Bernstein label $i_s \in \{0, \dots, m\}$ is zero-based and selects one coefficient inside an already chosen cell.

A degree- m tensor polynomial stores $(m + 1)^\ell$ local coefficient positions per physical cell. These positions are not another grid of point samples. The traversal order used by `cells`, `lbls`, `coeffs`, and rate-vertex rows varies later dimensions fastest; in two dimensions, `lbls()` lists $[0, 0], [0, 1], \dots, [0, m], [1, 0], \dots, [m, m]$. The general tensor ordering is defined in section [A.7](#).

The endpoint identities $B_0^m(0) = 1$ and $B_m^m(1) = 1$ make continuity easy to encode in coefficient data. For a `pdvar`, adjacent cells reuse the same YALMIP decision handles on their common face; in one dimension, $C^{(1)}[m]$ and $C^{(2)}[0]$ are the same decision matrix. Tensor cells similarly share every handle on a common face, so continuity requires no later equality constraint. For a coefficient-backed `pdmat`, `IsContinuous` instead reports whether the two numeric face-coefficient sets agree; explicitly mismatched local data remain cell-local and produce a warning.



1.8.2 A two-dimensional storage example

The first two-dimensional example uses $\mathcal{G}_1 = \{0, 0.5, 1\}$, $\mathcal{G}_2 = \{0, 1\}$, and degree two:

```
grid2 = {[0 .5 1], [0 1]};
A2 = pdmat(grid2, @(rho1,rho2) ...
    [1+rho1+rho2, rho1*rho2; ...
    rho1*rho2, 2+rho1^2], Degree=2);
physicalCells = A2.cells()
localLabels = A2.lbls()
secondCellCoefficients = A2.coeefs([2 1])
sameLeaf = A2.LocalValues{2}{1}
```

The grid has two physical hypercubes. The highlighted one below is $\mathbf{c} = (2, 1)$, whose domain is $[0.5, 1] \times [0, 1]$. Selecting it through `A2.coeefs([2 1])` reaches the same nested storage leaf as `A2.LocalValues{2}{1}`. The figure lists the nine matrices in `A2.lbls()` order, with i_1 varying more slowly than i_2 .

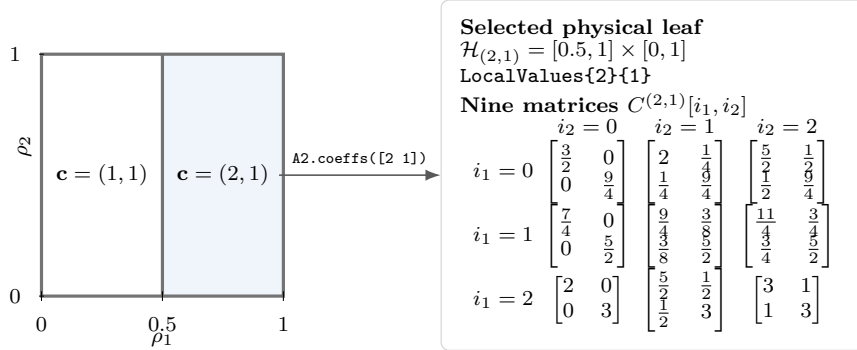


Figure 1.3: A physical hypercube selects one storage leaf. Within that leaf, `A2.lbls()` supplies the nine zero-based local labels in the displayed order. The matrices are Bernstein coefficients, not a 3-by-3 grid of point samples.

The selected leaf is a flat nine-entry coefficient cell: row q of `A2.lbls()` labels entry q . Sections A.4 and A.7 derive the ordering and conversion rules.

1.9 From Cellwise Algebra to Solver Constraints

This section first isolates the Lyapunov block $\dot{P} + \text{He}(PA)$ from (1.3) so that the cell, coefficient, and rate-row mechanics remain visible. It then reconnects that block to the complete induced- L_2 -gain model and solver objective. Known matrices are stored as `pdmat`, decisions as `pdvar`, cellwise rate derivatives are produced by `rhodiff`, and finite certificate selections are stored as `pdlmi`.

1.9.1 Store Known Matrices and Decision Matrices

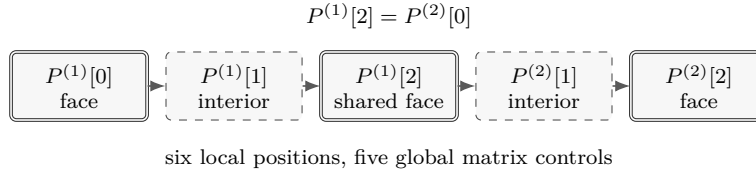
The known LPV matrix is affine in ρ on the two-cell grid $[0, 0.5] \cup [0.5, 1]$. Supplying **Degree=1** retains exact function evaluation together with cell-local Bernstein coefficient evidence.

```
yal mip('clear')
grid = {[0 0.5 1]};
A = pdmat(grid, @(rho) [-1, 0.5; -1, -2] + ...
    rho * [-1.3, -20; 2, -10], Degree=1);
P = pdvar(2, grid, "symmetric", Degree=2);
matrixSize = size(P)
physicalCellCount = A.ncell()
knownDegree = A.Degree
decisionDegree = P.Degree
isContinuous = P.IsContinuous
```

```
matrixSize = 1×2 double
    2    2

physicalCellCount = 2
knownDegree = 1
decisionDegree = 2
isContinuous =
    logical
    1
```

Each cell stores three local symmetric decision matrices. The upper face of cell 1 and lower face of cell 2 share one YALMIP decision handle, so six local positions contain five global controls. The interior controls remain independent.



1.9.2 Multiply in Bernstein Coefficient Space

On one physical cell $\mathcal{H}_{\mathbf{c}}$, write $P = \sum_i B_i^m P^{(\mathbf{c})}[i]$ and $A = \sum_j B_j^n A^{(\mathbf{c})}[j]$. The ordered product is

$$PA = \sum_{k=0}^{m+n} B_k^{m+n} R^{(\mathbf{c})}[k], \quad R^{(\mathbf{c})}[k] = \sum_{i+j=k} \frac{\binom{m}{i} \binom{n}{j}}{\binom{m+n}{k}} P^{(\mathbf{c})}[i] A^{(\mathbf{c})}[j].$$

For quadratic P and affine A , the four result coefficients are

$$\begin{aligned} R[0] &= P_0 A_0, & R[1] &= (P_0 A_1 + 2P_1 A_0)/3, \\ R[2] &= (2P_1 A_1 + P_2 A_0)/3, & R[3] &= P_2 A_1. \end{aligned}$$

The anti-diagonal grouping $k = i + j$ and binomial normalization are exact; this is not a pointwise sample product, and matrix order is preserved. Section A.8 derives the general tensor-product rule.

```
PA = P * A;
```

```

ATP = A' * P;
degreePA = PA.Degree
degreeATP = ATP.Degree
coefficientCountPA = numel(PA.coeffs(1))

```

```

degreePA = 3
degreeATP = 3
coefficientCountPA = 4

```

1.9.3 Differentiate and Enumerate Rate Vertices

For a degree- m cell of width $h_1^{(c)}$, differentiation is the forward difference

$$\frac{\partial P}{\partial \rho} = \frac{m}{h_1^{(c)}} \sum_{i=0}^{m-1} (P^{(c)}[i+1] - P^{(c)}[i]) B_i^{m-1}(\alpha).$$

Because the chain-rule term is affine in the bounded rate, checking the two vertices -1 and $+1$ covers $\nu \in [-1, 1]$. Section A.9 gives the coefficient derivation and its physical-cell scaling.

For ℓ parameters, let $h_s^{(c)} = \rho_s^{(c_s+1)} - \rho_s^{(c_s)}$ be the width of $\mathcal{H}_c$ in direction s , and let $\mathbf{e}_s$ be the corresponding unit multi-index. Differentiating the tensor-product representation in section 1.8 gives

$$\frac{\partial P^{(c)}}{\partial \rho_s} = \frac{m}{h_s^{(c)}} \sum_{\substack{\mathbf{i} \in \{0, \dots, m\}^\ell \\ i_s < m}} (P^{(c)}[\mathbf{i} + \mathbf{e}_s] - P^{(c)}[\mathbf{i}]) B_{i_s}^{m-1}(\alpha_s) \prod_{r \neq s} B_{i_r}^m(\alpha_r), \quad \dot{P}^{(c)} = \sum_{s=1}^{\ell} \nu_s \frac{\partial P^{(c)}}{\partial \rho_s}.$$

Each partial derivative has degree $m-1$ in its differentiated direction and degree m in every other direction. For $\ell > 1$, `rhodiff` therefore elevates the shortened direction back to degree m before summing the rate-weighted partials. If $\nu_s \in [\underline{\nu}_s, \bar{\nu}_s]$, the rate box $\mathcal{R}$ has 2^ℓ Cartesian vertices $\boldsymbol{\nu}^{(q)}$; because $\dot{P}$ is affine in $\boldsymbol{\nu}$, traversing those vertices is exact. For $m > 0$, each multidimensional cell consequently stores 2^ℓ rate rows with $(m+1)^\ell$ tensor-Bernstein coefficients per row.

```

D = rhodiff(P, [-1 1]);
derivativeDegree = D.Degree
isContinuous = D.IsContinuous
rateRowCount = size(D.coeffs(1),1)
coefficientsPerRow = size(D.coeffs(1),2)

```

```

derivativeDegree = 1
isContinuous =
    logical
    0
rateRowCount = 2
coefficientsPerRow = 2

```

For example, a degree-one decision on one two-parameter cell has four rate vertices and four tensor-Bernstein labels:

```

rb2 = [-1 2; -3 5];
P2 = pdvar(1, {[0 2], [10 20]}, Degree=1, RateBounds=rb2);
D2 = rhodiff(P2);
tensorDerivativeDegree = D2.Degree

```

```
rateRowCount2D = size(D2.coeffs([1 1]), 1)
coefficientsPerRow2D = size(D2.coeffs([1 1]), 2)
```

```
tensorDerivativeDegree = 1
rateRowCount2D = 4
coefficientsPerRow2D = 4
```

Thus a coefficient table is indexed independently by physical hypercube, stored rate row, and tensor-Bernstein label. Rate rows are neither physical cells nor additional Bernstein labels. Once the rows are stacked into the finite residual, subsequent certificate formulas suppress the temporary vertex index q .

1.9.4 Form a Residual on Every Hypercube

The MATLAB residual isolates the Lyapunov block $\dot{P} + \text{He}(PA)$ of the full bounded-real LMI in (1.3); the separate positivity comparison implements the numerical normalization chosen for (1.2). Addition performs exact degree elevation before the cell-local terms are combined.

```
residual = D + P * A + A' * P;
Cdecay = residual <= -1e-6 * eye(2);
Cpositive = P >= eye(2);
residualDegreeM = residual.Degree
decayConstraintCount = numel(Cdecay.Constraints)
positivityConstraintCount = numel(Cpositive.Constraints)
```

```
residualDegreeM = 3
decayConstraintCount = 16
positivityConstraintCount = 6
```

The assembled decay residual has degree $M = 3$: two physical cells times two stored rate rows times four local labels give sixteen coefficient constraints. Positivity has no rate rows and has three degree-two labels per cell. Shared faces are visited from each adjoining cell because certification remains cell-local. For a comparison $\mathcal{F} \preceq 0$, `pdlmi` stores the original residual and sign-normalizes the positive target as $S = -\mathcal{F}$; for $\mathcal{F} \succeq 0$, it uses $S = \mathcal{F}$.

1.9.5 Choose a Finite Certificate

Direct Bernstein-coefficient constraints are the default. Four opt-in selectors rebuild from the same original residual and relation; selections replace rather than compound one another.

```
Cpolya = Cdecay.applyPolya(1);
Cputinar = Cdecay.applyPutinar();
Csp = Cdecay.applySparseFullBoxPreorder(2);
Cbox = Cdecay.applyFullBoxPreorder();
counts = [numel(Cdecay.Constraints), ...
          numel(Cpolya.Constraints), ...
          numel(Cputinar.Constraints), ...
          numel(Csp.Constraints), ...
          numel(Cbox.Constraints)]
```

```
counts = 1×5 double
    16     20     24     24     24
```

Direct tests the coefficients of S . Pólya elevates S by increment d before applying the same test. In one parameter, the Gram paths use the parity-specific Markov–Lukács interval representation; for this degree- $M = 3$ residual, absolute order $r = 1$ is the default. Width $b = 2$ therefore reaches the dense endpoint and SparseFullBox canonicalizes to actual FullBox state, while Putinar delegates to the same interval form; all three wrappers store 24 constraints. In multiple parameters, Putinar uses the singleton-generator box module, SparseFullBox retains every FullBox generator-mask term and exact identity while restricting Gram support to bandwidth b , and the full-box preordering uses dense support for every box-generator product. The sparse support is implemented by overlapping tensor-window PSD cliques. All five software states are finite sufficient certificates for the continuous inequality. Failure of a selected state at fixed M, d, r, b is inconclusive. Sections B.4 to B.9 give the constructions, while section B.11 states their implemented boundaries.

1.9.6 Export to YALMIP and Recover the Solution

toYalmip returns exactly the stored finite constraints. It does not select a solver, add an objective, or insert a strictness margin.

```
Fdecay = toYalmip(Cdecay);
Fpositive = Cpositive.toYalmip();
opts = sdpsettings('solver', 'sedumi', 'verbose', 0);
sol = optimize([Fdecay, Fpositive], [], opts);
assert(sol.problem == 0, sol.info)
solvedP = value(P);
```

The solver choice and `sol.problem` semantics belong to YALMIP and the installed solver. After `problem == 0`, `value(P)` replaces symbolic coefficient payloads with numeric values while preserving the grid, degree, matrix shape, and cell ordering. In the complete bounded-real model, γ is a degree-zero `pdvar`; the tested construction and objective extraction appear in section 8.3. Retain `value(gamma.LocalValues\{1\}\{1\})` only after the same successful-status check. Certificate syntax and order boundaries appear in Chapter 6.

Chapter 2

Install and Verify the Package

2.1 One-Call Installation

Start MATLAB in the checked-out repository and run the root-level installer once. The installer adds only the repository root to the end of the MATLAB path; MATLAB resolves the package's class, package, and private directories from that root. It does not recursively scan the checkout, download dependencies, or edit `startup.m`.

```
report = install_pd_lmi();
```

The returned structure records the package root, the YALMIP root discovered on the current MATLAB path, the working SDP solver, the paths added by this run, and whether the path was persisted:

```
report.PackageRoot  
report.YALMIPRoot  
report.Solver  
report.AddedPaths  
report.Persisted
```

YALMIP must already be on the active MATLAB path. The installer validates `yalmip`, `sdpvar`, `sdpsettings`, `optimize`, and `getavailablesolvers`; it then probes available SDP solvers in the fixed commercial-first order MOSEK, COPT, SeDuMi, SDPT3, and LMILAB using a tiny bounded SDP. A failed probe falls through to the next candidate. If any precondition, shadow check, solver probe, or path persistence operation fails, the installer restores the original MATLAB path and stops with one of these stable errors:

```
install_pd_lmi:MissingYALMIP  
install_pd_lmi:IncompleteYALMIP  
install_pd_lmi:NoWorkingSDPSolver  
install_pd_lmi:PathConflict  
install_pd_lmi:PathSaveFailed
```

Repeated successful calls are idempotent: when the repository root is already present, the installer adds no duplicate path entry and returns an empty `AddedPaths`.

2.2 Validate the Installation

After the installer has selected and recorded a working solver, run the current MATLAB test entry point from the repository root:

```
results = tests.run_all();
```

The entry point covers installer rollback and persistence behavior, helper utilities, `pdbase`, `pdmatrix`, `pdvar`, and `pdlmi`. YALMIP-backed tests and the solver smoke tests use the same automatic commercial-first solver policy as the installer; their diagnostics identify the actual solver used.

2.3 Run a Minimal Solver Example

The installer already rejects shadowed package classes before it persists the path. To inspect the selected solver and exercise the solver-facing assembly boundary, use the report together with a small known-data example:

```
report.Solver
A = pdmat({[0 1]}, {1, 3}, Degree=1);
T = bernsteinTable(A, "oneLine");
disp(T)
```

CellSubscript	Expression
{[1]}	"(1-alpha)*1 + alpha*3"

```
yalmip('clear')
P = pdvar(2, {[0 1]}, "symmetric", Degree=4);
C = P >= 0;
F = toYalmip(C);
Cputinar = C.applyPutinar();
Fputinar = toYalmip(Cputinar);
Csp = C.applySparseFullBoxPreorder(2);
Fsp = toYalmip(Csp);
Cbox = C.applyFullBoxPreorder();
Fbox = toYalmip(Cbox);
```

`F` is the direct coefficient certificate; `Fputinar`, `Fsp`, and `Fbox` are the default Putinar, width-two SparseFullBox, and FullBox certificates for the same stored residual. Any one can be passed to ordinary YALMIP calls. Use the selected solver from the report when an explicit solver setting is useful:

```
opts = sdpsettings('solver', report.Solver, 'verbose', 0);
sol = optimize(F, objective, opts);
```

2.4 Navigate the Reference

Category	Entries
Backend	pdbase , constructor , cells , lbls , coeffs , ncell , ncoeff , npar , shape protocols , shared matrix operations
Known data	pdmat , construction , evaluate , bernsteinTable , disp/display , plot , operators , matrix methods
Decision variables	pdvar , construction , rhodiff , affine/mixed algebra , matrix methods , comparisons
LMIs	pdlmi , construction , direct coefficient assembly , applyPolya , applyPutinar , applySparseFullBoxPreorder , applyFullBoxPreorder , toYalmip , solver examples

Chapter 3

pdbase: Grid and Storage Backend

`pdbase` is the developer-facing backend parent for `pdmat` and `pdvar`. Ordinary users should model known data with `pdmat` and decision expressions with `pdvar`. This section documents `pdbase` because it owns the shared storage, traversal, inspection, matrix-shape, unary-transformation, reduction, and reordering implementations inherited by both public modeling classes. Central implementation does not remove these methods from `pdmat` or `pdvar`; their class-specific syntax, examples, validation, and limitations remain documented in sections 4.10 and 5.8.

Its traversal contract is the concrete form of the notation in section A.7: `cells()` enumerates one-based physical indices `c`, `lbls()` enumerates zero-based local labels `i`, and `coeffs(c)` returns $C^{(c)}[i]$ in that label order. If a leaf contains derivative data, rows additionally enumerate the already constructed vertices of $\mathcal{R}$; rows never change $\mathcal{H}_c$ or `i`. Public `elevate/elevVals` and the protected `bernElev/bernProd/bernTbl/mapVals/matSubs/mergeGrid` mechanisms preserve this separation while owning degree alignment, ordered convolution, tables, payload mapping, matrix subscripting, and common-grid refinement.

3.1 API Map

Task	Function or field
Create backend object	<code>pdbase</code>
Inspect cells/labels	<code>cells</code> , <code>lbls</code> , <code>coeffs</code>
Evaluate a stored function	<code>evaluate</code>
Differentiate a rate box	<code>rhodiff</code>
Count sizes and query shape	<code>ncell</code> , <code>ncoeff</code> , <code>npar</code> , <code>size/height/width/length/ndims/numel</code>
Transform matrix payloads	unary signs, transpose, diagonal, reshape, reductions, reordering, repetition, and concatenation dispatch
Elevate stored coefficients	<code>elevate</code> , <code>elevVals</code>
Backend algebra mechanisms	<code>elevLocalValues</code> , <code>bernElev</code> , <code>bernProd</code> , <code>bernTbl/mapVals/matSubs</code> , <code>mergeGrid</code>
Read storage fields	<code>GridInfo</code> , <code>LocalValues</code> , metadata fields

3.2 Construct pdbase Objects

Syntax

```
obj = pdbase(gridVectors, matrixSize, degree)
obj = pdbase(gridVectors, matrixSize, degree, localValues)
obj = pdbase(..., IsContinuous=tf)
obj = pdbase(..., ContainsDecision=tf)
obj = pdbase(..., HasRateDependence=tf)
obj = pdbase(..., RateBounds=rb)
obj = pdbase(..., SourceSummary=txt)
obj = pdbase(..., ValidationMode=mode)
```

Arguments

- **gridVectors** is a cell vector with one strictly increasing node vector per parameter, such as {[0 1 2], [10 20]}.
- **matrixSize** is a positive two-element matrix size, such as [2 2].
- **degree** is a nonnegative integer Bernstein degree, such as 1.
- **localValues** is the nested physical-cell storage. When omitted, **pdbase** creates a zero coefficient-backed object.
- **IsContinuous**, **ContainsDecision**, and **HasRateDependence** are logical metadata scalars. They default to false; subclasses set them to describe their own payload contract.
- **RateBounds** is empty or an $\ell \times 2$ matrix of finite lower and upper rate bounds, such as **RateBounds**=[-1 1; -2 2]. Nonempty bounds also mark the object as rate-dependent.
- **SourceSummary** is inspectable origin text and defaults to "coefficient-backed".
- **ValidationMode** is transient scalar text "fast" or "strict", case-insensitively, with default "fast". For a raw public **pdbase** construction in fast mode, repeated coefficient structure is checked only in the first supplied physical cell; malformed coefficient counts, payloads, or rate-row layouts in later supplied cells can therefore remain undetected. Strict mode audits every supplied cell. Grid, metadata, and options are validated globally in both modes, and no mode property is stored. The public **pdmatrix** and **pdvar** constructors normalize their source data before applying their class-specific guarantees; their fast-mode contract must not be inferred from this developer-facing raw-storage exception.

Examples and output

This backend example constructs a two-parameter parent object. The object has two physical cells because the first parameter has two intervals and the second parameter has one interval.

```
obj = pdbase({[0 1 2], [10 20]}, [2 2], 1);
objectClass = class(obj)
matrixSize = obj.MatrixSize
```

```

degree = obj.Degree
cellCount = obj.ncell()
coefficientCount = obj.ncoeff()
parameterCount = obj.npar()
objectSize = size(obj)

```

```

objectClass = 'pdbase'
matrixSize = 1×2 double
             2     2

```

```

degree = 1
cellCount = 2
coefficientCount = 4
parameterCount = 2
objectSize = 1×2 double
             2     2

```

The printed `ncoeff` value is $(\text{degree} + 1)^{\text{parameterCount}} = 4$, one local Bernstein coefficient for each tensor-product label in the two parameter directions.

Remarks

- Malformed grids and node vectors raise `pdbase:InvalidGrid` or `pdbase:InvalidGridVector`.
- Invalid matrix sizes, degrees, rate bounds, or validation modes raise `pdbase:InvalidMatrixSize`, `pdbase:InvalidDegree`, `pdbase:InvalidRateBounds`, or `pdbase:InvalidValidationMode`.
- The nested tree must contain one leaf per physical cell and $(m + 1)^\ell$ coefficient columns per leaf. Violations raise `pdbase:InvalidLocalValues` or `pdbase:InvalidCoefficientCell`.
- Payload matrices must have the declared size and be finite, real, and numeric or supported real YALMIP expressions. Malformed payloads raise `pdbase:InvalidCoefficientPayload`.
- Rate-affine payloads and `HasRateDependence=true` require compatible nonempty `RateBounds`.

3.3 Inspect Grid and Storage Properties

Every `pdbase` property below is publicly readable through dot access but has private set access. Users inspect `obj.Degree` or `obj.GridInfo.Vectors`; construction and package algebra create new objects instead of assigning these properties.

Property	Meaning
GridInfo	Grid vectors, node counts, and grid bounds.
MatrixSize	Size of each matrix coefficient or matrix expression.
Degree	Local Bernstein degree stored in every physical cell.
LocalValues	Nested cell tree indexed by physical-cell subscript; each leaf stores coefficient cells.
IsContinuous	Metadata flag; subclasses are responsible for boundary sharing if continuity is true.
ContainsDecision	True when coefficient payloads include YALMIP decision variables.
HasRateDependence	True when the object carries rate metadata or rate-vertex rows.
RateBounds	Parameter-rate lower and upper bounds used by <code>rhodiff</code> and <code>pdlmi</code> .
SourceSummary	Short origin label such as <code>coefficient-backed</code> , <code>decision</code> , or <code>derivative</code> .

```
obj = pdbase({[0 1 2], [10 20]}, [2 3], 1);
degree = obj.Degree
matrixSize = obj.MatrixSize
firstGrid = obj.GridInfo.Vectors{1}
source = obj.SourceSummary
```

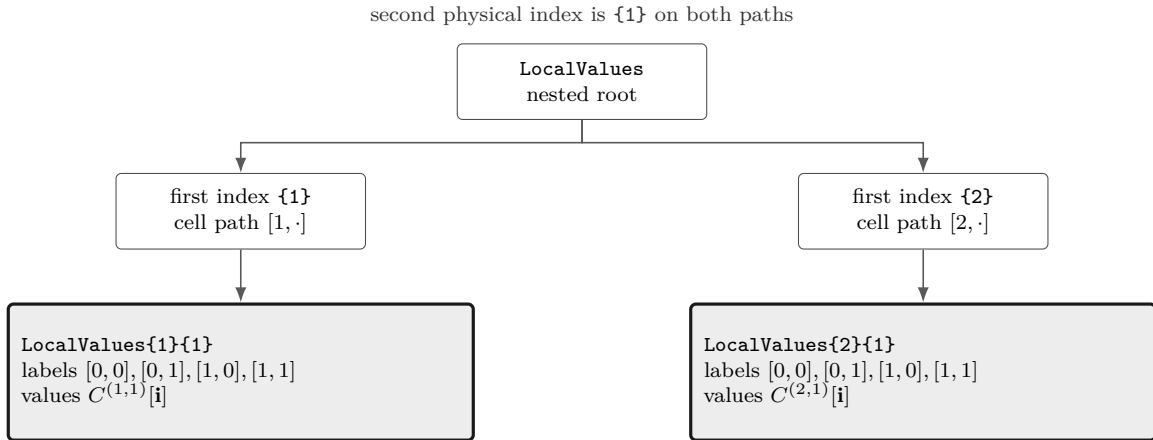


Figure 3.1: Nested cell-local storage for a two-parameter, degree-one object on $\{[0 \ 1 \ 2], [10 \ 20]\}$. There are two physical cells, $[1, 1]$ and $[2, 1]$. Each leaf is a flat local coefficient cell in `1bls` order: $[0, 0], [0, 1], [1, 0], [1, 1]$.

The nesting follows physical-cell subscripts, not the Bernstein labels. Start with `LocalValues{i1}`, then take one cell index per remaining parameter; this selects physical cell $[i_1, i_2, \dots, i_\ell]$. Only after that selection does the leaf's flat cell array enumerate the $(m+1)^\ell$ local Bernstein coefficients. Thus `coeffs(obj, [2 1])` returns the lower leaf in Fig 3.1, while `1bls(obj)` explains the order inside

that leaf.

3.4 Inspect cells, lbls, and coeffs

Syntax

```
subs = cells(obj)
subs = obj.cells()

labels = lbls(obj)
labels = obj.lbls()

c = coeffs(obj, cellSubs)
c = obj.coeffs(cellSubs)
```

Arguments

- `obj` is a `pdbase` object or a `pdmatrix`/`pdvar` subclass instance using the shared cell-local storage convention.
- `cellSubs` selects one physical cell for `coeffs`. Supply a numeric row vector such as `[2 1]` or a cell array of scalar subscripts such as `{2,1}`.
- `cells` returns physical hypercube subscripts in the shared traversal order;
- `lbls` returns local Bernstein labels in the shared coefficient order.
- `c` is the flat coefficient cell array for the selected physical cell, ordered consistently with `lbls(obj)`. Rate-dependent leaves return one row per active rate vertex in the stored row order.

Remarks

`coeffs` requires one positive integer physical-cell index per parameter, within the corresponding cell count. Invalid numeric or cell-array selectors raise `pdbase:InvalidCellSubs`.

Examples and output

The label order is shared by `pdmatrix`, `pdvar`, and `pdlmi`. In this two-parameter degree-one object, each physical cell stores four coefficient entries with labels `[0,0]`, `[0,1]`, `[1,0]`, and `[1,1]`.

```
obj = pdbase({[0 1 2], [10 20]}, [1 1], 1);
labels = lbls(obj)
physicalCells = cells(obj)
c = coeffs(obj, [2 1]);
coefficientCount = numel(c)
```

```

labels = 4×2 double
    0     0
    0     1
    1     0
    1     1

physicalCells = 2×2 double
    1     1
    2     1

coefficientCount = 4

```

3.5 evaluate Stored Coefficients

Syntax

```

val = evaluate(obj, point)
val = obj.evaluate(point)

```

Arguments

point is a finite real scalar for one parameter or a finite real vector with one coordinate per parameter, inside the physical grid bounds. The method locates one physical cell using right-cell ownership at interior grid nodes, converts the point to local coordinates, and reconstructs the tensor-product Bernstein polynomial. One ordinary coefficient row returns one numeric matrix or symbolic YALMIP expression with `size(obj)`; rate-row storage returns a 1-by- N cell array in stored row order.

Examples and output

```

obj = pdbase([0 2]), [1 1], 2, {[1, 3, 9]});
valueAtOne = obj.evaluate(1)
rateObj = pdbase([0 2]), [1 1], 1, ...
    {[1, 3; 10, 14]}, HasRateDependence=true, ...
    RateBounds=[-1 1]);
rateRows = cell2mat(rateObj.evaluate(0.5))

```

```

valueAtOne = 4
rateRows = 1×2 double
    1.5000    11.0000

```

Remarks

Malformed points raise `class(obj) + ":InvalidPoint"` and out-of-bounds points raise `class(obj) + ":PointOutOfBounds"`. `pdmat` overrides the function-backed path so exact stored handles remain exact, while coefficient-backed `pdmat` and all `pdvar` data use this row-aware backend reconstruction. Evaluation does not change the object, select a certificate, or assign YALMIP variables.

3.6 Differentiate Stored Coefficients with rhodiff

Syntax

```
D = rhodiff(A)
D = rhodiff(A, rateBounds)
```

Arguments

rhodiff differentiates coefficient-backed storage cell by cell and returns the same dynamic class as **A**. The no-argument form uses **A.RateBounds**; if those bounds are empty, explicit **rateBounds** is required. Explicit bounds supply missing metadata, but when **A.RateBounds** is already stored they must match it exactly. For one parameter, an input of degree $m > 0$ returns degree $m - 1$, while degree zero remains degree zero. For more than one parameter, each partial derivative is elevated back to the common tensor degree m , so **D.Degree**= m . Each physical-cell leaf has one row per rate-box vertex, in the order produced by applying **helper.combRows** to the parameter-wise bound rows. The result has **IsContinuous**=**false**, **HasRateDependence**=**true**, and derivative source metadata.

Examples and output

```
A = pdbase({[0 2]}, [1 1], 1, {[1, 5]}, ...
    RateBounds=[-2 3]);
D = rhodiff(A);
derivativeClass = class(D)
derivativeDegree = D.Degree
derivativeRows = cell2mat(D.coeffs(1))
```

```
derivativeClass = 'pdbase'
derivativeDegree = 0
derivativeRows = 2×1 double
    -4
     6
```

Remarks

- Function-only storage raises **pdbase:FunctionOnlyDiff**; an object that already has explicit rate rows raises **pdbase:InvalidDiff**.
- Missing, malformed, or inconsistent bounds raise **pdbase:MissingRateBounds**, **pdbase:InvalidRateBounds**, or **pdbase:RateBoundsMismatch**, respectively. Repeated differentiation is rejected because the first derivative already contains rate rows.

See also. Class-specific **pdmat** and **pdvar** differentiation is documented in sections [4.6](#) and [5.5](#).

3.7 Inspect Counts and Shape

Syntax

```
n = ncell(obj)
n = obj.ncell()

n = ncoeff(obj)
n = obj.ncoeff()

n = npar(obj)
n = obj.npar()

sz = size(obj)
d = size(obj, dim)
[m, n] = size(obj)
n = height(obj)
n = width(obj)
n = length(obj)
n = ndims(obj)
n = numel(obj)
idx = end(obj, k, n)
n = numArgumentsFromSubscript(obj, S, ctx)
```

Arguments

- `obj` is a `pdbase` object or subclass instance.
- `dim` is a positive integer matrix dimension for `size`; dimensions after row and column are singleton, and multiple outputs split the stored two-dimensional shape with trailing outputs equal to one.
- `ncell` returns $\prod_s (k_s - 1)$, the number of physical hypercube cells.
- `ncoeff` returns $(m + 1)^\ell$, the number of local Bernstein coefficients stored per physical cell.
- `npar` returns the parameter dimension ℓ .
- `size`, `height`, `width`, `length`, `ndims`, and one-argument `numel` report the matrix payload: respectively its two-element shape, row count, column count, largest dimension, the value two, and the product of the two dimensions.
- `end` reports the payload boundary used by inherited two-index syntax; one-index use returns the product of the dimensions, dimensions after two return one.
- `numArgumentsFromSubscript` always returns one so dot access and subclass indexing do not create comma-separated expansion even though `numel` follows payload semantics.

Examples and output

For a two-parameter ($\ell = 2$) degree-one backend object, the first grid has two physical intervals and the second has one. The matrix payload is 2-by-3, independently of the grid shape. Thus $n_{\text{cell}} = (3 - 1)(2 - 1) = 2$ and each cell stores $n_{\text{coeff}} = (1 + 1)^2 = 4$ entries, ordered by the explicit labels $[0, 0]$, $[0, 1]$, $[1, 0]$, $[1, 1]$.

```
obj = pdbase({[0 1 2], [10 20]}, [2 3], 1);
physicalCellCount = obj.ncell()
coefficientCount = obj.ncoeff()
parameterCount = obj.npar()
matrixSize = size(obj)
rowCount = size(obj, 1)
columnCount = size(obj, 2)
thirdDimension = size(obj, 3)
rowCountByName = height(obj)
columnCountByName = width(obj)
longestDimension = length(obj)
elementCount = numel(obj)
dimensionCount = ndims(obj)
```

```
physicalCellCount = 2
coefficientCount = 4
parameterCount = 2
matrixSize = 1×2 double
           2     3
```

```
rowCount = 2
columnCount = 3
thirdDimension = 1
rowCountByName = 2
columnCountByName = 3
longestDimension = 3
elementCount = 6
dimensionCount = 2
```

Remarks

`pdbase` is a scalar value container whose overloaded `numel(obj)` describes the payload rather than a MATLAB object array. MATLAB may call `numel` with subscripts while resolving indexing; that internal form returns one. Direct `pdbase` does not expose matrix `subsref/subsasgn`; `end` and `numArgumentsFromSubscript` support the derived `pdmatrix`/`pdvar` implementations documented in sections [4.11](#) and [5.9](#).

3.8 Apply Shared Matrix Operations

The methods in this section are implemented once by `pdbase`. Each operation maps the same MATLAB matrix transformation over every local coefficient, every active rate row, and both terms of legacy rate-affine payloads while preserving the physical grid, degree, cell/label order, and metadata. A direct backend input returns a direct `pdbase`; the `pdmat` and `pdvar` reconstruction hooks retain their respective classes and class-specific semantics. Function-only `pdmat` data have no Bernstein coefficients to map and raise `pdmat:FunctionOnlyAlgebra`.

3.8.1 Unary Signs, Transpose, Vectorization, Diagonals, And Reshape

Syntax

```
B = +A
B = -A
T = transpose(A)
H = ctranspose(A)
T = A.'
H = A'
v = vec(A)
d = diag(A)
d = diag(A, k)
D = diag(v)
D = diag(v, k)
t = trace(A)
R = reshape(A, m, n)
R = reshape(A, [m n])
S = squeeze(A)
```

Description

Unary plus returns the same value; unary minus, transpose, conjugate transpose, vectorization, diagonal extraction/construction, trace, reshape, and squeeze map coefficient payloads without changing the parameter representation. `vec` uses MATLAB column-major order. `trace` returns a 1-by-1 object and sums the main diagonal even for a row or column vector. `diag` accepts a finite integer offset; for a matrix, the selected diagonal must be nonempty because the package cannot store an empty payload. `reshape` supports exactly two positive dimensions, permits at most one inferred `[]` dimension, and preserves the element count. `squeeze` is the identity because the storage contract remains two-dimensional.

Examples and output

```
A = pdbase({[0 1]}, [2 2], 1, ...
    {[[1 2; 3 4], [10 20; 30 40]]});
T = A.';
v = vec(A);
d = diag(A);
t = trace(A);
R = reshape(A, 1, 4);
transposeCoefficients = T.coeffs(1);
vectorCoefficients = v.coeffs(1);
diagonalCoefficients = d.coeffs(1);
traceCoefficients = t.coeffs(1);
transposeSize = size(T)
transposeFirst = transposeCoefficients{1}
vectorFirst = vectorCoefficients{1}
diagonalFirst = diagonalCoefficients{1}
traceFirst = traceCoefficients{1}
reshapeSize = size(R)
```

```
transposeSize = 1×2 double
    2    2
transposeFirst = 2×2 double
    1    3
    2    4
vectorFirst = 4×1 double
    1
    3
    2
    4
diagonalFirst = 2×1 double
    1
    4
traceFirst = 5
reshapeSize = 1×2 double
    1    4
```

Remarks

Malformed or empty diagonal selections raise `class(A) + ":InvalidDiag"`. Unsupported reshape forms, multiple inferred dimensions, nonpositive dimensions, or an element-count mismatch raise `class(A) + ":InvalidReshape"`. These identifiers therefore appear as `pdbase:*`, `pdmat:*`, or `pdvar:*` according to the public input class.

3.8.2 Triangular Parts And Reductions

Syntax

```
L = tril(A)
L = tril(A, k)
U = triu(A)
U = triu(A, k)
S = sum(A)
S = sum(A, dim)
S = sum(A, vecdim)
S = sum(A, "all")
M = mean(A)
M = mean(A, dim)
M = mean(A, vecdim)
M = mean(A, "all")
C = cumsum(A)
C = cumsum(A, dim)
C = cumsum(A, direction)
C = cumsum(A, dim, direction)
```

Description

tril and **triu** retain entries on and below or above the finite integer offset **k**. **sum** and **mean** accept one nonempty vector of unique positive integer dimensions or the case-insensitive text "all"; dimensions above two are singleton no-ops. Without a dimension, each follows the first nonsingleton matrix dimension. **cumsum** accepts an optional positive integer scalar dimension and an optional case-insensitive "forward" or "reverse" direction. A direction without a dimension uses the first nonsingleton matrix dimension; a dimension above two leaves every matrix payload unchanged.

Examples and output

```
A = pdbase({[0 1]}, [2 2], 1, ...
    {[1 2; 3 4], [10 20; 30 40]});
L = tril(A);
S = sum(A, [1 2]);
M = mean(A, "all");
C = cumsum(A, 2, "reverse");
lowerCoefficients = L.coeffs(1);
sumCoefficients = S.coeffs(1);
meanCoefficients = M.coeffs(1);
cumsumCoefficients = C.coeffs(1);
lowerFirst = lowerCoefficients{1}
sumFirst = sumCoefficients{1}
meanFirst = meanCoefficients{1}
reverseCumsumFirst = cumsumCoefficients{1}
```

```
lowerFirst = 2×2 double
    1     0
    3     4
sumFirst = 10
meanFirst = 2.5000
```

```
reverseCumsumFirst = 2×2 double
    3    2
    7    4
```

Remarks

Invalid triangular offsets raise `class(A) + ":InvalidTriangularPart"`. Malformed reduction forms raise `class(A) + ":InvalidSum"`, `class(A) + ":InvalidMean"`, or `class(A) + ":InvalidCumsum"`. Missing-value, native-output, or other MATLAB reduction flags are not implemented.

3.8.3 Reordering, Rotation, And Repetition

Syntax

```
F = flip(A)
F = flip(A, dim)
F = flipud(A)
F = fliplr(A)
R = rot90(A)
R = rot90(A, k)
Q = repmat(A, m)
Q = repmat(A, m, n)
Q = repmat(A, [m n])
```

Description

`flip`, `flipud`, and `fliplr` reverse entries inside each coefficient matrix and never reverse a physical parameter grid. `flip(A)` follows MATLAB's first-nonsingleton behavior; `flip(A,dim)` requires a positive integer scalar dimension. `rot90` applies a finite integer number of counterclockwise quarter-turns, reducing the count modulo four. `repmat` supports a positive scalar interpreted as `[m m]`, two positive counts, or a two-element count vector; only the payload shape changes.

Examples and output

```
A = pdbase({[0 1]}, [2 2], 1, ...
    {[[1 2; 3 4], [10 20; 30 40]]});
F = flipud(A);
R = rot90(A);
Q = repmat(A, [1 2]);
flippedCoefficients = F.coeffs(1);
rotatedCoefficients = R.coeffs(1);
flippedFirst = flippedCoefficients{1}
rotatedFirst = rotatedCoefficients{1}
repeatedSize = size(Q)
```

```
flippedFirst = 2×2 double
     3     4
     1     2
rotatedFirst = 2×2 double
     2     4
     1     3
repeatedSize = 1×2 double
     2     4
```

Remarks

Invalid flip dimensions, rotation counts, or repetition forms raise `class(A) + ":InvalidFlip"`, `class(A) + ":InvalidRot90"`, or `class(A) + ":InvalidRepmat"`. N-dimensional repetition is outside the two-dimensional payload contract.

3.8.4 Concatenation Dispatch

Syntax

```
C = horzcat(A, B, ...)
C = [A, B, ...]
C = vertcat(A, B, ...)
C = [A; B; ...]
C = cat(dim, A, B, ...)
```

Description

`pdbase` owns the inherited `horzcat`/`vertcat` entry points, which dispatch compatible derived objects to the class-specific `cat` implementation. `pdmat` and `pdvar` therefore retain coefficient-wise concatenation, grid/degree alignment, numeric promotion, validation, and result construction as documented in sections [4.10.7](#) and [5.8.7](#). Direct `pdbase` operands are scalar backend containers rather than MATLAB object arrays and cannot be concatenated.

Examples and output

```
A = pdmat({[0 1]}, {1, 2}, Degree=1);
B = [A, A];
resultClass = class(B)
resultSize = size(B)
```

```
resultClass = 'pdmat'
resultSize = 1×2 double
    1     2
```

Remarks

Any direct `pdbase` operand in `cat`, `horzcat`, or `vertcat` raises `pdbase:Unsupported Concatenation`. Derived classes support only matrix dimensions one and two and report their own compatibility errors. `blkdiag` remains class-specific to `pdmat` and `pdvar`; it is not a `pdbase` implementation.

3.9 Transform Bernstein Storage

The following entries separate whole-object elevation from the protected helpers used to align degrees, convolve coefficients, and refine grids. They change coefficient representations or temporary storage trees; they do not change the meanings of `ncell`, `ncoeff`, `npar`, or `size` described above.

3.9.1 elevate

Syntax

```
out = obj.elevate(degreeIncrement)
```

Description

`degreeIncrement` is a finite nonnegative integer scalar added uniformly in every parameter direction. `out` is the same dynamic class as `obj`; it contains elevated coefficient evidence at `obj.Degree + degreeIncrement` while preserving the represented polynomial, physical cells, metadata, YALMIP decision variables, rate-row order, and subclass properties. The source object is unchanged, and elevation introduces no new decision freedom.

Examples and output

```
A = pdmat({[0 1]}, {1, 3}, Degree=1);
B = A.elevate(1);
sourceDegree = A.Degree
elevatedDegree = B.Degree
elevatedCoefficients = cell2mat(B.coeffs(1))
sameValue = [A.evaluate(0.25), B.evaluate(0.25)]
```

```
sourceDegree = 1
elevatedDegree = 2
elevatedCoefficients = 1×3 double
    1    2    3
sameValue = 1×2 double
    1.5000    1.5000
```

Remarks

Invalid increments raise `pdbase:InvalidDegreeIncrement`. Function-only `pdmat` data have no coefficient evidence and raise `pdbase:MissingCoefficientEvidence`; constructing the function with a verified explicit `Degree` supplies the needed evidence. `elevate` returns an object, whereas `elevVals` returns only an elevated `LocalValues`-shaped tree.

3.9.2 elevVals

Syntax

```
vals = obj.elevVals(degreeIncrement)
```

Arguments

- `degreeIncrement` is a finite nonnegative integer scalar. It is added to `obj.Degree` in every parameter direction; zero returns coefficient evidence at the current degree.

Description

`vals` is a nested `LocalValues`-shaped cell tree. Each physical cell contains coefficients of degree `obj.Degree + degreeIncrement`, with the represented polynomial, physical-cell tree, and any rate-row ordering preserved. The method does not modify `obj` or its stored coefficient evidence.

Examples and output

```
A = pdmat({[0 1]}, {1, 3}, Degree=1);
elevatedTree = A.elevVals(1);
elevated = cell2mat(elevatedTree{1})
```

```
elevated = 1×3 double
           1   2   3
```

Remarks

An invalid increment raises `pdbase:InvalidDegreeIncrement`. A function-only `pdmat` has no stored Bernstein coefficient evidence, so this method raises `pdbase:MissingCoefficientEvidence`.

Relationship to backend and public algebra. Unlike protected `bernElev`, which elevates one flat cell coefficient family for internal alignment, `elevVals` is a public whole-object query returning elevated `LocalValues`. Public coefficient algebra uses `bernElev` internally when compatible operands need degree alignment; it does not mutate either operand.

3.9.3 elevLocalValues

Internal/protected mechanism. This method elevates a temporary nested coefficient tree after public algebra has remapped operands to a common physical grid. It is documented as backend context, not as a supported user call form.

Syntax

```
vals = pdbase.elevLocalValues(vals, fromDeg, toDeg, grid)
```

Each leaf must be a cell array with one column per degree-`fromDeg` Bernstein label. The method elevates each rate-vertex row independently to `toDeg`; it never mixes rows. The required `grid` supplies the temporary physical-cell layout. Equal source and target degrees return the input tree unchanged. A malformed leaf raises `pdbase:InvalidCoefficientCell`. Public users encounter this behavior through grid merging, assignment, concatenation, and arithmetic rather than by calling the protected method directly.

Examples and output

```
A = pdmat({[0 1]}, @(rho) rho, Degree=1);
B = pdmat({[0 .5 1]}, @(rho) 2*rho, Degree=2);
C = A + B;
mergedGrid = C.GridInfo.Vectors{1}
resultDegree = C.Degree
cell1Coefficients = cell2mat(C.coeffs(1))
cell2Coefficients = cell2mat(C.coeffs(2))
AGrid = A.GridInfo.Vectors{1}
ADegree = A.Degree
BGrid = B.GridInfo.Vectors{1}
BDegree = B.Degree
```

```
mergedGrid = 1×3 double
            0   0.5000   1.0000
```

```
resultDegree = 2
```

```
cell1Coefficients = 1×3 double
```



```

      0      0.7500      1.5000
cell2Coefficients = 1×3 double
      1.5000      2.2500      3.0000

AGrid = 1×2 double
      0      1

ADegree = 1

BGrid = 1×3 double
      0      0.5000      1.0000

BDegree = 2

```

The example exercises the protected mechanism through public `pdmatrix` addition; users do not call protected `elevLocalValues` directly.

3.9.4 bernElev

Internal/protected mechanism. This signature documents the backend algorithm used by public algebra; it is not a supported user call form.

Syntax

```
out = pdbase.bernElev(coeffs, fromDeg, toDeg, nPar)
```

Arguments

-
- `coeffs` is a flat coefficient-cell family in Bernstein label order.
 - `fromDeg` and `toDeg` are nonnegative integer degrees, with `toDeg` $\geq$ `fromDeg`.
 - `nPar` is the positive integer parameter count.
 - `out` contains $(\text{toDeg} + 1)^{\text{nPar}}$ elevated coefficients in the same label order.

Examples and output

```

A = pdmat({[0 1]}, {1, 2}, Degree=1);
B = pdmat({[0 1]}, {1, 2, 3}, Degree=2);
C = A + B;
degree = C.Degree

```

```
degree = 2
```

The example invokes the protected helper through public addition; users do not call `bernElev` directly. Invalid degree values raise `pdbase:InvalidDegree`; a target below the source degree raises `pdbase:InvalidDegreeElevation`; a coefficient count inconsistent with the source degree and parameter dimension raises `pdbase:InvalidCoefficientCell`.

3.9.5 bernProd

Internal/protected mechanism. This signature documents the backend algorithm used by public matrix multiplication; it is not a supported user call form.

Syntax

```
out = bernProd(obj, lhs, lhsDeg, rhs, rhsDeg)
```

Arguments

- **lhs** and **rhs** are flat coefficient-cell families whose matrix payload inner dimensions are compatible for multiplication.
- **lhsDeg** and **rhsDeg** are their nonnegative local Bernstein degrees.
- **out** is the binomial-normalized coefficient convolution at degree **lhsDeg** + **rhsDeg**; payload products follow MATLAB matrix multiplication rules.

For one parameter, the protected helper implements

$$C[k] = \sum_{i+j=k} \frac{\binom{m}{i}\binom{n}{j}}{\binom{m+n}{k}} A[i]B[j].$$

It enumerates ordered payload products, groups them by the anti-diagonal $i + j = k$, and only then applies the binomial normalization. For tensor labels the same rule is applied componentwise. See the public **pdmat** multiplication entry in section 4.8.2, the **pdvar** known-data boundary in section 5.7, and the full derivation in Appendix A.

Examples and output

```
A = pdmat({[0 1]}, {1, 2}, Degree=1);  
P = A * A;  
degree = P.Degree  
coefficients = cell2mat(P.coeffs(1))
```

```
degree = 2  
coefficients = 1×3 double  
              1    2    4
```

The example invokes the protected helper through public matrix multiplication; users do not call **bernProd** directly. Even for scalar data, the middle coefficient comes from two ordered contributions rather than from one sample.

3.9.6 bernTbl, mapVals, and matSubs

Internal/protected mechanisms. These object-aware utilities are owned by **pdbase**; they are not functions in the public **+helper** namespace and are not supported user call forms.

Syntax

```
T = bernTbl(obj, errId, valFcn, exprFcn, rateVerts, ...)
mapped = mapVals(vals, fcn, grid)
[rows, cols] = matSubs(subs, sz, errId)
```

Description

`bernTbl` builds the detailed and one-line inspection tables used by `pdmat.bernsteinTable` and `pdvar.bernsteinTable`. It traverses `cells`, `lbls`, and `coeffs`, accepts at most one physical-cell selector and the case-insensitive `"oneLine"` option, and preserves the caller's error namespace. `mapVals` applies a coefficient mapping to every payload in every nested physical-cell leaf while preserving tree and label order; the owning class still validates matrix, symbolic, and rate semantics. `matSubs` normalizes exactly two row/column selectors, accepting `:`, finite positive integer vectors, or matching logical vectors, and rejects linear indexing, empty/out-of-range numeric indices, mismatched logical selectors, and array growth.

Examples and output

```
A = pdmat({[0 1]}, {[1 2; 3 4]}, [10 20; 30 40]), Degree=1);
T = A.bernsteinTable("oneLine");
column = A(:, 2);
tableHeight = height(T)
columnSize = size(column)
```

```
tableHeight = 1
columnSize = 1×2 double
           2     1
```

Remarks

Users reach `bernTbl` through `bernsteinTable`, `mapVals` through inherited unary operations, and `matSubs` through `pdmat/pdvar` indexing and assignment. Invalid table selectors and rate-row mismatches raise the caller-owned `errId`; indexing uses the class-specific identifier supplied by `subsref` or `subsasgn`. The six genuine cross-class helpers remain documented in [Chapter 7](#).

3.9.7 mergeGrid

Internal/protected mechanism. This signature documents the backend grid-refinement algorithm used by public algebra; it is not a supported user call form.

Syntax

```
grid = obj.mergeGrid(errId, other1, other2, ...)
```

Arguments

- **errId** is the error identifier used when parameter counts or physical bounds are incompatible.
- **other1, other2, ...** are coefficient-backed operands, including function-backed operands constructed with an explicit **Degree**. They must have equal parameter counts and matching first and last nodes on every parameter axis.
- **grid** has the sorted union of the interior nodes. Public algebra re-expresses operands on these cells before degree elevation, coefficient-wise addition, or product convolution.

This protected helper is invoked by public algebra methods rather than called directly by users.

Examples and output

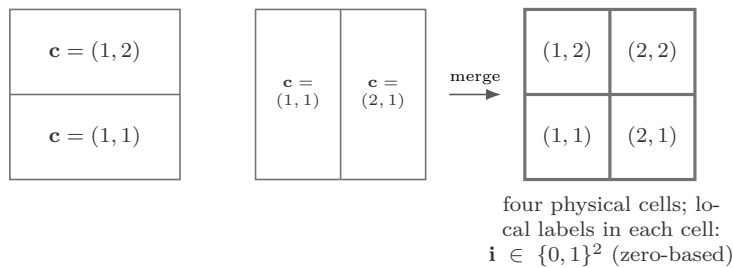
```
gridA = {[0 1], [0 0.5 1]};
gridB = {[0 0.5 1], [0 1]};
A = pdmat(gridA, @(rho1, rho2) rho1 + 2*rho2, Degree=1);
B = pdmat(gridB, @(rho1, rho2) 2*rho1 - rho2, Degree=1);
C = A + B;
rho1Grid = C.GridInfo.Vectors{1}
rho2Grid = C.GridInfo.Vectors{2}
```

```
rho1Grid = 1×3 double
    0    0.5000    1.0000

rho2Grid = 1×3 double
    0    0.5000    1.0000
```

The example invokes the protected helper through public addition; users do not call `mergeGrid` directly. Here $\ell = 2$. The physical-cell index $\mathbf{c} = (c_1, c_2)$ is one-based, while the degree-one local Bernstein label $\mathbf{i} = (i_1, i_2) \in \{0, 1\}^2$ is zero-based inside every cell.

$$G_A = \{[0, 1], [0, 0.5, 1]\} \quad G_B = \{[0, 0.5, 1], [0, 1]\} \quad G = \{[0, 0.5, 1], [0, 0.5, 1]\}$$



Each axis uses the sorted union of its input nodes. The result is the unique minimal same-bound tensor grid containing both inputs.

3.10 Boundaries and Related APIs

pdmat, pdvar, cells, coeffs, lbls, evaluate, ncell, ncoeff, npar, size, height, width, length, ndims, numel, end, numArgumentsFromSubscript, unary +/-, transpose, ctranspose, trace, vec, diag, reshape, squeeze, tril, triu, sum, mean, cumsum, flip, flipud, fliplr, rot90, repmat, cat, horzcat, vertcat, elevate, elevVals, elevLocalValues, bernElev, bernProd, bernTbl, mapVals, matSubs, mergeGrid.

Chapter 4

pdmat: Known Parameter-Dependent Matrices

`pdmat` stores known finite real matrix data on a tensor grid. Objects can be coefficient-backed, function-only, or function-backed with explicit Bernstein coefficient evidence. Coefficient-backed objects participate in algebra. Function-only objects evaluate exactly through the stored handle, but do not expose coefficient evidence for `bernsteinTable` or coefficient algebra. Shared matrix-shape, unary, structural, reduction, reordering, and repetition methods are implemented centrally by `pdbase` and remain complete public `pdmat` behavior; this chapter documents their known-data syntax, numeric examples, validations, and boundaries.

For coefficient-backed data, one storage leaf is the local expansion $A^{(c)}(\phi_c(\rho)) = \sum_i B_i^m(\alpha) A^{(c)}[i]$. Construction chooses the source of those coefficients; `evaluate` reconstructs the matrix value; `elevate` changes the coefficient representation without changing the polynomial; and ordered multiplication applies the binomial-weighted convolution in section A.8. A function-backed object keeps exact handle evaluation separate from its finite coefficient evidence, while a function-only object has no coefficient algebra to certify.

4.1 API Map

Task	Entry
Construct data	<code>pdmat</code>
Evaluate	<code>evaluate</code>
Inspect/display	<code>bernsteinTable</code> , <code>disp</code> , <code>display</code>
Inspect/elevate backend storage	<code>cells/lbls/coeffs</code> , <code>counts</code> , <code>elevVals</code> , and <code>elevate</code>
Plot	<code>plot</code>
Differentiate	<code>rhodiff</code>
Arithmetic	<code>+</code> , <code>-</code> , unary <code>+/-</code> , <code>*</code>
Matrix/index methods	shape and structural methods, indexing and assignment
Compare/certify	logical <code>==</code> and known-data <code><=/>=</code>

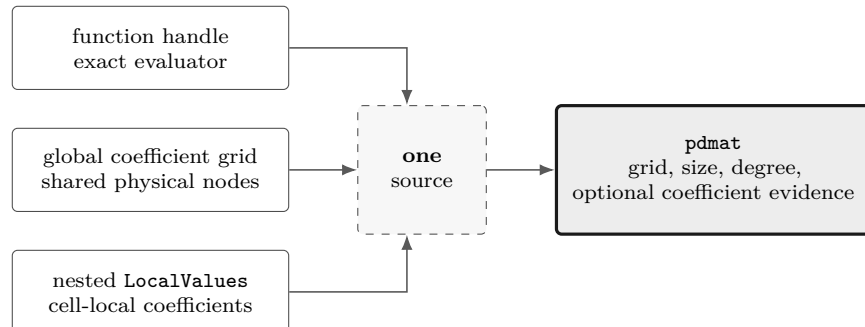
4.2 Construct pdmat Objects

Syntax

```
A = pdmat(gridVector, source)
A = pdmat(gridVector, source, Degree=m)
A = pdmat(gridVector, source, RateBounds=rb)
A = pdmat(gridVectors, source)
A = pdmat(gridVectors, source, Degree=m, RateBounds=rb)
A = pdmat(..., ValidationMode=mode)
```

Arguments

- **gridVector** is a numeric vector used as a one-parameter grid shorthand, such as `[0 1 2]`.
- **gridVectors** is a cell vector with one numeric grid vector per parameter, such as `{[0 1], [10 20]}`.
- **source** may be:
 1. A function handle, e.g., `@(rho) [rho rho^2]` is a one-parameter function with two matrix entries.
 2. A global cell grid of numeric Bernstein coefficients, e.g., `{[1 2], [3 4]}` is a two-cell degree-one grid with two 1×1 coefficients per cell.
 3. A explicit nested **LocalValues**, e.g., `{1, 2, 3}` is scalar degree-two data on one cell.
- **Degree=m** sets the local Bernstein degree. For function handles, omitting **Degree** creates a function-only object; passing a value such as **Degree=2** validates and stores Bernstein coefficient evidence while retaining the exact function handle for **evaluate**.
- **RateBounds=rb** is an ℓ -by-2 finite real matrix of lower and upper scheduling-rate bounds and is accepted with every source family. With an ordinary one-row source it is metadata used by **rhodiff**. An explicit nested leaf may instead contain either one ordinary row or all 2^ℓ rate-vertex rows, uniformly across every physical cell; their deterministic order is induced by applying helper `combRows` to `num2cell(rb,2).'`.
- **ValidationMode=mode** accepts scalar text `"fast"` or `"strict"`, case-insensitively, and defaults to `"fast"`. It controls only repeated post-normalization structural checks and is not stored. Public source validation remains complete in both modes.



Examples and output

The scalar-grid shorthand is the shortest coefficient-backed form. The three numeric cells below are the global Bernstein coefficients for two physical cells with degree one.

```
A = pdmat([0 1 2], {10, 20, 30}, Degree=1);
disp(A)
matrixSize = size(A)
degree = A.Degree
sourceSummary = A.SourceSummary
```

```
pdmat [1 1] over 1-D grid, degree 1, source coefficient-backed, rate rows false
```

```
matrixSize = 1×2 double
    1     1
```

```
degree = 1
sourceSummary = "coefficient-backed"
```

For tensor grids, pass one grid vector per parameter. The source cell array matches the global coefficient grid implied by the two parameter directions.

```
B = pdmat({[0 1], [10 20]}, {1 2; 3 4}, Degree=1);
disp(B)
secondGridVector = B.GridInfo.Vectors{2}
labels = B.lbls()
```

```
pdmat [1 1] over 2-D grid, degree 1, source coefficient-backed, rate rows false
```

```
secondGridVector = 1×2 double
    10    20
```

```
labels = 4×2 double
    0     0
    0     1
    1     0
    1     1
```

Use explicit nested `LocalValues` when each physical cell must be written directly. Here each cell stores two 1×2 row-vector coefficients.

```
localVals = {[[1 0], [0 1]], {[0 1], [0 2]}};
C = pdmat({[0 1 2]}, localVals, Degree=1);
disp(C)
matrixSize = size(C)
secondCellCoefficients = cell2mat(C.coeffs(2))
```

```
pdmat [1 2] over 1-D grid, degree 1, source coefficient-backed, rate rows false
```

```
matrixSize = 1×2 double
    1     2
```

```
secondCellCoefficients = 1×4 double
    0     1     0     2
```

When `source` is a function handle and `Degree` is omitted, the object keeps the exact evaluator but

does not claim coefficient evidence.

```
F = pdmat({[0 1]}, @(rho) [rho rho^2]);
disp(F)
sourceSummary = F.SourceSummary
evaluatedValue = F.evaluate(0.5)
```

pdmat [1 2] over 1-D grid, degree 1, source function, rate rows false

```
sourceSummary = "function"
evaluatedValue = 1×2 double
    0.5000    0.2500
```

Add **Degree** for a polynomial function when later coefficient inspection or algebra should use Bernstein evidence. The exact function handle is still used for point evaluation.

```
G = pdmat({[0 1]}, @(rho) rho.^2, Degree=2);
disp(G)
sourceSummary = G.SourceSummary
firstCellCoefficients = cell2mat(G.coeffs(1))
```

pdmat [1 1] over 1-D grid, degree 2, source function-bernstein, rate rows false

```
sourceSummary = "function-bernstein"
firstCellCoefficients = 1×3 double
    0    0    1
```

These forms all create finite known-data objects, but only the coefficient- backed and function-Bernstein forms can participate in coefficient algebra.

Explicit rate rows attach one complete coefficient row to each vertex of the rate box. This one-parameter example has two rate vertices and therefore two rows in its only physical-cell leaf.

```
A = pdmat([0 1], {[1, 3; 10, 14]}, ...
    Degree=1, RateBounds=[-1 2]);
disp("rateBounds =")
disp(A.RateBounds)
disp("rateRowValues =")
disp(cell2mat(A.coeffs(1)))
```

```
rateBounds =
    -1     2

rateRowValues =
     1     3
    10    14
```

Remarks

`FunctionHandle` retains the supplied function handle for function-only and function-Bernstein sources; it is empty for cell-backed sources. Its set access is private.

- Constructor option errors use `pdmatrix:InvalidOptions`, `pdmatrix:UnsupportedOption`, or `pdmatrix:UnknownOption`; grid errors use `pdmatrix:InvalidGrid` or `pdmatrix:InvalidGridVector`.
- Source and function validation uses `pdmatrix:InvalidSource`, `pdmatrix:InvalidFunctionHandle`, `pdmatrix:InvalidFunctionOutput`, or `pdmatrix:InvalidData`.
- Degree, rate, and storage failures use `pdmatrix:InvalidDegree`, `pdmatrix:InvalidRateBounds`, `pdmatrix:InvalidLocalValues`, `pdmatrix:InvalidCoefficientCell`, or `pdmatrix:InvalidCoefficientPayload`.
- Invalid validation modes raise `pdmatrix:InvalidValidationMode`. With explicit `Degree`, nonrepresentable functions raise `pdmatrix:NonBernsteinPolynomial`; invalid validation probes raise `pdmatrix:InvalidFunctionValue`.
- Mismatched shared faces are retained as discontinuous local data and emit warning `pdmatrix:DiscontinuousLocalValues`.

4.3 Inspect Object Properties

`pdmatrix` adds the read-only `FunctionHandle` property to the inherited `pdbase` properties in section 3.3. All use private set access. `FunctionHandle` contains the supplied handle for function and function-Bernstein sources and is empty for cell-backed sources. The inherited properties describe the grid, matrix size, Bernstein degree, cell-local evidence, continuity, and origin.

```
A = pdmatrix([0 1]), @(rho) rho.^2, Degree=2);
degree = A.Degree
source = A.SourceSummary
hasFunction = ~isempty(A.FunctionHandle)
firstCell = A.LocalValues{1}
```

Assigning `A.Degree`, `A.LocalValues`, or `A.FunctionHandle` is not a supported update operation; construct a new `pdmatrix` instead.

4.4 Inspect And Elevate Inherited Bernstein Storage

Syntax

```
cellSubs = cells(A)
labels = lbls(A)
coefficients = coeffs(A, cellSubs)
n = ncell(A)
n = ncoeff(A)
n = npar(A)
```

```
vals = A.elevVals(degreeIncrement)
B = A.elevate(degreeIncrement)
```

Description

The storage queries use the shared `pdbase` cell and label order. `cells` returns one one-based physical-cell subscript per row; `lbls` returns zero-based tensor Bernstein labels; `coeffs` returns the selected flat coefficient leaf. `ncell`, `ncoeff`, and `npar` count physical cells, coefficients per cell, and parameter directions. `elevVals` returns an elevated nested coefficient tree without modifying `A`; `elevate` returns a new `pdmat` with the same represented known function and dynamic-class metadata at the higher degree.

Examples and output

```
A = pdmat([0 1]), {1, 3}, Degree=1);
physicalCells = A.cells()
labels = A.lbls()
sourceCoefficients = cell2mat(A.coeffs(1))
counts = [A.ncell(), A.ncoeff(), A.npar()]
elevatedValues = A.elevVals(1);
elevatedLeaf = elevatedValues{1};
B = A.elevate(1);
elevatedTreeCoefficients = cell2mat(elevatedLeaf)
elevatedObjectCoefficients = cell2mat(B.coeffs(1))
degrees = [A.Degree, B.Degree]
```

```
physicalCells = 1
labels = 2×1 double
    0
    1
sourceCoefficients = 1×2 double
    1    3
counts = 1×3 double
    1    2    1
elevatedTreeCoefficients = 1×3 double
    1    2    3
elevatedObjectCoefficients = 1×3 double
    1    2    3
degrees = 1×2 double
    1    2
```

Remarks

Invalid physical-cell selectors raise `pdbase:InvalidCellSubs`. Invalid increments raise `pdbase:InvalidDegreeIncrement`. Function-only data support exact `evaluate`, but their placeholder storage is not Bernstein coefficient evidence and cannot be used by `elevVals`, `elevate`, `algebra`, or `bernsteinTable`; the elevation paths raise `pdbase:MissingCoefficientEvidence`. A function constructed with an explicit verified `Degree` retains both its exact handle and Bernstein evidence.

4.5 evaluate

Syntax

```
val = evaluate(A, point)
val = A.evaluate(point)
```

Arguments

- **A** is a `pdmat` object.
- **point** is a finite real scalar for a one-parameter object, such as `0.25`, or a finite real row vector with one entry per parameter, such as `[0.25 12]`. Every entry must lie within the corresponding grid bounds.
- **val** is a numeric matrix with `size(A)`. Coefficient-backed objects use local Bernstein interpolation; function-backed objects call their stored function handle.

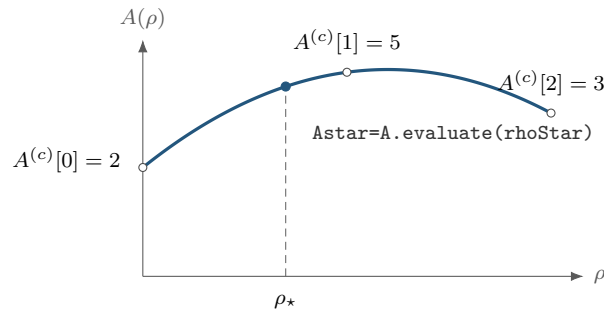
Remarks

A point with the wrong type, length, or finite-real shape raises `pdmat:InvalidPoint`; a point outside any grid bound raises `pdmat:PointOutOfBounds`. If a stored function fails or returns a nonfinite, complex, empty, or size-inconsistent matrix at the requested point, `evaluate` raises `pdmat:InvalidFunctionValue`.

For a degree-one scalar coefficient-backed object on $[\rho_1^{(1)}, \rho_1^{(2)}]$,

$$A(\rho) = (1 - \alpha)A^{(c)}[0] + \alpha A^{(c)}[1], \quad \alpha = \frac{\rho_1 - \rho_1^{(1)}}{\rho_1^{(2)} - \rho_1^{(1)}}.$$

The same local-coordinate rule is applied entrywise for matrix-valued data.



Evaluation follows the degree-two Bernstein curve with coefficients $[2, 5, 3]$; the dashed line locates the requested physical point.

Examples and output

Use the function-call form when the object is naturally one input among several MATLAB expressions.

```
A = pdmat({[0 1]}, {2, 4}, Degree=1);  
val = evaluate(A, 0.25)
```

```
val = 2.5000
```

At $\rho = 0.25$, the public local coordinate is $\alpha = 0.25$, so the interpolated value is $0.75 \cdot 2 + 0.25 \cdot 4 = 2.5$. The dot-call form is equivalent and can be convenient in command-window exploration.

```
A = pdmat({[0 1]}, {2, 4}, Degree=1);  
val = A.evaluate(0.75)
```

```
val = 3.5000
```

Function-backed objects route through the retained function handle, so non-polynomial expressions can still be evaluated exactly at a point.

```
F = pdmat({[0 pi]}, @(rho) sin(rho));  
val = F.evaluate(pi/2)
```

```
val = 1
```

4.6 Differentiate Known Data with rhodiff

Syntax

```
D = rhodiff(A)  
D = rhodiff(A, rateBounds)
```

Arguments

rhodiff differentiates coefficient-backed **pdmat** data cell by cell and returns a **pdmat** with numeric rate-vertex rows. Explicit **rateBounds** supplies missing metadata; when **A.RateBounds** is already stored, the explicit bounds must match it exactly. Without either source of bounds, the call raises **pdmat:MissingRateBounds**. For one parameter, degree $m > 0$ returns degree $m - 1$, while degree zero remains degree zero. For more than one parameter, the partial derivatives are elevated back to the common tensor degree m . The result is rate-dependent and generally discontinuous across cell faces. A function-only source raises **pdmat:FunctionOnlyDiff**; malformed or inconsistent bounds raise **pdmat:InvalidRateBounds** or **pdmat:RateBoundsMismatch**; data that already contain explicit rate rows, including a previous derivative, raise **pdmat:InvalidDiff**.

Examples and output

```
A = pdmat([0 1], {0, 2}, Degree=1, RateBounds=[-1 2]);
D = rhodiff(A);
disp("derivativeClass =")
disp(class(D))
disp("derivativeDegree =")
disp(D.Degree)
disp("derivativeRowValues =")
disp(cell2mat(D.coeffs(1)))
```

```
derivativeClass =
pdmat
derivativeDegree =
    0

derivativeRowValues =
    -2
     4
```

4.7 Inspect Coefficients and Plot Values

4.7.1 bernsteinTable

Syntax

```
T = bernsteinTable(A)
T = bernsteinTable(A, cellSubs)
T = bernsteinTable(A, "oneLine")
T = bernsteinTable(A, cellSubs, "oneLine")
```

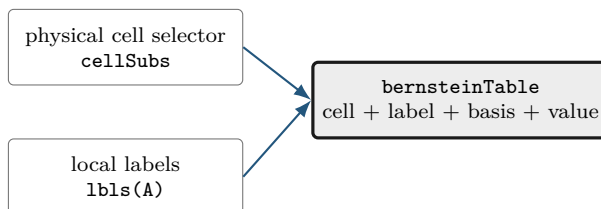
Arguments

- `cellSubs` selects one physical cell.
- `"oneLine"` returns one row per selected physical cell and rate vertex with a compact Bernstein expression.
- Without `"oneLine"`, an ordinary object uses seven metadata columns; an explicit-rate-row object additionally includes `RateVertexIndex` and `RateVertex`. Their deterministic vertex order is induced by applying `helper.combRows` to `num2cell(A.RateBounds,2)'`.

For one parameter, the `Basis` column reports

$$\binom{m}{j} (1 - \alpha)^{m-j} \alpha^j.$$

For multiple parameters, the basis is the tensor product of that expression in each local coordinate, using `alpha1`, `alpha2`, and so on. `LocalIndex` is the local Bernstein label. `CoeffSubscript` maps the local label back to the global coefficient subscript on the physical grid. `IsPhysicalNode` is true when that coefficient lies on a physical grid node.



The table reports stored coefficient evidence. It does not sample a function and does not create solver constraints.

Examples and output

```
A = pdmat({[0 0.2 1]}, {[0 1], [1 1], [1 2]}, Degree=1);
T = bernsteinTable(A, "oneLine");
disp(T)
T1 = bernsteinTable(A, 1, "oneLine");
disp(T1)
```

CellSubscript	Expression
{[1]}	"(1-alpha)*[0 1] + alpha*[1 1]"
{[2]}	"(1-alpha)*[1 1] + alpha*[1 2]"

CellSubscript	Expression
{[1]}	"(1-alpha)*[0 1] + alpha*[1 1]"

Use the full table when the metadata columns are the point of the example:

```
A = pdmat({[0 1]}, {1, 2, 3}, Degree=2);
T = bernsteinTable(A);
disp(T)
```

TermIndex	CellSubscript	CoeffSubscript	LocalIndex	Basis	IsPhysicalNode	Value
1	{[1]}	{[1]}	{[0]}	"(1-alpha)^2"	true	{[1]}
2	{[1]}	{[2]}	{[1]}	"2(1-alpha)alpha"	false	{[2]}
3	{[1]}	{[3]}	{[2]}	"alpha^2"	true	{[3]}

Remarks

A function-only object has no coefficient evidence and raises `pdmat:FunctionOnlyBernsteinTable`. Unknown text options, repeated selectors, or invalid physical-cell selectors raise `pdmat:InvalidBernsteinTableInput`.

For an explicit-rate-row object, `bernsteinTable(A, "oneLine")` returns one expression for every stored vertex rather than merging the alternatives:

```

A = pdmat([0 1], {{1, 3; 10, 14}}, ...
    Degree=1, RateBounds=[-1 2]);
T = bernsteinTable(A, "oneLine");
disp("vertexIndices =")
disp(T.RateVertexIndex)
disp("rateVertices =")
disp(vertcat(T.RateVertex{:}))
disp("expressionCount =")
disp(height(T))

```

```

vertexIndices =
     1
     2

rateVertices =
    -1
     2

expressionCount =
     2

```

4.7.2 disp And display

Syntax

```

disp(A)
display(A)
A

```

Arguments

`disp(A)` prints a compact one-line summary. `display(A)` is the command-window display used when an expression is not terminated by a semicolon; it prints grid, degree, coefficient, and source metadata.

Examples and output

```
A = pdmat({[0 1]}, {1, 2}, Degree=1);
disp(A)
display(A)
```

```
pdmat [1 1] over 1-D grid, degree 1, source coefficient-backed, rate rows false
A =
  pdmat with 1-by-1 matrix values
  Parameters: 1
    rho_1: [0, 1], 2 nodes
  Degree: 1
  Physical cells: 1
  Coefficients per cell: 2
  Continuous: true
  Rate metadata: false
  Explicit rate rows: false
  Source: coefficient-backed
  Function handle: false
```

4.7.3 plot

Syntax

```
h = plot(A)
h = plot(A, dims)
h = plot(A, SamplesPerCell=n)
h = plot(A, dims, SamplesPerCell=n)
h = plot(A, ..., RateVertex=k)
h = plot(A, ..., Name, Value)
```

Arguments

- `dims` is one or two parameter indices. For one-parameter objects, the default is 1; otherwise the default is `[1 2]`, as in `plot(A, [1 2])`.
- `SamplesPerCell` is the package-owned sampling option. The default is 15; use `SamplesPerCell=4` for a coarse diagnostic plot.
- `RateVertex=k` selects one one-based stored rate row in the deterministic vertex order induced by applying `helper.combRows` to `num2cell(A.RateBounds,2).'`. Rate-row objects default to row one. Supplying this option for ordinary data or selecting beyond 2^ℓ raises `pdmat:InvalidRateVertex`.
- Remaining name-value pairs pass through to MATLAB graphics. One-dimensional plots pass them to `plot`; two-dimensional plots pass them to `surf`. Examples include `LineWidth`, `FaceAlpha`, and `EdgeColor`.
- The output `h` is the vector of graphics handles, one entry per matrix element.

Remarks

An invalid parameter index, a repeated dimension, or a request for more than two plotting dimensions raises `pdmatrix:InvalidPlotDimensions`. Malformed plotting options or a nonpositive integer `SamplesPerCell` raise `pdmatrix:InvalidPlotOptions`. Errors from MATLAB graphics options remain MATLAB graphics errors. Every unselected parameter is fixed at its lower grid bound, `A.GridInfo.Bounds(:,1)`. A plot of a three-parameter object is therefore a one- or two-dimensional slice, not a volume plot.

For explicit rate rows, select the vertex as part of the plotting call:

```
A = pdmat([0 1], {0, 2}, Degree=1, RateBounds=[-1 2]);
D = rhodiff(A);
plot(D, RateVertex=2, SamplesPerCell=20, LineWidth=2);
```

Examples and output

Each source listing below is the complete source used by `generate_plot_figures.m` for the adjacent result.

One-dimensional entries.

```
A = pdmat({[-1 1]}, @(rho) [rho, rho.^2]);
h = plot(A, SamplesPerCell=80, LineWidth=2);
set(h(1), "Color", [35 86 125]/255, "LineStyle", "-")
set(h(2), "Color", [0.25 0.25 0.25], "LineStyle", "--")
grid on
xlabel("rho")
ylabel("matrix entry")
title("One-dimensional pdmat entries")
legend(["A(1,1)", "A(1,2)"], "Location", "eastoutside")
```

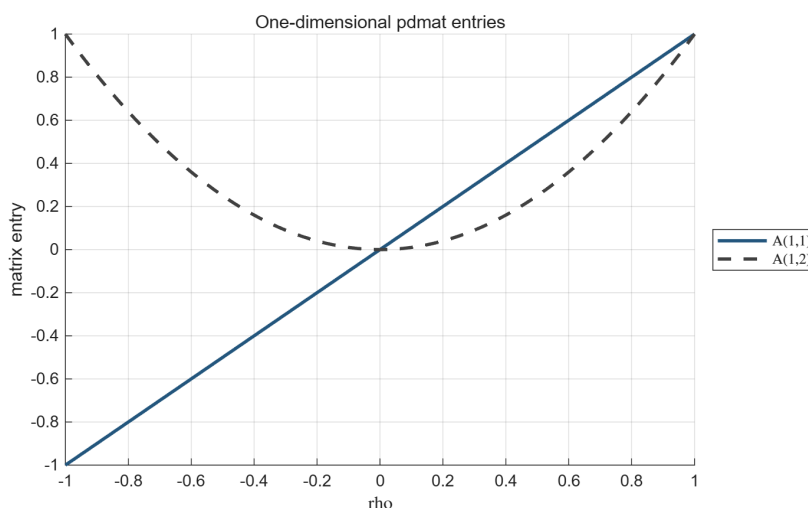


Figure 4.1: Result of the one-dimensional source above. The method returns one line handle per matrix entry.

Two-dimensional surface.

```
A = pdmat({[-1 1], [0 2]}, @, ...
    @(rho1, rho2) 1 + rho1.^2 + 0.5*rho2 + rho1.*rho2);
h = plot(A, [1 2], SamplesPerCell=45, ...
    EdgeColor="none", FaceAlpha=0.78);
set(h, "FaceColor", [35 86 125]/255)
grid on
view(35, 25)
xlabel("rho1", "Interpreter", "none")
ylabel("rho2", "Interpreter", "none")
zlabel("matrix entry")
title("Two-dimensional pdmat surface")
```

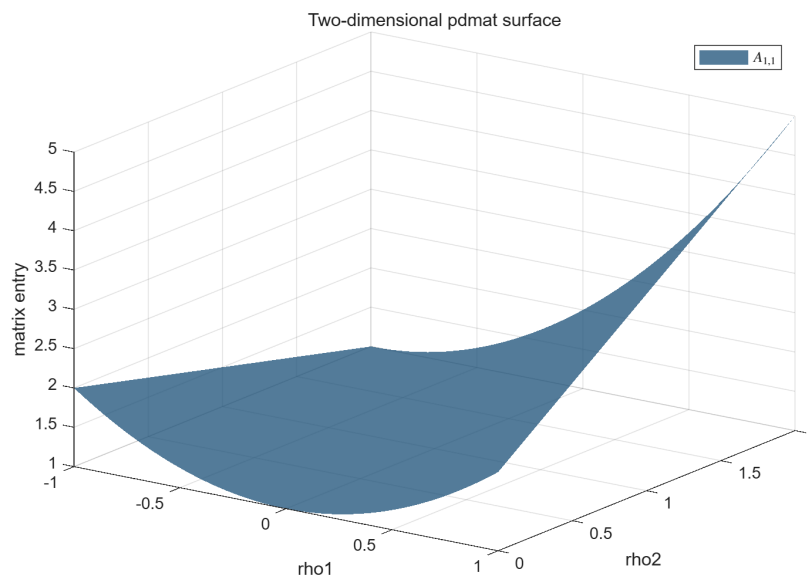


Figure 4.2: Result of the two-dimensional source above.

Two-dimensional matrix-valued surface.

```
A = pdmat({[-1 1], [0 2]}, @, (rho1, rho2) [ ...
    1 + 0.8*rho1 + 0.25*rho2; ...
    1 - 0.8*rho1 + 0.25*rho2]);
h = plot(A, [1 2], SamplesPerCell=45, ...
    EdgeColor="none", FaceAlpha=0.62);
set(h(1), "FaceColor", [35 86 125]/255)
set(h(2), "FaceColor", [0.55 0.55 0.55])
grid on
view(35, 25)
xlabel("rho1", "Interpreter", "none")
ylabel("rho2", "Interpreter", "none")
zlabel("matrix entry")
title("Two entries of a 2-by-1 pdmat cross at rho1 = 0")
legend(h, ["A(1,1)", "A(2,1)"], ...
    "Location", "eastoutside")
```

The two entries are equal for every ρ_2 along $\rho_1 = 0$. Their ordering reverses on opposite sides: the first entry is lower when $\rho_1 < 0$ and higher when $\rho_1 > 0$. The crossing is therefore a property of the matrix function, not an artificial plotting offset.

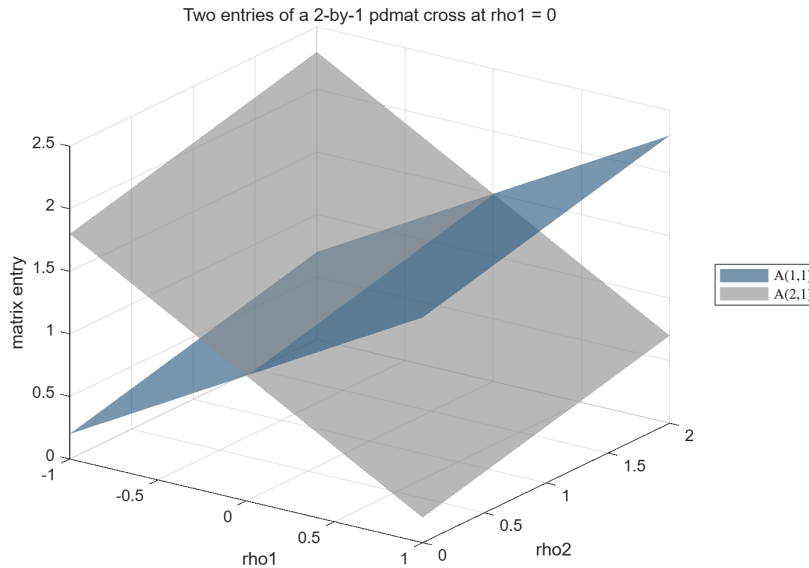


Figure 4.3: Two entry surfaces of a 2-by-1 matrix-valued function. The outside legend distinguishes the entries while their intersection remains visible along $\rho_1 = 0$.

4.8 Combine Known Matrices

A `pdmat` operation acts on complete matrix coefficients while the physical grid remains attached to the result. The blocks below deliberately show one operation family at a time: purpose, supported syntax, input, immediate output, and—when the result is spatial—a before/after picture.

4.8.1 Signs, Addition, And Subtraction

Use signs, sums, and differences to build known matrix expressions. Compatible physical bounds are refined to the union grid, and lower-degree operands are elevated exactly before matching coefficients are combined. The corresponding MATLAB overload names are `plus`, `minus`, `uplus`, and `uminus`.

Syntax

```
S = A + B
S = A + M
S = M + A
D = A - B
D = A - M
D = M - A
P = +A
N = -A
```

Here M is a finite real scalar or size-compatible matrix. A scalar broadcasts to the matrix payload size. The first example makes the degree alignment visible.

Examples and output

```
A = pdmat({[0 1]}, {1, 2}, Degree=1);
B = pdmat({[0 1]}, {10, 20, 30}, Degree=2);
S = A + B;
sumDegree = S.Degree
sumCoefficients = cell2mat(S.coeffs(1))
```

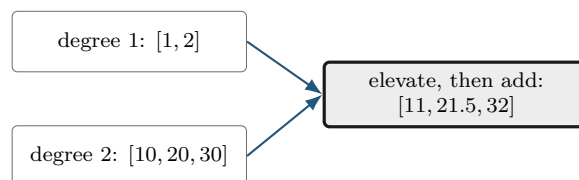
```
sumDegree = 2
sumCoefficients = 1×3 double
    11.0000    21.5000    32.0000
```

The middle coefficient of the elevated A is $(1 + 2)/2 = 1.5$, so the middle sum is $1.5 + 20 = 21.5$. Subtraction and unary signs act immediately on the same local coefficients.

```
D = 5 - A;
N = -A;
differenceCoefficients = cell2mat(D.coeffs(1))
negativeAtOne = N.evaluate(1)
positiveClass = class(+A)
```

```
differenceCoefficients = 1×2 double
     4     3
```

```
negativeAtOne = -2
positiveClass = 'pdmat'
```



When grids have the same outer bounds but different interior nodes, the union grid is used before addition or subtraction. For example, `pdmat({[0 1]},...)+pdmat({[0 .5 1]},...)` has grid `[0 .5 1]`; each operand is represented exactly on both resulting cells.

4.8.2 Ordered Matrix Multiplication

Use `mtimes` for a matrix product. The inner payload dimensions must match. Two parameter-dependent factors produce the degree sum; a numeric factor is constant and does not raise the degree.

Syntax

```
K = A * B
K = A * M
K = M * A
```

For one parameter, fix physical cell c and let $A = \sum_i B_i^m A^{(c)}[i]$ and $D = \sum_j B_j^n D^{(c)}[j]$. Then $K = AD$ has coefficients

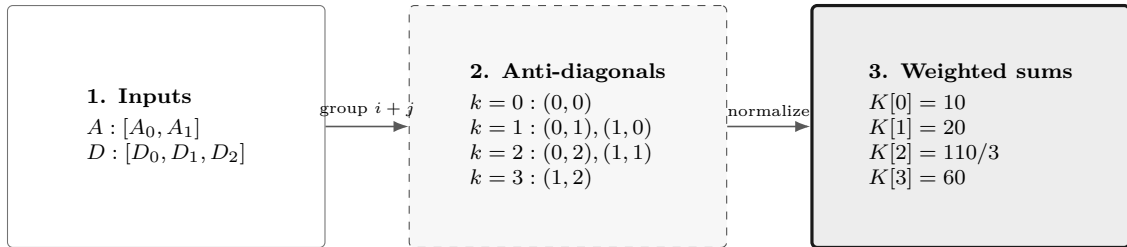
$$K^{(c)}[k] = \sum_{i+j=k} \frac{\binom{m}{i} \binom{n}{j}}{\binom{m+n}{k}} A^{(c)}[i] D^{(c)}[j].$$

The left/right matrix order is never exchanged. It is worthnoting that the above formula is a weighted convolution of the local coefficients, not a simple sum of products. The binomial weights are the Bernstein basis evaluations at the local coordinate $\alpha = i/(m+n)$ for each anti-diagonal $i+j=k$.

Examples and output

```
A = pdmat({[0 1]}, {1, 2}, Degree=1);
B = pdmat({[0 1]}, {10, 20, 30}, Degree=2);
K = A * B;
productDegree = K.Degree
productCoefficients = cell2mat(K.coeffs(1))
```

```
productDegree = 3
productCoefficients = 1×4 double
    10.0000    20.0000    36.6667    60.0000
```



Coefficient labels add under the weighted convolution; payload matrices multiply in the written order. The three panels make the convolution readable without crossing contribution arrows.

For tensor data, label addition and binomial weights are applied independently in each parameter direction. If $\mathbf{m}, \mathbf{n} \in \mathbb{N}^\ell$ are direction-wise basis degrees, the ℓ -dimensional ordered convolution is

$$K^{(c)}[\mathbf{k}] = \sum_{\mathbf{i}+\mathbf{j}=\mathbf{k}} \left(\prod_{s=1}^{\ell} \frac{\binom{m_s}{i_s} \binom{n_s}{j_s}}{\binom{m_s+n_s}{k_s}} \right) A^{(c)}[\mathbf{i}] D^{(c)}[\mathbf{j}].$$

Two degree-one factors in two parameters produce degree two and $3^2 = 9$ coefficients per physical cell.

Remarks

Incompatible operands for `+`, `-`, and `*` raise `pdmat:InvalidAddition`, `pdmat:InvalidSubtraction`, or `pdmat:InvalidMultiplication`. Parameter counts or outer bounds that cannot be merged raise `pdmat:MixedGrid`. Coefficient algebra on a function-only object raises `pdmat:FunctionOnlyAlgebra`; construct the function with an explicit verified `Degree` when coefficient evidence is required.

4.9 Compare and Certify Known Data

Syntax

```
tf = A == B
C = A <= B
C = A >= B
C = A <= M
C = A >= M
```

Arguments

For two `pdmat` operands, `A==B` is a scalar logical coefficient-evidence comparison. Subtraction performs the compatibility checks, grid alignment, and degree elevation; the result is true exactly when the aligned coefficient difference is provably zero. It never creates a `pdlmi`. By contrast, `<=` and `>=` create a `pdlmi` from a compatible coefficient-backed known residual. A function-only object cannot supply finite coefficient evidence. Numeric `M` follows the ordinary finite-real size and scalar-broadcast rules.

Examples and output

```
A = pdmat([0 1], {1, 3}, Degree=1);
B = pdmat([0 1], {1, 2, 3}, Degree=2);
same = A == B
different = A == 2*A

yalmip('clear')
K = pdmat([0 1], {{1, 2; 3, 4}}, ...
    Degree=1, RateBounds=[-1 2]);
direct = K >= 0;
directCertificate = toYalmip(direct)
polya = direct.applyPolya(1);
polyaCertificate = toYalmip(polya)
```

```
same = logical
      1
```

```
different = logical
0

directCertificate = logical
1

polyaCertificate = logical
1
```

Remarks

Equality with a non-`pdmatrix` operand raises `pdmatrix:InvalidEquality`. Between two `pdmatrix` objects, subtraction owns compatibility checks and representation alignment, so incompatible operands propagate the corresponding subtraction, grid, continuity, or rate-bound error. A known `pdmatrix` equality is deliberately not a `pdlmi` residual. Direct and Pólya known-data exports reduce the complete family to one scalar logical certificate using the wrapper’s global inequality classification and inclusive absolute tolerance 10^{-10} . In semidefinite mode, every coefficient matrix is symmetrized as $(C + C^T)/2$ and its eigenvalues are tested against the oriented tolerance; scalar entries are not checked independently. Only element-wise mode tests every scalar matrix entry. A false result emits `pdlmi:InconclusiveCertificate` when exported and does not prove a continuous violation or indefiniteness. Putinar, SparseFullBox, and FullBox still return YALMIP constraints and may introduce auxiliary Gram variables even when the residual is known.

4.10 Reshape and Transform Known Matrices

These methods transform every local matrix coefficient in the same way. They are inherited from the centralized `pdbase` implementation, while the `pdmatrix` reconstruction hook preserves the `pdmatrix` class, known-data payloads, and metadata. They do not transpose, reshape, or concatenate the physical grid. Except for the documented evaluator/display/shape paths, structural methods require coefficient evidence.

4.10.1 Shape, Equality, And Display Protocols

Shape queries describe the matrix payload, not the grid. `isequal` compares object representations after compatible grid refinement and degree elevation; scalar logical operator equality is documented in section 4.9. The protocol methods support ordinary MATLAB indexing syntax and dot access; they are not a second storage API.

Syntax

```
sz = size(A)
d = size(A, dim)
n = height(A)
n = width(A)
n = length(A)
n = numel(A)
n = ndims(A)
tf = isequal(A, B, ...)
idx = end(A, k, n)
n = numArgumentsFromSubscript(A, S, ctx)
```

Examples and output

```
A = pdmat({[0 1]}, ...
    {[1 2 3; 4 5 6], [10 20 30; 40 50 60]}, Degree=1);
matrixSize = size(A)
rowCount = height(A)
columnCount = width(A)
longestDimension = length(A)
elementCount = numel(A)
dimensionCount = ndims(A)
equalsUnaryPlus = isequal(A,+A)
```

```
matrixSize = 1×2 double
           2     3

rowCount = 2
columnCount = 3
longestDimension = 3
elementCount = 6
dimensionCount = 2
equalsUnaryPlus =
    logical
         1
```

A function-only object can use shape queries and `isequal` compares its stored metadata and function handle. A coefficient-backed comparison that cannot form a common grid raises `pdmat:InvalidEquality`. The detailed multi-line `display` transcript is shown in section [4.7.2](#).

4.10.2 Transpose And Conjugate Transpose

Both transpose forms act on every coefficient. Because `pdmat` payloads are finite and real, conjugation does not change their values.

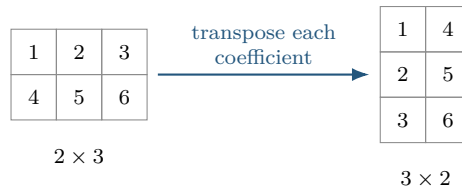
Syntax

```
T = transpose(A)
H = ctranpose(A)
T = A.'
H = A'
```

Examples and output

```
A = pdmat({[0 1]}, ...
          {[1 2 3; 4 5 6], [10 20 30; 40 50 60]}, Degree=1);
T = A.';
H = A';
transposeSize = size(T)
formsEqual = isequal(T,H)
transposeAtZero = T.evaluate(0)
```

```
transposeSize = 1×2 double
               3     2
formsEqual =
    logical
         1
transposeAtZero = 3×2 double
                 1     4
                 2     5
                 3     6
```



4.10.3 Vectorization, Reshape, And Squeeze

Use these methods to change the matrix view while preserving column-major element order, the parameter grid, the degree, and every coefficient value. **reshape** accepts exactly two positive dimensions with at most one inferred [] dimension. **squeeze** retains the package's two-dimensional matrix convention.

Syntax

```
V = vec(A)
R = reshape(A, [m n])
R = reshape(A, m, n)
S = squeeze(A)
```

Examples and output

```
A = pdmat({[0 1]}, ...
          {[1 2 3; 4 5 6], [10 20 30; 40 50 60]}, Degree=1);
V = vec(A);
R = reshape(A, [3 2]);
S = squeeze(A);
vectorSize = size(V)
reshapeSize = size(R)
squeezeSize = size(S)
vectorAtZero = V.evaluate(0).'
```

```
vectorSize = 1×2 double
    6     1
reshapeSize = 1×2 double
    3     2
squeezeSize = 1×2 double
    2     3
vectorAtZero = 1×6 double
    1     4     2     5     3     6
```

Malformed or element-count-changing reshape requests raise `pdmat:InvalidReshape`.

4.10.4 Diagonals And Trace

`diag` extracts a selected diagonal from a matrix, or builds a square diagonal matrix from a vector. A finite integer offset `k` may not select an empty diagonal. `trace` returns a 1-by-1 `pdmat`.

Syntax

```
d = diag(A)
d = diag(A, k)
D = diag(v)
D = diag(v, k)
t = trace(A)
```

Examples and output

```
A = pdmat({[0 1]}, {[1 2; 3 4]}, [10 20; 30 40]), Degree=1);
d = diag(A);
D = diag(d);
t = trace(A);
diagonalAtZero = d.evaluate(0)
rebuiltAtZero = D.evaluate(0)
traceAtZero = t.evaluate(0)
```

```
diagonalAtZero = 2×1 double
    1
    4
rebuiltAtZero = 2×2 double
    1    0
    0    4
traceAtZero = 5
```

Invalid offsets or empty selections raise `pdmat:InvalidDiag`.

4.10.5 Triangular Parts And Reductions

Triangular extraction preserves or zeros entries relative to a selected diagonal. Reductions act along matrix dimensions; "all" is supported by `sum` and `mean`.

Syntax

```
L = tril(A)
L = tril(A, k)
U = triu(A)
U = triu(A, k)
S = sum(A)
S = sum(A, dim)
S = sum(A, vecdim)
S = sum(A, "all")
M = mean(A)
M = mean(A, dim)
M = mean(A, vecdim)
M = mean(A, "all")
C = cumsum(A)
C = cumsum(A, dim)
C = cumsum(A, direction)
C = cumsum(A, dim, direction)
```

First inspect the two triangular results.

Examples and output

```
A = pdmat({[0 1]}, {[1 2; 3 4]}, [10 20; 30 40]), Degree=1);
L = tril(A);
U = triu(A);
lowerAtZero = L.evaluate(0)
upperAtZero = U.evaluate(0)
```

```
lowerAtZero = 2×2 double
    1     0
    3     4
upperAtZero = 2×2 double
    1     2
    0     4
```

Then apply reductions to the same coefficient matrix.

```
rowSum = sum(A, 2);
allMean = mean(A, "all");
running = cumsum(A, 2);
rowSumAtZero = rowSum.evaluate(0)
allMeanAtZero = allMean.evaluate(0)
runningAtZero = running.evaluate(0)
```

```
rowSumAtZero = 2×1 double
     3
     7
allMeanAtZero = 2.5000
runningAtZero = 2×2 double
     1     3
     3     7
```

`vecdim` is a nonempty vector of unique positive integer dimensions; dimensions above two are singleton no-ops. `direction` is "forward" or "reverse", case-insensitively. A direction supplied without a dimension uses the first nonsingleton matrix dimension, and `cumsum` along a dimension above two returns the same coefficient payloads. Invalid triangular offsets raise `pdmat:InvalidTriangularPart`. Invalid reduction calls raise `pdmat:InvalidSum`, `pdmat:InvalidMean`, or `pdmat:InvalidCumsum`; missing-value and native-output flags are not supported.

4.10.6 Reordering And Rotation

These methods reorder rows or columns inside every coefficient; they never reverse the scheduling grid.

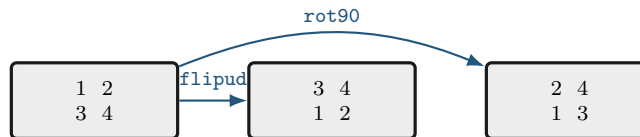
Syntax

```
B = flip(A)
B = flip(A, dim)
B = flipud(A)
B = fliplr(A)
B = rot90(A)
B = rot90(A, k)
```

Examples and output

```
A = pdmat({[0 1]}, {[1 2; 3 4], [10 20; 30 40]}, Degree=1);
F = flipud(A);
R = rot90(A);
flippedAtZero = F.evaluate(0)
rotatedAtZero = R.evaluate(0)
```

```
flippedAtZero = 2×2 double
    3     4
    1     2
rotatedAtZero = 2×2 double
    2     4
    1     3
```



Invalid dimensions or quarter-turn counts raise `pdmat:InvalidFlip` or `pdmat:InvalidRot90`.

4.10.7 Concatenation And Block Assembly

Concatenation joins payload matrices after compatible grid refinement and degree elevation. Numeric blocks are promoted to constant known data. `cat` supports only dimensions 1 and 2.

Syntax

```
H = horzcat(A, B, ...)
V = vertcat(A, B, ...)
H = [A, B]
V = [A; B]
C = cat(dim, A, B, ...)
D = blkdiag(A, B, ...)
```

Examples and output

```
a = pdmat({[0 1]}, {1, 2}, Degree=1);
b = pdmat({[0 1]}, {10, 20}, Degree=1);
H = [a, b];
V = [a; b];
D = blkdiag(a, 5);
horizontalAtZero = H.evaluate(0)
verticalAtZero = V.evaluate(0)
blockDiagonalAtZero = D.evaluate(0)
```

```
horizontalAtZero = 1×2 double
    1    10
verticalAtZero = 2×1 double
    1
   10
blockDiagonalAtZero = 2×2 double
    1    0
    0    5
```



Incompatible blocks or syntax raise `pdmat:InvalidConcatenation`; unsupported dimensions raise `pdmat:UnsupportedCatDimension`; invalid block-diagonal inputs raise `pdmat:InvalidBlkdiag`. Function-only operands raise `pdmat:FunctionOnlyAlgebra`.

4.10.8 Repetition

`repmat` tiles the matrix payload. It accepts a positive scalar `m`, interpreted as `[m m]`, two positive counts, or a two-element count vector. Grid and Bernstein degree remain unchanged.

Syntax

```
R = repmat(A, m)
R = repmat(A, m, n)
R = repmat(A, [m n])
```

Examples and output

```
v = pdmat({[0 1]}, {[1 2], [3 4]}, Degree=1);
R = repmat(v, [2 2]);
repeatedSize = size(R)
```

```
repeatedDegree = R.Degree
repeatedAtZero = R.evaluate(0)
```

```
repeatedSize = 1×2 double
    2    4
repeatedDegree = 1
repeatedAtZero = 2×4 double
    1    2    1    2
    1    2    1    2
```

Invalid repetition shapes raise `pdmat:InvalidRepmat`.

4.11 Index and Assign pdmat Entries

Parenthesis indexing selects a matrix block from every coefficient. Assignment writes the selected block into every coefficient after compatible promotion and degree alignment. It does not select a physical cell; use `coeffs` for cell inspection.

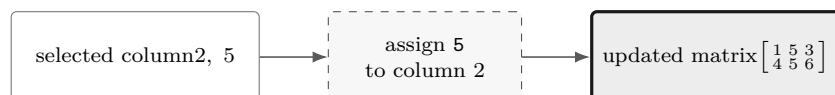
Syntax

```
B = A(rows, cols)
B = A(1:end, end)
value = A.PropertyName
A(rows, cols) = numericValue
A(rows, cols) = B
```

Examples and output

```
A = pdmat({[0 1]}, ...
    {[1 2 3; 4 5 6], [10 20 30; 40 50 60]}, Degree=1);
lastCol = A(:, end);
A(:, 2) = 5;
selectedSize = size(lastCol)
assignedSize = size(A)
selectedAtZero = lastCol.evaluate(0)
assignedAtZero = A.evaluate(0)
```

```
selectedSize = 1×2 double
    2    1
assignedSize = 1×2 double
    2    3
selectedAtZero = 2×1 double
    3
    6
assignedAtZero = 2×3 double
    1    5    3
    4    5    6
```



`subsref`, `subsasgn`, `end`, and `numArgumentsFromSubscript` implement this two-index MATLAB protocol. Invalid selectors raise `pdmatrix:InvalidSubscript`. Linear indexing, deletion, or non-parenthesis assignment raises `pdmatrix:UnsupportedAssignment`; an incompatible right-hand side raises `pdmatrix:InvalidAssignment`. Slicing and assignment require coefficient evidence and otherwise raise `pdmatrix:FunctionOnlyAlgebra`.

4.12 Boundaries and Related APIs

1. Function-only objects created without `Degree` do not have Bernstein coefficient evidence for `bernsteinTable`, coefficient algebra, differentiation, or inequalities.
2. Numeric operands must be finite real nonempty matrices with compatible sizes, or finite real scalars that can broadcast to the required size.
3. Ordinary one-row operands broadcast across explicit rate rows. Two explicit-rate-row operands must have exactly matching grids and nonempty rate bounds; they are not automatically refined. Multiplication supports explicit rate rows on at most one side and rejects rate-row by rate-row products.
4. Algebra, concatenation, indexing, and assignment preserve compatible rate metadata and stored row order. Mismatched rate boxes, grids, row counts, or rate-dependent product forms raise the owning operation's class-specific error.

Related APIs. `pdmatrix`, `evaluate`, `rhodiff`, `bernsteinTable`, `plot`, `pdlmi`, `pdbase`.

Chapter 5

pdvar: Decision Matrix Functions

`pdvar` creates continuous cell-local Bernstein decision expressions backed by YALMIP `sdpvar` coefficients. It participates in supported affine and known-data algebra. It does not choose solvers or call `optimize`. Comparison overloads create `pdlmi` objects, whose `toYalmip` method later converts them to YALMIP constraints. Shared matrix-shape, unary, structural, reduction, reordering, and repetition methods are implemented centrally by `pdbase` and remain complete public `pdvar` behavior; this chapter documents their decision-expression syntax, stable shape examples, validations, and rate-aware boundaries.

A degree- m `pdvar` represents a decision such as $P(\boldsymbol{\rho})$ or $y_k(\boldsymbol{\rho})$. For $m \geq 1$, neighboring cells reuse every decision handle on their complete shared face, which enforces value continuity without auxiliary equality constraints; for $m = 0$, the entire grid reuses one decision matrix. `rhodiff` applies cellwise forward differences scaled by $1/h_s^{(c)}$ and stores one row per vertex of $\mathcal{R}$, while `value` recovers numeric coefficient payloads only after the caller has validated YALMIP's solver status. Products remain affine only when the other factor is known numeric or `pdmat` data.

5.1 API Map

Task	Entry
Construct decisions	<code>pdvar</code>
Inspect coefficients	<code>bernsteinTable</code> , <code>shared storage/evaluation/elevation</code>
Convert assigned coefficients	<code>value</code>
Differentiate rates	<code>rhodiff</code>
Affine/mixed algebra	<code>+</code> , <code>-</code> , <code>*</code> with known data
Matrix/index methods	<code>structural methods</code> , <code>indexing and assignment</code>
Build LMIs	<code><=</code> , <code>>=</code> , <code>==</code>

5.2 Construct `pdvar` Decisions

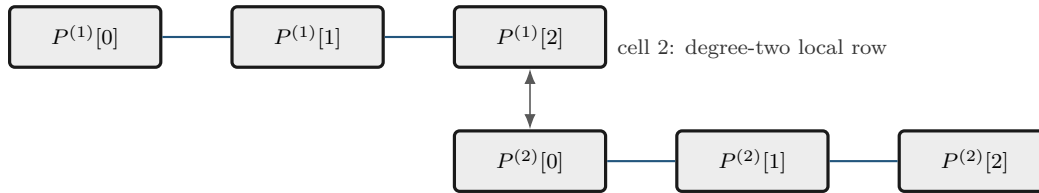
Syntax

```
P = pdvar(n, gridVectors)
P = pdvar(n, n, gridVectors)
P = pdvar(n, p, gridVectors)
P = pdvar(..., "full")
P = pdvar(..., "symmetric")
P = pdvar(..., Degree=0)
P = pdvar(..., Degree=1)
P = pdvar(..., Degree=m)
P = pdvar(..., RateBounds=rb)
P = pdvar(..., ValidationMode=mode)
```

Arguments

- `pdvar(n, gridVectors)` creates an $n \times n$ square variable using scalar-size shorthand; for example, `pdvar(2, {[0 1]})`. Square variables default to symmetric YALMIP coefficients.
- `pdvar(n, n, gridVectors)` is the explicit square-size form; for example, `pdvar(2, 2, {[0 1]})`. It has the same default symmetric structure as the shorthand.
- `pdvar(n, p, gridVectors)` creates an $n \times p$ variable. Rectangular variables default to full YALMIP coefficients; for example, `pdvar(2, 3, {[0 1]})`.
- "full" and "symmetric" explicitly choose the YALMIP coefficient structure. "symmetric" requires a square size.
- `Degree=1` is the default continuous Bernstein decision variable. Every finite nonnegative integer `Degree=m` is supported. `Degree=0` creates one parameter-independent decision matrix shared across the grid; `m >= 1` creates continuous piecewise Bernstein data whose neighboring cells share complete control faces.
- `RateBounds=rb` stores parameter-rate metadata for later `rhodiff(P)` calls. Use `RateBounds=[-1 1]` for one parameter; use a two-row matrix for two parameters, as shown in the rate-metadata example below.
- `ValidationMode=mode` accepts scalar text "fast" or "strict", case-insensitively, and defaults to "fast". It affects repeated validation of package-generated storage only; public options, grid, structure, affine safety, continuity, and source semantics remain global in both modes. The mode is not stored.

cell 1: degree-two local row



same YALMIP handle: $P^{(1)}[2] = P^{(2)}[0]$

For positive degree, neighboring physical cells reuse complete boundary-face YALMIP handles; no extra continuity LMI is generated.

Examples and output

The square shorthand creates a 2×2 continuous decision object. The default degree is one and the object carries no rate metadata.

```
yalmip('clear')
Ps = pdvar(2, {[0 1]});
P= Ps.coeffs([1]);
symmetricCoefficient = ishermitian(P{1})
variableClass = class(Ps)
matrixSize = size(Ps)
degree = Ps.Degree
rateDependent = Ps.HasRateDependence
```

```
symmetricCoefficient = 1
variableClass = 'pdvar'
matrixSize = 1×2 double
    2    2

degree = 1

rateDependent =
    logical
    0
```

Use two dimension arguments for a rectangular full variable.

```
yalmip('clear')
Pf = pdvar(2, 3, {[0 1]});
variableClass = class(Pf)
matrixSize = size(Pf)
degree = Pf.Degree
```

```
variableClass = 'pdvar'
matrixSize = 1×2 double
    2    3

degree = 1
```

The structure flag makes the coefficient structure explicit. This is useful when a square variable should use full, not symmetric, coefficients.

```
yalmip('clear')
Pfull = pdvar(2, 2, {[0 1]}, "full");
P = Pfull.coeffs([1]);
fullCoefficient = ishermitian(P{1})
variableClass = class(Pfull)
matrixSize = Pfull.MatrixSize
degree = Pfull.Degree
```

```
fullCoefficient = logical
0
```

```
variableClass = 'pdvar'
matrixSize = 1×2 double
2 2
```

```
degree = 1
```

Use Degree=0 for a parameter-independent decision variable stored with grid metadata.

```
yalmip('clear')
P0 = pdvar(1, [0 1 2], Degree=0);
variableClass = class(P0)
degree = P0.Degree
cellCount = P0.ncell()
```

```
variableClass = 'pdvar'
degree = 0
cellCount = 2
```

Rate bounds are metadata for later `rhodiff(P)` calls. `pdlmi` constrains rate vertices only after an expression such as `rhodiff` has stored rate-vertex rows.

```
yalmip('clear')
Pr = pdvar(1, {[0 1], [10 20]}, RateBounds=[-1 2; -3 4]);
variableClass = class(Pr)
rateDependent = Pr.HasRateDependence
rateBounds = Pr.RateBounds
```

```
variableClass = 'pdvar'
rateDependent =
logical
1
```

```
rateBounds = 2×2 double
-1 2
-3 4
```

Remarks

- Missing or misplaced dimensions and grids raise `pdvar:InvalidInput`; invalid sizes and degrees raise `pdvar:InvalidMatrixSize` or `pdvar:InvalidDegree`.
- Option failures use `pdvar:InvalidOptions`, `pdvar:UnknownOption`, or `pdvar:UnsupportedOption`; invalid modes raise `pdvar:InvalidValidationMode`.
- A symmetric nonsquare request raises `pdvar:InvalidStructure`; grid failures use `pdvar:InvalidGrid` or `pdvar:InvalidGridVector`.
- `RateBounds` must be a finite ℓ -by-2 matrix with each lower bound no greater than its upper bound; violations raise `pdbase:InvalidRateBounds`.

5.3 Inspect Object Properties

`pdvar` declares no additional class-owned properties. It exposes the read-only inherited `pdbase` properties listed in section 3.3; their private set access prevents user assignment. In particular, decision coefficients are inspected through `P.LocalValues`, while `P.Degree`, `P.IsContinuous`, and `P.RateBounds` describe their representation and rate metadata.

```
yalmip('clear')
P = pdvar(2, {[0 1]}, "symmetric", Degree=2, RateBounds=[-1 1]);
degree = P.Degree
isContinuous = P.IsContinuous
rateBounds = P.RateBounds
firstCellCoefficientCount = numel(P.coeffs(1))
```

```
degree = 2

isContinuous =
    logical
         1

rateBounds = 1×2 double
         -1         1

firstCellCoefficientCount = 3
```

Use public algebra to create a modified value; direct assignments such as `P.Degree = 1` are not supported.

5.4 Inspect, Evaluate, And Elevate Inherited Storage

Syntax

```
cellSubs = cells(P)
labels = lbls(P)
coefficients = coeffs(P, cellSubs)
n = ncell(P)
n = ncoeff(P)
n = npar(P)
X = evaluate(P, point)
X = P.evaluate(point)
vals = P.elevVals(degreeIncrement)
Q = P.elevate(degreeIncrement)
```

Description

The storage queries use the shared **pdbase** cell and label order and return the symbolic **sdpvar** coefficient payloads of the selected decision expression. **evaluate** reconstructs the affine symbolic matrix at one in-bounds parameter point; a derivative/rate-row object returns a 1-by- 2^ℓ cell array in stored rate-vertex order. **elevVals** returns only an elevated nested coefficient tree. **elevate** returns a new **pdvar** of the same dynamic class with the same represented expression, physical cells, YALMIP decision variables, metadata, and rate-row order at the higher degree; it does not create new decision freedom or modify P.

Examples and output

```
yalmip('clear')
P = pdvar(2, {[0 1]}, "full", Degree=1);
physicalCells = P.cells()
labels = P.lbls()
coefficientTableSize = size(P.coeffs(1))
counts = [P.ncell(), P.ncoeff(), P.npar()]
X = P.evaluate(0.25);
Q = P.elevate(2);
sampleClass = class(X)
sampleSize = size(X)
elevatedClass = class(Q)
degrees = [P.Degree, Q.Degree]
elevatedCoefficientTableSize = size(Q.coeffs(1))
```

```
physicalCells = 1
labels = 2×1 double
    0
    1
coefficientTableSize = 1×2 double
    1    2
counts = 1×3 double
    1    2    1
sampleClass = 'sdpvar'
sampleSize = 1×2 double
    2    2
elevatedClass = 'pdvar'
```

```

degrees = 1×2 double
    1     3
elevatedCoefficientTableSize = 1×2 double
    1     4

```

Remarks

Invalid physical-cell selectors raise `pdbase:InvalidCellSubs`. Malformed points raise `pdvar:InvalidPoint`; out-of-bounds points raise `pdvar:PointOutOfBounds`. Invalid elevation increments raise `pdbase:InvalidDegreeIncrement`. Evaluation returns symbolic expressions until the user assigns or solves the underlying YALMIP variables; use `value` for the post-assignment numeric conversion documented in section 5.10.

5.5 rhodiff

Syntax

```

D = rhodiff(P)
D = rhodiff(P, rb)

```

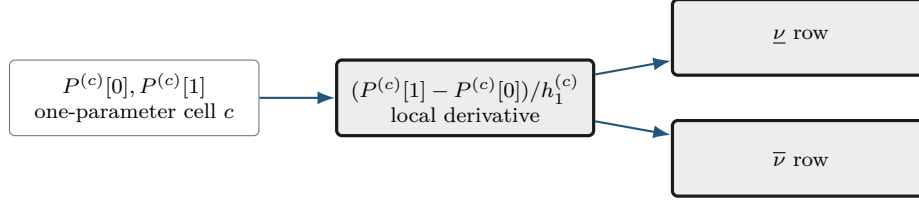
Arguments

- `P` is a `pdvar` without existing rate-vertex rows.
- `rb` is a finite ℓ -by-2 numeric matrix of lower and upper parameter-rate bounds, with each lower bound no greater than its upper bound. It must equal `P.RateBounds` when the object already carries nonempty bounds.
- Without `rb`, `rhodiff(P)` requires nonempty `P.RateBounds`.
- `D` is a discontinuous `pdvar` with one coefficient row per rate vertex. In one parameter a degree- m derivative has degree $m - 1$, while a degree-zero derivative remains zero; multivariate partials are elevated into the implementation's common tensor degree.

For a one-parameter degree-one coefficient pair $P^{(c)}[0], P^{(c)}[1]$ on $\mathcal{H}_c = [\rho_1^{(c)}, \rho_1^{(c+1)}]$, define $h_1^{(c)} = \rho_1^{(c+1)} - \rho_1^{(c)} > 0$. The rate-dependent derivative row is then

$$\dot{P}(\rho, \nu) = \nu \frac{P^{(c)}[1] - P^{(c)}[0]}{h_1^{(c)}}.$$

This is the one-dimensional special case: the two rows correspond to the lower and upper allowed scheduling rates.



Differentiation changes local coefficients; rate substitution creates rows. Neither operation adds physical cells.

For ℓ parameters, work cell by cell with $h_s^{(c)} = \rho_s^{(c_s+1)} - \rho_s^{(c_s)} > 0$ in direction s of $\mathcal{H}_c$. Thus nonuniform grids use a distinct step length in every direction and cell:

$$\dot{P}^{(c)}(\boldsymbol{\rho}, \boldsymbol{\nu}) = \sum_{s=1}^{\ell} \nu_s \frac{\partial P^{(c)}}{\partial \rho_s}(\boldsymbol{\rho}).$$

`RateBounds` fixes $\mathcal{R}$, so `rhodiff` evaluates this affine expression at its 2^ℓ vertices and stores one row per vertex. Supply those bounds explicitly as `rhodiff(P, rb)`, or initialize `P` with `RateBounds=rb`; if both are present they must match. Without bounds there is no finite vertex set to store, so `rhodiff(P)` raises `pdvar:MissingRateBounds`. Section A.9.1 gives the exact hypercube-local row and coefficient layout.

Remarks

Unsupported call arity or differentiating an expression that already contains rate-vertex rows raises `pdvar:InvalidDiff`. Explicit malformed rate bounds raise `pdvar:InvalidRateBounds`. Calling `rhodiff(P)` without stored bounds raises `pdvar:MissingRateBounds`; explicit bounds that disagree with `P.RateBounds` raise `pdvar:RateBoundsMismatch`.

Examples and output

When rate bounds live on the object, `rhodiff(P)` uses them directly.

```
yalmp('clear')
P = pdvar(2, {[0 1 2]}, "symmetric", RateBounds=[-1 1]);
D = rhodiff(P);
derivativeClass = class(D)
derivativeDegree = D.Degree
isContinuous = D.IsContinuous
coefficientTableSize = size(D.coeffs(1))
rateBounds = D.RateBounds
```

```
derivativeClass = 'pdvar'
derivativeDegree = 0
isContinuous =
    logical
    0
coefficientTableSize = 1×2 double
     2     1
rateBounds = 1×2 double
    -1     1
```

For a two-parameter object, the rate-vertex rows enumerate all lower/upper combinations from the two rows of `RateBounds`.

```

yalmp('clear')
Q = pdvar(1, {[0 1], [10 20]}, RateBounds=[-1 1; -2 2]);
E = rhodiff(Q);
% Rows 1:4 use [-1 -2], [-1 2], [1 -2], and [1 2], respectively.
derivativeClass = class(E)
derivativeDegree = E.Degree
coefficientTableSize = size(E.coeffs([1 1]))
rateVertexCount = coefficientTableSize(1)
coefficientsPerRateVertex = coefficientTableSize(2)
rateBounds = E.RateBounds

```

```

derivativeClass = 'pdvar'
derivativeDegree = 1
coefficientTableSize = 1×2 double
    4    4
rateVertexCount = 4
coefficientsPerRateVertex = 4
rateBounds = 2×2 double
    -1    1
    -2    2

```

5.6 bernsteinTable

Syntax

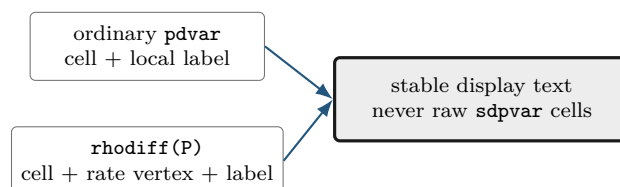
```

T = bernsteinTable(P)
T = bernsteinTable(P, cellSubs)
T = bernsteinTable(P, "oneLine")
T = bernsteinTable(P, cellSubs, "oneLine")

```

Arguments

- P is a pdvar object.
- cellSubs selects one physical cell; omit it to list every cell.
- "oneLine" is the only supported text option. It returns one compact Bernstein expression per selected cell.
- T is a table. Ordinary rows contain the local label, basis, physical-node flag, and symbolic or numeric value. A rate-dependent derivative also includes RateVertexIndex and RateVertex.



Examples and output

```
yalmip('clear')
P = pdvar(1, {[0 1]});
T = bernsteinTable(P, "oneLine");
tableHeight = height(T)
tableWidth = width(T)
hasExpression = ismember("Expression", string(T.Properties.VariableNames))
```

```
tableHeight = 1
tableWidth = 2
hasExpression = logical
1
```

For $D = \text{rhodiff}(P)$, `bernsteinTable(D, cellSubs, "oneLine")` emits one row for every selected physical cell and rate-box vertex, rather than one row per local Bernstein coefficient. `RateVertexIndex` and `RateVertex` identify the lower/upper rate tuple; their tensor order is defined in Section A.9.1.

```
yalmip('clear')
P = pdvar(1, {[0 1], [0 1]}, RateBounds=[-1 1; -2 2]);
D = rhodiff(P);
T = bernsteinTable(D, [1 1], "oneLine");
vertexIndices = T.RateVertexIndex
rateVertices = vertcat(T.RateVertex{:})
expressionCount = height(T)
```

```
vertexIndices = 4×1 double
1
2
3
4

rateVertices = 4×2 double
-1 -2
-1 2
1 -2
1 2

expressionCount = 4
```

Remarks

`bernsteinTable` is an inspection method, not a solver call. Text options other than "oneLine", repeated selectors, invalid physical-cell selectors, or inconsistent rate-vertex rows raise `pdvar:InvalidBernsteinTableInput`.

5.7 Build Affine Algebra with Known Data

A `pdvar` result must remain affine in the underlying YALMIP decisions. The following blocks separate affine sums from products so the supported boundary is visible beside each example.

5.7.1 Signs, Addition, And Subtraction

Use this family to combine compatible decision expressions, coefficient-backed known data, finite real numeric data, and affine `sdpvar` matrices. Operands with the same physical bounds are aligned on a common grid and degree. The corresponding overload names are `plus`, `minus`, `uplus`, and `uminus`.

Syntax

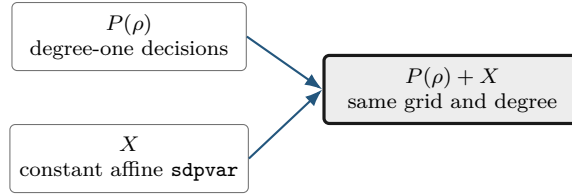
```
R = P + Q
R = P + A
R = A + P
R = P + M
R = M + P
R = P + X
R = X + P
R = P - Q
R = P - A
R = A - P
R = P - M
R = M - P
R = +P
R = -P
```

Here `A` is coefficient-backed `pdmat`, `M` is numeric, and `X` is affine `sdpvar`. A numeric scalar broadcasts to the payload size; an affine `sdpvar` is promoted as parameter-independent data.

Examples and output

```
yalmip('clear')
P = pdvar(2, {[0 1]}, "full");
X = sdpvar(2, 2, 'full');
S = P + X;
D = eye(2) - P;
N = -P;
sumClass = class(S)
sumSize = size(S)
sumDegree = S.Degree
sumContainsDecision = S.ContainsDecision
differenceClass = class(D)
negativeClass = class(N)
```

```
sumClass = 'pdvar'
sumSize = 1×2 double
         2     2
sumDegree = 1
sumContainsDecision =
    logical
         1
differenceClass = 'pdvar'
negativeClass = 'pdvar'
```



A nonlinear symbolic operand such as `eta*eta` is rejected. Rate-vertex rows from `rhodiff` may be added to compatible ordinary expressions; the ordinary rows are broadcast over the active rate vertices.

5.7.2 Multiplication By Known Or Numeric Data

The `mtimes` overload supports multiplication only when decision dependence occurs on at most one side. A coefficient-backed `pdmat` or finite real numeric matrix may multiply a `pdvar` from either side. A scalar `pdvar` may scale a known matrix. Two decision-bearing factors, including `P*Q` and `P*eta` for a decision `sdpvar` `eta`, are nonlinear and rejected.

Syntax

```

R = A * P
R = P * A
R = M * P
R = P * M

```

The first example shows numeric left and right matrix products without exposing incidental YALMIP variable identifiers.

Examples and output

```

yal mip('clear')
P = pdvar(2, 1, {[0 1]}, "full");
L = [1 2] * P;
R = P * [4 5];
S = 3 * P;
leftSize = size(L)
rightSize = size(R)
scaledSize = size(S)
degrees = [L.Degree R.Degree S.Degree]

```

```

leftSize = 1×2 double
    1     1
rightSize = 1×2 double
    2     2
scaledSize = 1×2 double
    2     1
degrees = 1×3 double
    1     1     1

```

A known parameter-dependent factor adds its Bernstein degree through ordered coefficient convolu-

tion. On one cell, a quadratic decision and affine known factor produce

$$(PA)[k] = \sum_{i+j=k} \frac{\binom{2}{i}\binom{1}{j}}{\binom{3}{k}} P[i]A[j], \quad k = 0, 1, 2, 3.$$

The same backend rule is documented at `bernProd` in section 3.9.5 and derived step by step in Appendix A.

```

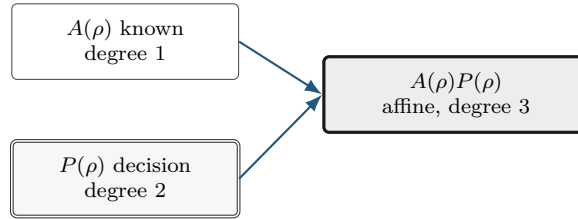
yalmip('clear')
P = pdvar(2, {[0 1]}, "symmetric", Degree=2);
A = pdmat({[0 1]}, {eye(2), 2*eye(2)}, Degree=1);
L = A * P;
R = P * A;
knownLeftClass = class(L)
knownLeftDegree = L.Degree
knownRightClass = class(R)
knownRightDegree = R.Degree
coefficientsPerCell = numel(L.coeffs(1))

```

```

knownLeftClass = 'pdvar'
knownLeftDegree = 3
knownRightClass = 'pdvar'
knownRightDegree = 3
coefficientsPerCell = 4

```



A derivative object may be multiplied by numeric data or a matching-grid known object while preserving every rate row. Products with rate dependence on both sides, products of a derivative with an ordinary decision expression, and products involving metadata-only rate dependence are rejected.

Two decision-bearing factors are also rejected: their product would be quadratic in YALMIP decision coefficients. The supported boundary is numeric or known `pdmat` data multiplying one affine `pdvar` expression on either side, with the written matrix order preserved.

Remarks

Incompatible affine operands raise `pdvar:InvalidAddition` or `pdvar:InvalidSubtraction`. Bad dimensions, two decision-bearing factors, nonlinear symbolic data, or unsupported rate combinations raise `pdvar:InvalidMultiplication`. Incompatible parameter bounds raise `pdvar:MixedGrid`; a function-only `pdmat` operand raises `pdvar:FunctionOnlyAlgebra`.

5.8 Reshape and Transform Decision Matrices

Every method below is inherited from the centralized `pdbase` implementation and transforms each local affine YALMIP payload while the `pdvar` reconstruction hook preserves the physical grid, degree, class, decision metadata, and compatible rate rows. The examples report stable classes, dimensions, and metadata rather than YALMIP's internal variable numbers.

5.8.1 Shape, Equality, And MATLAB Protocols

Shape queries describe the matrix payload. `isequal` is an exact identity check: grid, degree, matrix size, metadata, and local symbolic payloads must already match; it does not merge or elevate operands.

Syntax

```
sz = size(P)
d = size(P, dim)
n = height(P)
n = width(P)
n = length(P)
n = numel(P)
n = ndims(P)
tf = isequal(P, Q, ...)
idx = end(P, k, n)
n = numArgumentsFromSubscript(P, S, ctx)
```

Examples and output

```
yalmip('clear')
P = pdvar(2, 3, {[0 1]}, "full");
matrixSize = size(P)
rowCount = height(P)
columnCount = width(P)
longestDimension = length(P)
elementCount = numel(P)
dimensionCount = ndims(P)
equalsUnaryPlus = isequal(P,+P)
```

```
matrixSize = 1×2 double
          2      3
```

```
rowCount = 2
columnCount = 3
longestDimension = 3
elementCount = 6
dimensionCount = 2
equalsUnaryPlus =
    logical
         1
```

`end`, `subsref`, `subsasgn`, and `numArgumentsFromSubscript` support the two-index and dot-access

forms shown in section 5.9; they do not expose physical-cell storage.

5.8.2 Transpose And Conjugate Transpose

Both forms transpose every local affine payload without breaking the YALMIP coefficient graph.

Syntax

```
T = transpose(P)
H = ctranspose(P)
T = P.'
H = P'
```

Examples and output

```
yalmp('clear')
P = pdvar(2, 3, {[0 1]}, "full");
T = P.';
H = P';
transposeSize = size(T)
conjugateTransposeSize = size(H)
transposeClass = class(T)
conjugateTransposeClass = class(H)
```

```
transposeSize = 1×2 double
    3     2
conjugateTransposeSize = 1×2 double
    3     2
transposeClass = 'pdvar'
conjugateTransposeClass = 'pdvar'
```



5.8.3 Vectorization, Reshape, And Squeeze

`vec` uses MATLAB column-major order. `reshape` accepts two positive dimensions with at most one inferred `[]` dimension and preserves the element count. `squeeze` retains the two-dimensional package convention.

Syntax

```
V = vec(P)
R = reshape(P, [m n])
R = reshape(P, m, n)
S = squeeze(P)
```


Examples and output

```
yalmip('clear')
P = pdvar(2, 3, {[0 1]}, "full");
V = vec(P);
R = reshape(P, [3 2]);
S = squeeze(P);
vectorSize = size(V)
reshapeSize = size(R)
squeezeSize = size(S)
```

```
vectorSize = 1×2 double
         6         1
reshapeSize = 1×2 double
         3         2
squeezeSize = 1×2 double
         2         3
```

Malformed or size-changing reshape requests raise `pdvar:InvalidReshape`.

5.8.4 Diagonals And Trace

`diag` extracts a diagonal or constructs a diagonal matrix from a vector; the selected offset may not be empty. `trace` returns a scalar decision expression.

Syntax

```
d = diag(P)
d = diag(P, k)
D = diag(v)
D = diag(v, k)
t = trace(P)
```

Examples and output

```
yalmp('clear')
P = pdvar(3, {[0 1]}, "full");
d = diag(P);
D = diag(d);
t = trace(P);
diagonalSize = size(d)
rebuiltSize = size(D)
traceSize = size(t)
```

```
diagonalSize = 1×2 double
      3      1
rebuiltSize = 1×2 double
      3      3
traceSize = 1×2 double
      1      1
```

Invalid offsets or empty selections raise `pdvar:InvalidDiag`.

5.8.5 Triangular Parts And Reductions

Triangular methods zero the complementary part of every coefficient. Reductions act on matrix dimensions; "all" is available for `sum` and `mean`.

Syntax

```
L = tril(P)
L = tril(P, k)
U = triu(P)
U = triu(P, k)
S = sum(P)
S = sum(P, dim)
S = sum(P, vecdim)
S = sum(P, "all")
M = mean(P)
M = mean(P, dim)
M = mean(P, vecdim)
M = mean(P, "all")
C = cumsum(P)
C = cumsum(P, dim)
C = cumsum(P, direction)
C = cumsum(P, dim, direction)
```

First form the two triangular views.

Examples and output

```
yalmip('clear')
P = pdvar(3, {[0 1]}, "full");
L = tril(P);
U = triu(P);
lowerSize = size(L)
upperSize = size(U)
degrees = [L.Degree U.Degree]
```

```
lowerSize = 1×2 double
    3     3
upperSize = 1×2 double
    3     3
degrees = 1×2 double
    1     1
```

Then reduce a rectangular expression.

```
yalmip('clear')
P = pdvar(2, 3, {[0 1]}, "full");
rowSum = sum(P, 2);
allMean = mean(P, "all");
running = cumsum(P, 2);
rowSumSize = size(rowSum)
allMeanSize = size(allMean)
runningSize = size(running)
```

```
rowSumSize = 1×2 double
    2     1
allMeanSize = 1×2 double
    1     1
runningSize = 1×2 double
    2     3
```

`vecdim` is a nonempty vector of unique positive integer dimensions; dimensions above two are singleton no-ops. `direction` is "forward" or "reverse", case-insensitively. A direction supplied without a dimension uses the first nonsingleton matrix dimension, and `cumsum` along a dimension above two returns the same symbolic payloads. Invalid triangular offsets raise `pdvar:InvalidTriangularPart`. Invalid reduction calls raise `pdvar:InvalidSum`, `pdvar:InvalidMean`, or `pdvar:InvalidCumsum`; missing-value and native-output flags are not supported.

5.8.6 Reordering And Rotation

These methods rearrange matrix entries inside every symbolic coefficient; they do not reverse the parameter grid.

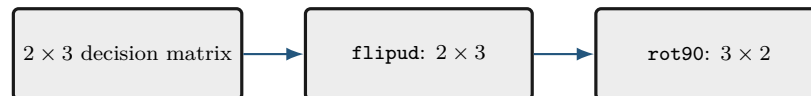
Syntax

```
Q = flip(P)
Q = flip(P, dim)
Q = flipud(P)
Q = fliplr(P)
Q = rot90(P)
Q = rot90(P, k)
```

Examples and output

```
yalmp('clear')
P = pdvar(2, 3, {[0 1]}, "full");
F = flipud(P);
R = rot90(P);
flippedSize = size(F)
rotatedSize = size(R)
flippedClass = class(F)
rotatedClass = class(R)
```

```
flippedSize = 1×2 double
             2     3
rotatedSize = 1×2 double
             3     2
flippedClass = 'pdvar'
rotatedClass = 'pdvar'
```



Invalid calls raise `pdvar:InvalidFlip` or `pdvar:InvalidRot90`.

5.8.7 Concatenation And Block Assembly

Assembly accepts compatible `pdvar`, coefficient-backed `pdmat`, affine `sdpvar`, and finite real numeric blocks. It aligns grids and degrees while preserving an affine expression. `cat` supports only matrix dimensions 1 and 2.

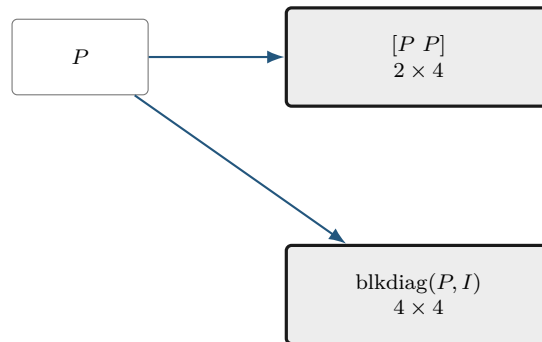
Syntax

```
H = horzcat(P, Q, ...)
V = vertcat(P, Q, ...)
H = [P, Q]
V = [P; Q]
C = cat(dim, P, Q, ...)
D = blkdiag(P, Q, ...)
```

Examples and output

```
yalmip('clear')
P = pdvar(2, {[0 1]}, "full");
H = [P, P];
V = [P; P];
D = blkdiag(P, eye(2));
horizontalSize = size(H)
verticalSize = size(V)
blockDiagonalSize = size(D)
```

```
horizontalSize = 1×2 double
    2    4
verticalSize = 1×2 double
    4    2
blockDiagonalSize = 1×2 double
    4    4
```



Incompatible blocks or rate metadata raise `pdvar:InvalidConcatenation`; unsupported dimensions raise

`pdvar:UnsupportedCatDimension`; invalid block-diagonal inputs raise `pdvar:InvalidBlkdiag`. Incompatible bounds raise `pdvar:MixedGrid`; function-only known blocks raise `pdvar:FunctionOnlyAlgebra`.

5.8.8 Repetition

`repmat` tiles the matrix payload while retaining the parameter metadata. It accepts a positive scalar, two positive counts, or a two-element count vector.

Syntax

```
Q = repmat(P, m)
Q = repmat(P, m, n)
Q = repmat(P, [m n])
```

Examples and output

```
yalmip('clear')
P = pdvar(1, 2, {[0 1]}, "full");
Q = repmat(P, [2 2]);
repeatedSize = size(Q)
repeatedDegree = Q.Degree
containsDecision = Q.ContainsDecision
```

```
repeatedSize = 1×2 double
              2      4
repeatedDegree = 1
containsDecision =
    logical
         1
```

Invalid repetition shapes raise `pdvar:InvalidRepmat`.

5.9 Index and Assign `pdvar` Entries

Parenthesis indexing selects a matrix block from every symbolic coefficient. Assignment writes a compatible numeric, `pdvar`, coefficient-backed `pdmat`, or affine `sdpvar` block into that position. It does not select one physical cell.

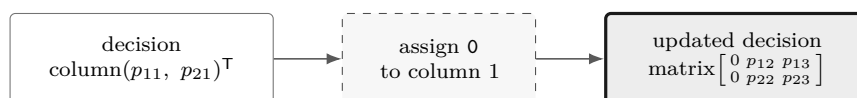
Syntax

```
Q = P(rows, cols)
Q = P(1:end, end)
value = P.PropertyName
P(rows, cols) = numericValue
P(rows, cols) = Q
P(rows, cols) = A
P(rows, cols) = X
```

Examples and output

```
yalmip('clear')
P = pdvar(2, 3, {[0 1]}, "full");
lastCol = P(:, end);
P(:, 1) = 0;
selectedSize = size(lastCol)
assignedSize = size(P)
assignedClass = class(P)
assignedDegree = P.Degree
containsDecision = P.ContainsDecision
```

```
selectedSize = 1×2 double
             2     1
assignedSize = 1×2 double
             2     3
assignedClass = 'pdvar'
assignedDegree = 1
containsDecision =
    logical
         1
```



Invalid selectors raise `pdvar:InvalidSubscript`. Linear indexing, deletion, or non-parenthesis assignment raises `pdvar:UnsupportedAssignment`; incompatible or nonlinear right-hand sides raise `pdvar:InvalidAssignment`.

5.10 value

Syntax

```
A = value(P)
rows = value(rhodiff(P))
```

`value` converts assigned `pdvar` coefficients into known, coefficient-backed `pdmat` data. An ordinary expression returns one `pdmat`, preserving its grid, matrix size, degree, local coefficient order, and continuity metadata. A rate-vertex expression created by `rhodiff` returns a 1-by- 2^ℓ cell array of `pdmat` objects in the deterministic lower/upper vertex order induced by applying `helper.combRows` to `num2cell(RateBounds,2)'`. The returned `pdmat` objects do not carry rate metadata. Every symbolic coefficient must have an assigned finite real YALMIP value; otherwise the method raises `pdvar:UnassignedValue`.

Examples and output

```
yal mip('clear')
P = pdvar(1, [0 1], Degree=2);
c = P.coeffs(1);
assign([c{:}], [1 2 4]);
A = value(P);
assignedCoefficients = cell2mat(A.coeffs(1))
```

```
assignedCoefficients = 1×3 double
    1     2     4
```

5.11 Comparisons To pdlmi

The `le`, `ge`, and `eq` overloads implement `<=`, `>=`, and `==`. Inequalities select either semidefinite or entry-wise meaning once for the complete original residual; equality is always direct coefficient equality.

Syntax

```
C = P <= 0
C = P >= 0
C = lhs <= rhs
C = lhs >= rhs
C = lhs == rhs
```

Arguments

- `rhs` may be compatible numeric, coefficient-backed `pdmat`, affine `sdpvar`, or `pdvar` data. Ordinary operands use the same grid refinement, degree alignment, and promotion as subtraction.
- For `<=` and `>=`, every coefficient in every physical cell and rate row is scanned before assembly. The complete relation is semidefinite only if every original coefficient is square Hermitian. Numeric Hermitian testing uses the inclusive rule $\|A - A^T\|_\infty \leq 10^{-10}$; symbolic coefficients use YALMIP's `ishermitian`. If any coefficient fails, the complete inequality is entry-wise and warning `pdlmi:ElementwiseInequality` is issued once. Modes are never mixed coefficient by coefficient or entry by entry.
- `==` accepts rectangular and nonsymmetric residuals without that warning. It equates corresponding coefficient entries directly. Ordinary rows compare with ordinary rows; derivative rows compare only with compatible derivative rows.
- `C` is a `pdlmi` wrapper. A comparison assembles constraints but does not solve an optimization problem.

Examples and output

Comparison creates a `pdlmi` wrapper with one stored constraint for each physical cell and local Bernstein coefficient.

```
yalmip('clear')
P = pdvar(2, {[0 0.5 1]}, "symmetric");
Cneg = P <= 0;
negativeWrapperClass = class(Cneg)
negativeConstraintCount = numel(Cneg.Constraints)
Cpos = P >= 0;
positiveWrapperClass = class(Cpos)
positiveConstraintCount = numel(Cpos.Constraints)
```

```
negativeWrapperClass = 'pdlmi'
negativeConstraintCount = 4
positiveWrapperClass = 'pdlmi'
positiveConstraintCount = 4
```

A rectangular residual selects entry-wise inequality globally. Equality is entry-wise by definition and is warning-free.

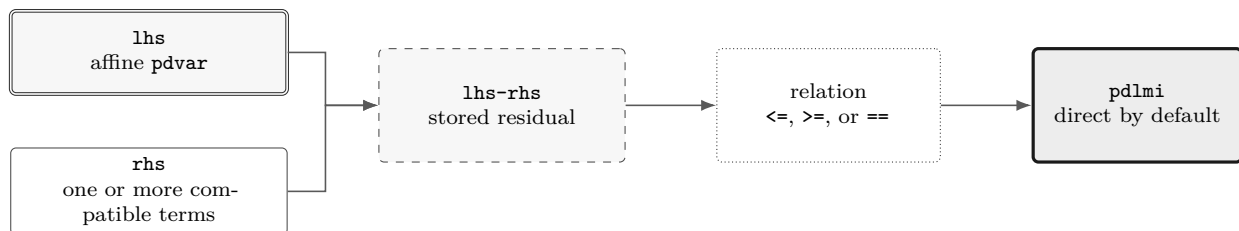
```
yalmip('clear')
V = pdvar(3, 2, {[0 1]}, "full");
warnId = "pdlmi:ElementwiseInequality";
warnState = warning("off", warnId);
restoreWarning = onCleanup(@() warning(warnState));
Centry = V >= 0;
Ceq = V == sdpvar(3, 2, 'full');
entrywiseConstraintGroups = numel(Centry.Constraints)
equalityConstraintGroups = numel(Ceq.Constraints)
identity = V == V;
identityConstraints = identity.toYalmip()
clear restoreWarning
```

```
entrywiseConstraintGroups = 2
equalityConstraintGroups = 2
```

```
identityConstraints =
```

```
[]
```

Each group above corresponds to one local coefficient and contains six scalar entries in MATLAB column-major order. Under Pólya, Putinar, SparseFullBox, or FullBox, each entry of an entry-wise inequality receives an independent scalar certificate. Equality is direct-only: constructor certificate options and `applyPolya`, `applyPutinar`, `applySparseFullBoxPreorder`, or `applyFullBoxPreorder` raise `pdlmi:UnsupportedEqualityCertificate` before validating an increment, width, or order. Mixed ordinary/derivative row kinds raise `pdvar:InvalidEqualityRows`. Use overloaded `==` to build a modeling equality; use `isequal` only for MATLAB structural comparison.



5.12 Boundaries and Related APIs

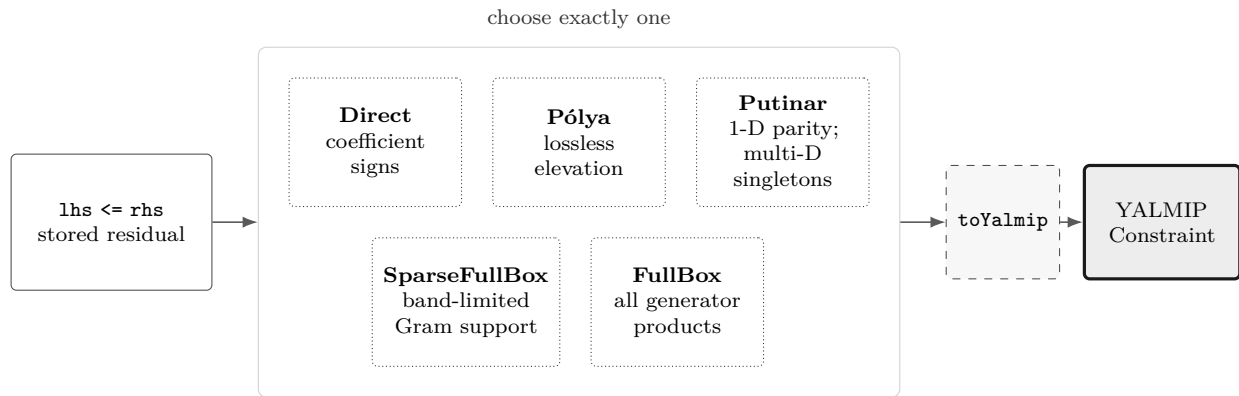
1. Constructor-created `pdvar` objects support every finite nonnegative integer degree. Higher degree increases the number of Bernstein control matrices per cell.
2. Products may contain decision variables on at most one side; nonlinear decision products such as `P * Q` are outside this slice.
3. Function-only `pdmat` operands cannot enter `pdvar` algebra unless they were constructed with explicit Bernstein evidence.
4. `rhodiff` of an existing rate-vertex derivative expression is not part of the current public workflow.

Related APIs. `pdvar`, `rhodiff`, `bernsteinTable`, `mtimes`, `pdlmi`, `pdmat`.

Chapter 6

pdlmi: Finite Certificate Assembly

`pdlmi` stores finite certificates assembled from `pdvar` expressions or coefficient-backed `pdmatrix` inequalities. Square Hermitian inequality families use semidefinite constraints; other inequalities use entry-wise scalar constraints; `pdvar` equality uses direct coefficient equations. Direct assembly is the default, while cell-local Pólya elevation, a dimension-aware Putinar selector, `FullBox` with band-limited Gram support (`SparseFullBox`), and the dense full-box preordering (`FullBox`) are explicit inequality alternatives. It is the package boundary between Bernstein objects and ordinary YALMIP solver calls.



The five certificate paths are alternatives. Selecting Pólya, Putinar, SparseFullBox, or FullBox rebuilding replaces, rather than compounds, the previous selection.

6.1 API Map

Task	Entry
Construct wrapper	<code>pdlmi</code> , <code><=/>=</code>
Count constraints	<code>direct coefficient constraints</code>
Select Pólya elevation	<code>UsePolya</code> , <code>PolyaDegree</code> , and <code>applyPolya</code>
Select Putinar certificate	<code>UsePutinar</code> , <code>PutinarOrder</code> , and <code>applyPutinar</code>
Select SparseFullBox	<code>UseSparseFullBoxPreorder</code> , <code>SparseFullBoxOrder</code> , <code>BandWidth</code> , and <code>applySparseFullBoxPreorder</code>
Select FullBox	<code>UseFullBoxPreorder</code> , <code>FullBoxOrder</code> , and <code>applyFullBoxPreorder</code>
Export to YALMIP	<code>toYalmip</code>
Solve	<code>solver-facing use</code> , <code>Examples</code>

6.2 Inspect Certificate State

All `pdlmi` properties have private set access and are readable with dot notation. `Constraints` contains the assembled YALMIP entries; `Residual` and `Relation` retain the original comparison for rebuilding. The remaining properties record the selected certificate state.

Property	Meaning
<code>Constraints</code>	Stored finite YALMIP constraint entries.
<code>Residual, Relation</code>	Original <code>pdvar</code> residual and one of " <code><=</code> ", " <code>>=</code> ", or " <code>==</code> ".
<code>UsePolya, PolyaDegree</code>	Pólya selector and nonnegative degree increment.
<code>UsePutinar, PutinarOrder</code>	Putinar selector and absolute Gram order.
<code>UseSparseFullBoxPreorder, SparseFullBoxOrder, BandWidth</code>	<code>SparseFullBox</code> selector, absolute Gram order, and band width; intermediate support is implemented by overlapping tensor-window PSD cliques.
<code>UseFullBoxPreorder, FullBoxOrder</code>	Full-box selector and absolute Gram order.

For example, `C.Relation`, `C.Residual.Degree`, and `C.UseSparseFullBoxPreorder` inspect a wrapper. `Apply` methods return a rebuilt value; assigning these properties is unsupported. An intermediate `SparseFullBox` result stores all three sparse properties. A width-one request canonicalizes to actual Direct state, where those fields are `false`, zero, and zero; a request with `BandWidth >= SparseFullBoxOrder + 1` canonicalizes to actual `FullBox` state, where only `UseFullBoxPreorder/FullBoxOrder` record the selection.

6.3 Construct pdlmi Comparisons

Syntax

```
C = pdlmi(expr, relation)
C = pdlmi(expr, "==")
C = pdlmi(expr, relation, "UsePolya")
C = pdlmi(expr, relation, UsePolya=true, PolyaDegree=d)
C = pdlmi(expr, relation, "UsePolya", "PolyaDegree", d)
C = pdlmi(expr, relation, "UsePutinar")
C = pdlmi(expr, relation, UsePutinar=true, PutinarOrder=r)
C = pdlmi(expr, relation, PutinarOrder=r)
C = pdlmi(expr, relation, "UseSparseFullBoxPreorder")
C = pdlmi(expr, relation, UseSparseFullBoxPreorder=true)
C = pdlmi(expr, relation, BandWidth=b)
C = pdlmi(expr, relation, SparseFullBoxOrder=r)
C = pdlmi(expr, relation, UseSparseFullBoxPreorder=true, ...
    BandWidth=b, SparseFullBoxOrder=r)
C = pdlmi(expr, relation, "UseFullBoxPreorder")
C = pdlmi(expr, relation, UseFullBoxPreorder=true)
C = pdlmi(expr, relation, FullBoxOrder=r)
C = pdlmi(expr, relation, UseFullBoxPreorder=true, FullBoxOrder=r)
C = pdlmi(expr, relation, UseFullBoxPreorder=false, FullBoxOrder=r)
C = pdlmi(expr, relation, ValidationMode=mode)
C = lhs <= rhs
C = lhs >= rhs
C = lhs == rhs
```

Arguments

- **expr** is a **pdvar** residual, or a coefficient-backed **pdmat** residual for "**<=**" or "**>=**"; **relation** is exactly "**<=**", "**>=**", or "**==**". Inequalities are classified globally from the original coefficient family. **pdvar** equality is entry-wise direct coefficient assembly and accepts rectangular or nonsymmetric residuals. **pdmat** equality is instead the scalar logical comparison **A==B** and never creates a **pdlmi**.
- **UsePolya** is a logical scalar option with default **false**. Its bare form, "**UsePolya**", and **UsePolya=true** without a degree select increment one.
- **PolyaDegree=d** is a finite nonnegative integer scalar. It elevates the residual by **d** in every parameter direction before assembling constraints. Supplying it without **UsePolya** enables Pólya and issues **pdlmi:ImplicitUsePolya**; **UsePolya=false** with positive **d** raises **pdlmi:ConflictingPolyaOptions**.
- **UsePutinar** is a logical scalar option with default **false**. Its bare form and the explicit value **true** select the dimension-appropriate Putinar certificate at the smallest admissible order.
- **PutinarOrder=r** is a finite nonnegative integer scalar and is an absolute Bernstein Gram order, not a degree increment. For assembled residual degree M , its default and minimum are $\lfloor M/2 \rfloor$ in one parameter and $\lceil M/2 \rceil$ in two or more parameters. Supplying the order without **UsePutinar**

enables Putinar assembly. Pairing `UsePutinar=false` with any explicit order, including zero, raises `pdlmi:ConflictingPutinarOptions`.

- `UseSparseFullBoxPreorder` is a logical scalar option with default `false`. Its bare form and the explicit value `true` select `SparseFullBox` with default `BandWidth=2`; `BandWidth` or `SparseFullBoxOrder` alone also enables this family. Explicit `UseSparseFullBoxPreorder=false` conflicts with either sparse parameter.
- `BandWidth=b` is a finite positive integer band width. Intermediate widths restrict the `FullBox` Gram support and are realized by overlapping axis-aligned tensor-window PSD clique blocks. `SparseFullBoxOrder=r` is a finite nonnegative absolute Gram order whose default and minimum are $\lfloor M/2 \rfloor$ in one parameter and $\lceil M/2 \rceil$ otherwise, where $M = \mathbf{C}.\text{Residual.Degree}$. Width one canonicalizes to actual Direct state after order validation; width at least $r + 1$ canonicalizes to actual `FullBox` state; intermediate widths select `SparseFullBox` state.
- `UseFullBoxPreorder` is a logical scalar option with default `false`. Its bare form and the explicit value `true` select full-box assembly at the smallest admissible order.
- `FullBoxOrder=r` is a finite nonnegative integer scalar and is an absolute order, not a degree increment. Supplying it without `UseFullBoxPreorder` enables full-box assembly. When `FullBox` is enabled, the minimum is $\lfloor M/2 \rfloor$ for a one-parameter residual of degree M , and $\lceil M/2 \rceil$ for two or more parameters. Explicitly pairing `UseFullBoxPreorder=false` with `FullBoxOrder=r` suppresses that automatic selection: `C` remains direct and records `r` only as inspectable metadata; the order is not applied to assembly.
- Pólya, Putinar, `SparseFullBox`, and `FullBox` selection are mutually exclusive and apply only to inequalities. Any recognized certificate option on an equality raises `pdlmi:UnsupportedEqualityCertificate` before width or order validation. Enabling more than one inequality family raises `pdlmi:ConflictingRelaxations`.
- `ValidationMode=mode` accepts scalar text `"fast"` or `"strict"`, case-insensitively, and defaults to `"fast"`. It is transient and not stored. Every `apply...` call accepts its own trailing `ValidationMode`; a constructor selection is not inherited by later calls.
- `C` is direct coefficient-wise assembly unless one opt-in inequality mode is selected. Comparisons such as `lhs >= rhs` are the ordinary user-facing entry point and accept compatible numeric, `pdmat`, affine `sdpcvar`, and `pdvar` right-hand sides.

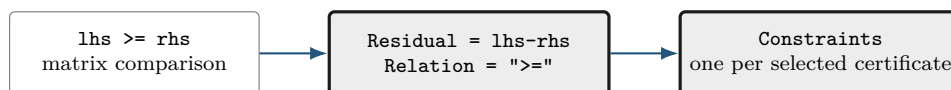
The inspectable properties `Constraints`, `Residual`, and `Relation` expose the assembled finite constraints and the original comparison data used for rebuilding. `UsePolya/PolyaDegree`, `UsePutinar/PutinarOrder`, `UseSparseFullBoxPreorder/SparseFullBoxOrder/BandWidth`, and `UseFullBoxPreorder/FullBoxOrder` report the selected mode and parameters. Their set access is private. Comparison-created direct objects have every selector false and every numeric certificate property zero. The full-box explicit-false combination described above is the exception: it retains `FullBoxOrder=r` while `UseFullBoxPreorder` remains false and the stored constraints remain direct. Putinar and `SparseFullBox` deliberately have no corresponding exception.

Examples and output

The direct constructor is equivalent to the comparison path when the relation and options are explicit.

```
yal mip('clear')
P = pdvar(2, {[0 1]}, "symmetric");
C = pdlmi(P, ">=", UsePolya=false, PolyaDegree=0);
wrapperClass = class(C)
constraintCount = numel(C.Constraints)
usePolya = C.UsePolya
polyaDegree = C.PolyaDegree
usePutinar = C.UsePutinar
putinarOrder = C.PutinarOrder
useFullBox = C.UseFullBoxPreorder
fullBoxOrder = C.FullBoxOrder
useSparseFullBox = C.UseSparseFullBoxPreorder
sparseFullBoxOrder = C.SparseFullBoxOrder
bandWidth = C.BandWidth
```

```
wrapperClass = 'pdlmi'
constraintCount = 2
usePolya =
    logical
    0
polyaDegree = 0
usePutinar =
    logical
    0
putinarOrder = 0
useFullBox =
    logical
    0
fullBoxOrder = 0
useSparseFullBox =
    logical
    0
sparseFullBoxOrder = 0
bandWidth = 0
```



Remarks

1. A residual outside the supported `pdvar`/`pdmatrix` forms raises `pdlmi:InvalidExpression`; a function-only `pdmatrix` raises `pdlmi:MissingCoefficientEvidence`; `pdlmi(A,"==")` for `pdmatrix` raises `pdlmi:UnsupportedPdmatrixEquality`; an unsupported relation raises `pdlmi:InvalidRelation`.
2. Malformed, unknown, or duplicate options raise `pdlmi:InvalidOptions`, `pdlmi:UnknownOption`, or `pdlmi:DuplicateOption`; invalid validation modes raise `pdlmi:InvalidValidationMode`; nonlogical selectors use their corresponding `InvalidUse...` identifier.

3. Malformed Pólya increments, Putinar orders, sparse orders, widths, and full-box orders raise, respectively, `pdlmi:InvalidPolyaDegree`, `pdlmi:InvalidPutinarOrder`, `pdlmi:InvalidSparseFullBoxOrder`, `pdlmi:InvalidBandWidth`, and `pdlmi:InvalidFullBoxOrder`. An integer below an active Gram-family minimum raises `pdlmi:PutinarOrderTooLow`, `pdlmi:SparseFullBoxOrderTooLow`, or `pdlmi:FullBoxOrderTooLow`.
4. Explicit false plus a Putinar order raises `pdlmi:ConflictingPutinarOptions`; explicit false plus a sparse order or width raises `pdlmi:ConflictingSparseFullBoxOptions`; explicit false plus a positive Pólya increment raises `pdlmi:ConflictingPolyaOptions`. The full-box explicit-false form remains direct and retains the supplied order as metadata.
5. An inequality with any nonsquare or non-Hermitian original coefficient is assembled entry-wise and emits `pdlmi:ElementwiseInequality`; it is not rejected. Numeric Hermitian testing accepts equality at the inclusive 10^{-10} infinity-norm tolerance.
6. Equality is direct-only. Every `apply...` method raises `pdlmi:UnsupportedEqualityCertificate` before validating its supplied increment, width, or order.

6.4 Assemble Direct Coefficient Constraints

For a `pdvar` residual, including equality behavior, each physical cell, local Bernstein coefficient, and rate-vertex row if present creates one stored YALMIP constraint entry. For an ordinary expression with N_c physical cells and N_b Bernstein coefficients per cell, the number of entries is $N_c N_b$; with N_v rate vertices it is $N_c N_b N_v$. A semidefinite-mode entry is one matrix constraint. An entry-wise-mode entry is the column-major vector of scalar matrix entries compared independently. An equality entry is the corresponding direct coefficient equation and may be rectangular. For a coefficient-backed known `pdmat` inequality, Direct and Pólya assembly instead scan the complete numeric coefficient family and store exactly one logical cell containing the global certificate; `toYalmip` exports that state as one scalar logical.

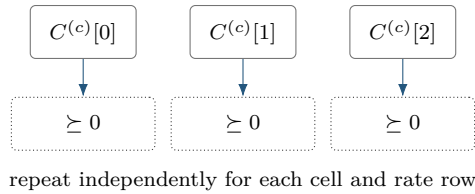
The idea is the Bernstein convex-hull property. After orienting the original residual $\mathcal{F}$ as the sign-normalized positive target S , suppress the already-stacked rate-row index and write

$$S^{(\mathbf{c})}(\boldsymbol{\alpha}) = \sum_{\mathbf{i}} B_{\mathbf{i}}^M(\boldsymbol{\alpha}) C^{(\mathbf{c})}[\mathbf{i}],$$

where M is the assembled residual degree. The semidefinite direct certificate requires $C^{(\mathbf{c})}[\mathbf{i}] \succeq 0$ for every physical cell $\mathbf{c}$ and local label $\mathbf{i}$, repeated independently for every stored rate row. This is sufficient, not necessary.

For an entry-wise inequality, replace each matrix coefficient by $\mathbb{C}(\cdot)$ and impose the oriented scalar sign on every component. The global classification is made before Pólya elevation or Gram assembly, so every coefficient and every certificate in one wrapper uses the same mode. Putinar, `SparseFullBox`, and `FullBox` create one independent scalar Gram certificate for each column-major entry. Direct equality simply imposes $\mathbb{C}(\cdot) == 0$; it does not use a positivity certificate or an implicit tolerance.

The diagram specializes to one parameter, degree two, and one ordinary (non-rate) row; therefore c is a scalar one-based cell index and $i \in \{0, 1, 2\}$.



Syntax

```
C = pdlmi(expr, relation)
C = lhs <= rhs
C = lhs >= rhs
C = lhs == rhs
```

The constructor and comparison forms above create direct coefficient constraints when no inequality certificate is selected. For a `pdvar` residual, each stored coefficient is constrained in the oriented positive-semidefinite or entry-wise mode; equality constrains each coefficient entry directly to zero. Its stored-constraint count is therefore the number of physical cells times the number of local Bernstein coefficients, with an additional factor for active rate vertices. As stated above, a coefficient-backed known `pdmatrix` inequality instead stores exactly one logical cell containing the global numeric certificate.

Examples and output

```
yal mip('clear')
P = pdvar(1, {[0 1]}, Degree=2);
C = P >= 0;

constraintCount = numel(C.Constraints)
usePolya = C.UsePolya
usePutinar = C.UsePutinar
useSparseFullBox = C.UseSparseFullBoxPreorder
useFullBox = C.UseFullBoxPreorder
```

```
constraintCount = 3
usePolya =
    logical
    0
usePutinar =
    logical
    0
useSparseFullBox =
    logical
    0
useFullBox =
    logical
    0
```

Here the degree-two, one-parameter residual has three local Bernstein coefficients, so direct assembly stores three coefficient constraints. The wrapper remains direct because no certificate selector was enabled; a call such as `C.applyPolya(1)` returns a new wrapper with an elevated coefficient family and leaves `C` unchanged.

6.5 Select Pólya Assembly with applyPolya

With `UsePolya=true`, `pdlmi` first elevates the stored residual by `PolyaDegree` in every parameter direction. For a `pdvar` residual it then constrains every elevated local coefficient and stored rate row: if the assembled residual degree is M , the count is $N_c(M+d+1)^\ell$ for ordinary expressions and $N_c(M+d+1)^\ell N_v$ for rate-row expressions. A coefficient-backed known `pdmatrix` residual instead stores exactly one logical cell containing the global numeric certificate, including after Pólya elevation. An increment $d = 0$ is valid and has the same coefficient count as `Direct`. For an entry-wise `pdvar` inequality, elevation preserves the global mode and the oriented sign is applied independently to every column-major scalar entry of each elevated coefficient. Equality wrappers reject this method.

Syntax

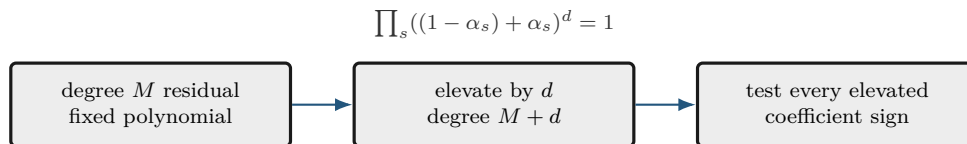
```
Cpolya = C.applyPolya()
Cpolya = C.applyPolya(d)
Cpolya = C.applyPolya(..., ValidationMode=mode)
```

`applyPolya` is a value-class method: it returns a new `pdlmi`, leaves `C` unchanged, and rebuilds from `C.Residual`. Selecting a new increment therefore replaces rather than compounds a previous selection. The no-argument form uses one; an invalid increment raises `pdlmi:InvalidPolyaDegree`.

Examples and output

```
yal mip('clear')
P = pdvar(1, {[0 1]}, Degree=1);
direct = P >= 0;
polya = direct.applyPolya(2);
directDegree = direct.PolyaDegree
usePolya = polya.UsePolya
polyaDegree = polya.PolyaDegree
constraintCount = numel(polya.Constraints)
```

```
directDegree = 0
usePolya =
    logical
         1
polyaDegree = 2
constraintCount = 4
```



Pólya degree is a uniform representation increment, not a new decision degree, a grid refinement, or a Gram order.

The mathematical background and the limits of fixed-increment coefficient positivity are developed in Chapter B.

6.6 Select Putinar Assembly with `applyPutinar`

For an entry-wise inequality, the constructions below are applied independently to each scalar matrix entry in MATLAB column-major order. Gram variables are not shared between entries. Equality wrappers reject Putinar selection.

The Putinar selector is dimension-aware. With one parameter it deliberately uses the same exact even/odd Markov–Lukács assembly as `applyFullBoxPreorder`. For selected absolute order r , the even form matches at degree $2r$,

$$S = \mathcal{Z}_r^\top Q_0 \mathcal{Z}_r + \alpha(1 - \alpha) \mathcal{Z}_{r-1}^\top Q_1 \mathcal{Z}_{r-1},$$

where the second block is omitted at $r = 0$. The odd form matches at degree $2r + 1$,

$$S = (1 - \alpha) \mathcal{Z}_r^\top Q_L \mathcal{Z}_r + \alpha \mathcal{Z}_r^\top Q_U \mathcal{Z}_r.$$

All present Gram matrices are positive semidefinite. The one-parameter default and minimum are $r = \lfloor M/2 \rfloor$ for assembled residual degree M .

For two or more parameters, the selector instead uses the fixed-order, box-specific singleton-generator Bernstein–Gram quadratic module. After normalizing the relation so that $S(\boldsymbol{\alpha})$ is positive semidefinite, it represents

$$S(\boldsymbol{\alpha}) = S_0(\boldsymbol{\alpha}) + \sum_{s=1}^{\ell} \alpha_s(1 - \alpha_s) S_s(\boldsymbol{\alpha}).$$

Only singleton box generators appear in this multivariate branch: products such as $\alpha_1(1 - \alpha_1)\alpha_2(1 - \alpha_2)$ belong to the full-box preordering, not this module.

Syntax

```
Cputinar = C.applyPutinar()
Cputinar = C.applyPutinar(r)
Cputinar = C.applyPutinar(..., ValidationMode=mode)
```

Here $\mathbf{r}$ is an absolute Bernstein Gram order. The no-argument form uses the dimension-dependent minima above; an explicit order must satisfy the same applicable bound. In the multivariate branch, the unweighted block S_0 uses tensor basis degree $r\mathbf{1}$, while S_s uses degree $r\mathbf{1} - \mathbf{e}_s$. A nominal block with a negative component is omitted, which occurs for every multiplier at $r = 0$. The target is degree-elevated without changing its represented polynomial, and the multivariate Gram sum is matched exactly at tensor degree $2r$.

Examples and output

```
yalmip('clear')
P = pdvar(1, {[0 1]}, Degree=2);
direct = P >= 0;
putinar = direct.applyPutinar();
directUsesPutinar = direct.UsePutinar
directOrder = direct.PutinarOrder
directConstraintCount = numel(direct.Constraints)
selectedUsesPutinar = putinar.UsePutinar
```

```
selectedOrder = putinar.PutinarOrder
selectedConstraintCount = numel(putinar.Constraints)
```

```
directUsesPutinar =
    logical
    0
directOrder = 0
directConstraintCount = 3
selectedUsesPutinar =
    logical
    1
selectedOrder = 1
selectedConstraintCount = 5
```

For this one-parameter scalar degree-two cell, order one uses the exact even Markov–Lukács form: an unweighted Gram block of dimension two and one generator-weighted block of dimension one, followed by three exact coefficient identities. The result is identical to `direct.applyFullBoxPreorder()`. Each physical cell and each active rate-vertex row receives independent copies of these PSD blocks and identities.

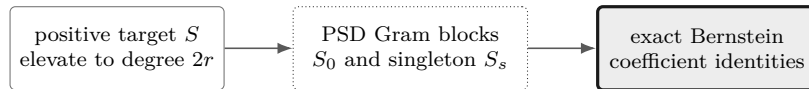
Remarks

For the multivariate backend, the smallest useful worked case is $\ell = 2$. Let $g_s = \alpha_s(1 - \alpha_s)$. At absolute order r , Putinar uses exactly the singleton-generator module

$$S = S_0 + g_1 S_1 + g_2 S_2,$$

where S_0 has Bernstein basis degree (r, r) , S_1 has degree $(r - 1, r)$, and S_2 has degree $(r, r - 1)$. Each S_j is a PSD Gram form. MATLAB elevates the sign-normalized target and all present terms to tensor degree $(2r, 2r)$, then creates one exact coefficient match per local Bernstein label. These PSD blocks and equalities are assembled independently for every physical cell, active rate row, and scalar matrix entry; they are never shared across those axes.

The diagram below specializes to the even one-parameter example above.



The method changes the finite certificate, not the stored residual, decision degree, or physical grid.

`applyPutinar` is a value-class method: it returns a new `pdlmi`, leaves `C` unchanged, and rebuilds from the original `Residual` and `Relation`. Reapplication replaces any earlier Pólya, Putinar, SparseFullBox, or FullBox selection. It adds no implicit margin and makes no solver call.

Remarks

1. Malformed orders raise `pdlmi:InvalidPutinarOrder`; an integer below $\lfloor M/2 \rfloor$ in one parameter, or below $\lceil M/2 \rceil$ in two or more parameters, raises `pdlmi:PutinarOrderTooLow`.
2. A malformed selector raises `pdlmi:InvalidUsePutinar`; explicit false plus any supplied order raises `pdlmi:ConflictingPutinarOptions`.

3. Simultaneous relaxation families raise `pdlmi:ConflictingRelaxations`; expression, relation, and option validation in section 6.3 still applies. Global inequality classification is unchanged: the family is semidefinite only when every original coefficient across all cells and rate rows is square Hermitian. Otherwise the complete family uses independent entry-wise scalar Putinar certificates in MATLAB column-major order.

6.7 Select SparseFullBox with applySparseFullBoxPreorder

SparseFullBox is FullBox with band-limited Gram support. It uses the same one-parameter parity terms, multivariate box-generator masks, relation orientation, and exact target-coefficient identities as FullBox. The implementation realizes the restricted support by replacing each dense Gram block with free positive-semidefinite clique blocks on overlapping axis-aligned windows of the tensor Bernstein basis. This implementation mechanism changes only allowed Gram entries: the original residual $\mathcal{F}$, physical grid, decision degree m , assembled residual degree M , generator weights, and coefficient identities remain unchanged.

Syntax

```
Csp = C.applySparseFullBoxPreorder()
Csp = C.applySparseFullBoxPreorder(b)
Csp = C.applySparseFullBoxPreorder(b, r)
Csp = C.applySparseFullBoxPreorder(..., ValidationMode=mode)
```

Arguments

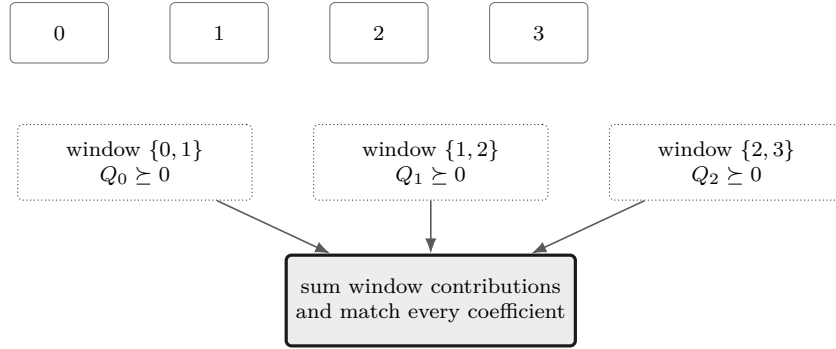
`b` is a finite positive integer band width and defaults to two; for intermediate state it is also the side length of each overlapping implementation window. `r` is an optional absolute SparseFullBox Gram order and defaults to $\lfloor M/2 \rfloor$ for a one-parameter residual of degree M , or $\lceil M/2 \rceil$ for two or more parameters. Every explicit order is validated against that same minimum before endpoint normalization.

Description

For a FullBox Gram term with tensor basis degree $\mathbf{d}$, define the complete label box $\mathcal{I}_{\mathbf{d}} = \prod_s \{0, \dots, d_s\}$. SparseFullBox sets $w_s = \min(b, d_s + 1)$ and introduces one independent PSD block for every window

$$W_{\mathbf{t}} = \prod_{s=1}^{\ell} \{t_s, \dots, t_s + w_s - 1\}, \quad 0 \leq t_s \leq d_s - w_s + 1.$$

Each block contributes only through the basis labels in its own $W_{\mathbf{t}}$; the weighted Bernstein maps of all windows and all parity/mask terms are summed and matched exactly to every coefficient of the sign-normalized target. In one parameter, $b = 2$ permits adjacent basis-label interactions and gives block-tridiagonal Gram support, while $b = 3$ gives block-pentadiagonal support. Increasing the width gives a nested sufficient hierarchy because a smaller-window PSD block can be embedded in a larger window, but feasibility need not improve strictly on any particular model.



One-dimensional width two uses every adjacent basis window; tensor problems take the Cartesian window in every coordinate rather than adjacency in a flattened label list.

Examples and output

```

yalmp('clear')
P = pdvar(1, {[0 1]}, Degree=4);
direct = P >= 0;
sparse = direct.applySparseFullBoxPreorder(2, 2);
directEndpoint = direct.applySparseFullBoxPreorder(1, 2);
denseEndpoint = direct.applySparseFullBoxPreorder(3, 2);
sparseState = [sparse.UseSparseFullBoxPreorder, ...
    sparse.SparseFullBoxOrder, sparse.BandWidth, ...
    sparse.UseFullBoxPreorder, numel(sparse.Constraints)]
directEndpointState = [directEndpoint.UseSparseFullBoxPreorder, ...
    directEndpoint.UseFullBoxPreorder, directEndpoint.BandWidth, ...
    numel(directEndpoint.Constraints)]
denseEndpointState = [denseEndpoint.UseSparseFullBoxPreorder, ...
    denseEndpoint.UseFullBoxPreorder, denseEndpoint.FullBoxOrder, ...
    numel(denseEndpoint.Constraints)]

```

```

sparseState = 1×5 double
    1    2    2    0    8
directEndpointState = 1×4 double
    0    0    0    5
denseEndpointState = 1×4 double
    0    1    2    7

```

The intermediate result uses three PSD blocks—two width-two windows for the unweighted degree-two basis and one complete width-two window for the generator-weighted degree-one basis—followed by five exact coefficient identities. Width one returns actual Direct state with no sparse metadata or Gram variables. Since $b = 3 = r + 1$, the dense endpoint returns actual FullBox state with two dense PSD blocks followed by the same five identities. These are canonical public states, not merely equivalent constraint lists.

Remarks

1. `applySparseFullBoxPreorder` is a value-class method: it returns a new wrapper, leaves the source unchanged, and rebuilds from `Residual` and `Relation`, replacing any earlier Pólya,

Putinar, SparseFullBox, or FullBox selection. It adds no positivity margin and makes no solver call.

2. Every physical cell and active rate row owns independent window Gram blocks and coefficient identities. Semidefinite mode builds matrix Gram blocks; entry-wise mode builds an independent scalar certificate for every matrix entry in MATLAB column-major order.
3. Width one canonicalizes to actual Direct state. A width satisfying $b \geq r + 1$ canonicalizes to actual FullBox state. For $1 < b < r + 1$, the result records `UseSparseFullBoxPreorder=true`, `SparseFullBoxOrder=r`, and `BandWidth=b`.
4. Invalid widths raise `pdlmi:InvalidBandWidth`. Malformed orders raise `pdlmi:InvalidSparseFullBoxOrder`; orders below the dimension-dependent minimum raise `pdlmi:SparseFullBoxOrderTooLow`. More than two positional inputs raise `pdlmi:InvalidApplyOptions`.
5. Constructor selector conflicts use `pdlmi:ConflictingSparseFullBoxOptions` or `pdlmi:ConflictingRelaxations`. Equality raises `pdlmi:UnsupportedEqualityCertificate` before width or order validation.

6.8 Select FullBox with `applyFullBoxPreorder`

The mathematical construction is a full-box preordering in the forward local coordinates $\alpha \in [0, 1]^\ell$; the software state is `FullBox`. It is a box-specific Bernstein–Gram representation, not a general Putinar certificate, general-domain SOS interface, or full-space SOS representation. It adds no implicit positivity margin. For an entry-wise inequality, every column-major scalar entry receives an independent copy of the selected Gram construction. Equality wrappers reject `FullBox` selection.

Syntax

```
Cbox = C.applyFullBoxPreorder()
Cbox = C.applyFullBoxPreorder(r)
Cbox = C.applyFullBoxPreorder(..., ValidationMode=mode)
```

The no-argument form selects the smallest admissible absolute order. The explicit form selects `r`; it does not add `r` to the residual degree. Both forms are value-class selectors: they return a new `pdlmi`, leave `C` unchanged, and rebuild from `C.Residual`. Reapplication therefore replaces an earlier `FullBox` order or a Pólya, Putinar, or `SparseFullBox` selection rather than compounding it.

The idea is to represent the oriented positive target as a sum of PSD Gram forms multiplied by functions that are nonnegative on the local box:

$$S(\alpha) = \sum_{J \subseteq \{1, \dots, \ell\}} \left(\prod_{s \in J} \alpha_s (1 - \alpha_s) \right) Z_J(\alpha)^\top Q_J Z_J(\alpha), \quad Q_J \succeq 0.$$

In one parameter the implementation uses the even/odd Markov–Lukács forms, with minimum absolute order $\lfloor M/2 \rfloor$ for assembled residual degree M . For two or more parameters it uses every subset product of the ℓ box generators and minimum order $\lceil M/2 \rceil$. The selected order fixes the dense Bernstein Gram bases; it is not added to M .

Examples and output

```

yalmip('clear')
P = pdvar(1, {[0 1]}, Degree=2);
direct = P >= 0;
box = direct.applyFullBoxPreorder();
directConstraintCount = numel(direct.Constraints)
useFullBox = box.UseFullBoxPreorder
fullBoxOrder = box.FullBoxOrder
fullBoxConstraintCount = numel(box.Constraints)

```

```

directConstraintCount = 3
useFullBox =
    logical
         1
fullBoxOrder = 1
fullBoxConstraintCount = 5

```

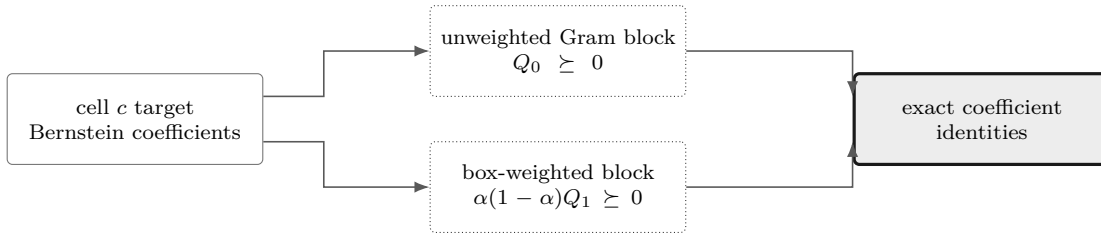
At this order the scalar degree-two cell has two PSD Gram constraints followed by three exact Bernstein coefficient identities.

Remarks

For $\ell = 2$, the FullBox state differs from Putinar by retaining the product generator. With $g_s = \alpha_s(1 - \alpha_s)$, the full-box preordering has the exact backend form

$$S = S_0 + g_1 S_1 + g_2 S_2 + g_1 g_2 S_{12}.$$

The Bernstein basis degrees are respectively (r, r) , $(r - 1, r)$, $(r, r - 1)$, and $(r - 1, r - 1)$; each present term is a PSD Gram block. All four terms are elevated and coefficient-matched at tensor degree $(2r, 2r)$. Thus the additional S_{12} block is precisely the two-generator distinction from the singleton Putinar module, not a change to the residual, grid, or decision degree. The implementation builds the PSD blocks and exact equalities independently for each cell, rate row, and scalar matrix entry.



Appendix B derives the one-dimensional parity forms, explains the multivariate subset masks, and distinguishes this implemented box preordering from the multivariate singleton-generator Putinar branch, full-space SOS, and general polynomial parsers. Assembly is independent for every physical cell and every active rate-vertex row; Gram variables are not shared across either boundary. For $\geq$, the method represents the residual. For $\leq$, it negates the target before forming the positive-semidefinite representation. Within each cell and row, the stored constraints are ordered as all PSD Gram blocks followed by exact equalities matching every Bernstein coefficient. Consequently, the number of stored constraints is not the direct coefficient count.

Remarks

1. Malformed orders raise `pdlmi:InvalidFullBoxOrder`; an admissible integer below the minimum raises `pdlmi:FullBoxOrderTooLow`. The constructor rejects malformed selectors with `pdlmi:InvalidUseFullBoxPreorder`.
2. Simultaneous Pólya, Putinar, SparseFullBox, or FullBox selection raises `pdlmi:ConflictingRelaxations`; expression, relation, and option validation in section 6.3 still applies.
3. Global inequality classification is unchanged: the family is semidefinite only when every original coefficient across all cells and rate rows is square Hermitian. Otherwise the complete family uses independent entry-wise scalar FullBox certificates in MATLAB column-major order.

6.9 Export with toYalmip

Syntax

```
F = toYalmip(C)
F = C.toYalmip()
```

Arguments

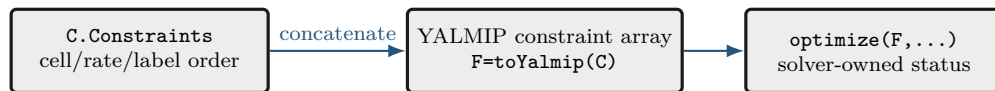
For a `pdvar` residual, `toYalmip` concatenates the stored constraint cells into a YALMIP constraint array for Direct, Pólya, Putinar, SparseFullBox, and FullBox objects. For a coefficient-backed known `pdmat` residual, Direct and Pólya instead return the one stored scalar logical certificate. Its numeric check uses the wrapper’s global inequality classification and inclusive absolute tolerance 10^{-10} : semidefinite mode symmetrizes each coefficient matrix as $(C + C^T)/2$ and tests its eigenvalues, whereas only element-wise mode tests individual scalar entries. A false result emits warning `pdlmi:InconclusiveCertificate`: failure of this sufficient coefficient certificate does not prove a violation or indefiniteness of the continuous function. Known-data Putinar, SparseFullBox, and FullBox objects still export YALMIP constraints and may contain auxiliary Gram variables. The method has no package-specific options and does not add a solver wrapper, objective, margin, or diagnostic layer.

Examples and output

The function-call and dot-call forms return equivalent YALMIP constraint arrays.

```
yalmip('clear')
P = pdvar(2, {[0 1]}, "symmetric");
C = P >= 0;
wrapperClass = class(C)
storedConstraintCount = numel(C.Constraints)
F1 = toYalmip(C);
F2 = C.toYalmip();
firstIsConstraint = isa(F1, "lmi") || isa(F1, "constraint")
firstConstraintCount = length(F1)
secondIsConstraint = isa(F2, "lmi") || isa(F2, "constraint")
secondConstraintCount = length(F2)
```

```
wrapperClass = 'pdlmi'
storedConstraintCount = 2
firstIsConstraint =
    logical
     1
firstConstraintCount = 2
secondIsConstraint =
    logical
     1
secondConstraintCount = 2
```



After this boundary, the package is no longer assembling Bernstein data; YALMIP owns solver choice, diagnostics, and optimization status.

Examples and output

Known coefficient evidence can be certified without introducing a solver model on the Direct and Pólya paths.

```
yalmip('clear')
A = pdmat([0 1], {[1, 2; 3, 4]}, ...
    Degree=1, RateBounds=[-1 2]);
direct = A >= 0;
directCertificate = toYalmip(direct)
polya = direct.applyPolya(1);
polyaCertificate = toYalmip(polya)
```

```
directCertificate = logical
 1
polyaCertificate = logical
 1
```

6.10 Boundaries and Related APIs

`pdvar`, `rhodiff`, `applyPolya`, `applyPutinar`, `applySparseFullBoxPreorder`, `applyFullBoxPreorder`, `toYalmip`, `optimize`.

6.10.1 Solve with YALMIP

`pdlmi` does not own solver settings or wrap `optimize`. After `toYalmip`, use ordinary YALMIP and validate the returned status before retaining an objective or recovered coefficient data:

```
opts = sdpsettings("solver", solverName, "verbose", 0);
sol = optimize([C1.toYalmip, C2.toYalmip], objective, opts);
assert(sol.problem == 0, sol.info)
objectiveValue = value(objective);
```

A result with `problem ~= 0` is not an optimized certificate value. In particular, numerical, unknown, aborted, memory, and space-exhaustion statuses are inconclusive about both the original continuous problem and the value of γ .

Chapter 7

Shared Helper Utilities

The `+helper` namespace contains exactly six implementation utilities whose ownership genuinely crosses class boundaries: `cellGet`, `chk`, `combRows`, `isZero`, `mkGrid`, and `mkNest`. They are developer-facing building blocks, not an alternative modeling API: ordinary users should enter through `pdbase`, `pdmatrix`, `pdvar`, and `pdlmi`. The object-aware `bernTbl`, `mapVals`, and `matSubs` mechanisms are protected `pdbase` backend utilities and are documented in section 3.9.6, not as `helper.*` functions.

7.1 `helper.cellGet`

Purpose. Read one flat coefficient leaf from a nested physical-cell tree.

Syntax

```
leaf = helper.cellGet(vals, subs)
```

Description

`vals` is addressed as `vals{i1}{i2}...{i_ell}` and `subs` supplies one index per level. The returned `leaf` is the selected flat coefficient cell or rate-row table; no copy, reshaping, or multi-index cell interpretation is introduced.

Examples and output

```
vals = {{{10, 20}}, {{30, 40}}};  
leaf = cell2mat(helper.cellGet(vals, [2 1]))
```

```
leaf = 1×2 double  
      30      40
```

Remarks

The helper deliberately relies on ordinary MATLAB cell indexing. Callers validate the subscript length and bounds before traversal.

7.2 `helper.chk`

Purpose. Apply shared validation predicates while retaining a caller-owned error identifier and message.

Syntax

```
val = helper.chk(val, errId, msg, tags...)
val = helper.chk(val, errId, msg, tags..., Name, Value)
```

Description

Predicate tags are `numeric`, `real`, `cell`, `struct`, `nonempty`, `scalar`, `vector`, `matrix`, `finite`, `integer`, `positive`, `nonnegative`, `increasing`, and `rowbounds`. Named tests are `Size`, `Numel`, `MinNumel`, `Min`, and `Max`. Successful validation returns `val` unchanged.

Examples and output

```
validated = helper.chk([1 2], "demo:BadValue", "bad value", ...
    "numeric", "real", "finite", "Size", [1 2])
```

```
validated = 1×2 double
           1     2
```

Remarks

A failed data predicate raises `errId`. An unknown predicate, or a named test without a value, raises `helper.InvalidValidatorCall`; these errors signal a validator-programming defect rather than bad caller data.

7.3 helper.combRows

Purpose. Enumerate Cartesian combinations in the ordering shared by physical cells, local labels, and rate vertices.

Syntax

```
rows = helper.combRows(vecs)
```

Description

`vecs` is a cell array containing one vector per tensor axis. `rows` contains every Cartesian combination, with earlier dimensions varying more slowly.

Examples and output

```
rows = helper.combRows({0:1, 10:11})
```

```
rows = 4×2 double
    0    10
    0    11
    1    10
    1    11
```

Remarks

This ordering is part of the storage contract. Reordering the rows would misalign tensor labels, physical-cell traversal, and derivative rate vertices.

7.4 helper.isZero

Purpose. Prove the distinct zero notions used by known-data and affine algebra fast paths.

Syntax

```
tf = helper.isZero(val, "num")
tf = helper.isZero(val, "add", matrixSize)
tf = helper.isZero(vals, "vals")
tf = helper.isZero(obj, "obj")
```

Description

"num" requires a nonempty finite real numeric zero; "add" additionally enforces scalar expansion or the supplied matrix shape; "vals" recursively checks nested, symbolic, and rate-structured payloads; and "obj" accepts coefficient-backed `pdmat` or `pdvar` evidence.

Examples and output

```
numericZero = helper.isZero(zeros(2), "add", [2 2])
A = pdmat({[0 1]}, {0, 0}, Degree=1);
objectZero = helper.isZero(A, "obj")
```

```
numericZero =
    logical
    1
objectZero =
    logical
    1
```

Remarks

Function-only `pdmatrix` placeholders are not coefficient evidence and therefore do not prove object zero. Invalid modes raise `helper:InvalidZeroMode`; the wrong mode-specific arity raises `helper:InvalidZeroCall`.

7.5 `helper.mkGrid`

Purpose. Validate tensor-grid vectors and construct the shared `GridInfo` record.

Syntax

```
info = helper.mkGrid(grid)
info = helper.mkGrid(grid, owner)
```

Description

`grid` is a nonempty cell array of finite, real, strictly increasing vectors with at least two nodes each. `owner` defaults to `"pdbase"` and prefixes validation identifiers. The result contains `Vectors`, Cartesian-product `Points`, per-axis `Bounds`, and `NumNodes`.

Examples and output

```
info = helper.mkGrid({[0 1], [10 20]}, "demo");
bounds = info.Bounds
points = info.Points
```

```
bounds = 2x2 double
    0     1
   10    20
```

```
points = 4x2 double
    0    10
    0    20
    1    10
    1    20
```

Remarks

Malformed containers use `owner + ":InvalidGrid"`; malformed individual axes use `owner + ":InvalidGridVector"`. Cell counts are derived by consumers and are not duplicated in `GridInfo`.

7.6 `helper.mkNest`

Purpose. Allocate recursive physical-cell storage and construct each leaf from its complete cell subscript.

Syntax

```
vals = helper.mkNest(nCell, mkLeaf)
vals = helper.mkNest(nCell, mkLeaf, prefix)
```

Description

`nCell` gives the positive cell count per parameter direction and `mkLeaf` receives one completed one-based subscript row. `prefix` is recursion state used internally; ordinary callers omit it. The result is addressed as `vals{i1}{i2}...{i_ell}`.

Examples and output

```
vals = helper.mkNest([2 1], @(subs) {sum(subs)});
secondLeaf = cell2mat(helper.cellGet(vals, [2 1]))
```

```
secondLeaf = 3
```

Remarks

`mkLeaf` owns the leaf payload. The helper supplies deterministic physical-cell subscripts but does not infer coefficient count, matrix size, or rate semantics.

Chapter 8

Worked Solver Examples

8.1 Deterministic Putinar, SparseFullBox, And FullBox Selection

This example selects default Putinar and FullBox certificates, an explicit FullBox order, and the three canonical SparseFullBox outcomes without solving an optimization problem. The reported values depend only on the public assembly contract, not on incidental YALMIP decision-variable identifiers.

```
yalmisclear
P = pdvar(1, {[0 1]}, Degree=2);
direct = P >= 0;
putinarDefault = direct.applyPutinar();
boxDefault = direct.applyFullBoxPreorder();
boxHigher = boxDefault.applyFullBoxPreorder(2);
polya = direct.applyPolya(1);
boxFromPolya = polya.applyFullBoxPreorder();
putinarFromBox = boxHigher.applyPutinar();
Fbox = toYalmip(boxDefault);

directState = [direct.UseFullBoxPreorder, direct.FullBoxOrder, ...
    numel(direct.Constraints)]
defaultState = [boxDefault.UseFullBoxPreorder, boxDefault.FullBoxOrder, ...
    numel(boxDefault.Constraints)]
putinarState = [putinarDefault.UsePutinar, putinarDefault.PutinarOrder, ...
    numel(putinarDefault.Constraints)]
higherState = [boxHigher.FullBoxOrder, numel(boxHigher.Constraints)]
replacementState = [boxFromPolya.UsePolya, ...
    boxFromPolya.UseFullBoxPreorder, boxFromPolya.FullBoxOrder]
putinarReplacementState = [putinarFromBox.UsePutinar, ...
    putinarFromBox.UseFullBoxPreorder, putinarFromBox.PutinarOrder]
exportedConstraintCount = length(Fbox)
```

```
directState = 1×3 double
    0     0     3
defaultState = 1×3 double
    1     1     5
putinarState = 1×3 double
    1     1     5
higherState = 1×2 double
    2     7
replacementState = 1×3 double
    0     1     1
putinarReplacementState = 1×3 double
    1     0     1
exportedConstraintCount = 5
```

For the degree-two scalar residual, order one uses two PSD Gram blocks followed by three exact coefficient identities. Order two matches a degree-four target and stores two PSD blocks followed by

five identities. The original `direct` object remains unchanged, and the selection made from `polya` replaces Pólya rather than elevating its already assembled constraints. Likewise, selecting Putinar from `boxHigher` rebuilds from the original residual and clears the full-box selection.

The next degree-four residual makes width two an intermediate `SparseFullBox` certificate, while widths one and three expose its canonical endpoints.

```
P4 = pdvar(1, {[0 1]}, Degree=4);
direct4 = P4 >= 0;
sparse4 = direct4.applySparseFullBoxPreorder(2, 2);
direct4Endpoint = sparse4.applySparseFullBoxPreorder(1, 2);
full4Endpoint = sparse4.applySparseFullBoxPreorder(3, 2);
sparse4State = [sparse4.UseSparseFullBoxPreorder, ...
    sparse4.SparseFullBoxOrder, sparse4.BandWidth, ...
    numel(sparse4.Constraints)]
direct4State = [direct4Endpoint.UseSparseFullBoxPreorder, ...
    direct4Endpoint.UseFullBoxPreorder, ...
    numel(direct4Endpoint.Constraints)]
full4State = [full4Endpoint.UseSparseFullBoxPreorder, ...
    full4Endpoint.UseFullBoxPreorder, full4Endpoint.FullBoxOrder, ...
    numel(full4Endpoint.Constraints)]
```

```
sparse4State = 1×4 double
    1     2     2     8
direct4State = 1×3 double
    0     0     5
full4State = 1×4 double
    0     1     2     7
```

The intermediate wrapper stores three width-two PSD blocks followed by five identities. Width one rebuilds actual Direct state with five coefficient constraints. Width three satisfies $b = r + 1$ and rebuilds actual FullBox state with two dense PSD blocks followed by the five identities. Every apply call is value-semantic and starts from the stored original residual, so the source wrapper and any earlier selection remain unchanged.

8.2 Solve for a Parameter-Dependent Lyapunov Matrix

The first solver smoke test in `+tests/+pdlmi/test_solver_smoke.m` contrasts a constant Lyapunov matrix with a degree-one, parameter-dependent one. It solves the same decay and positivity constraints in both cases. The degree-one solution is then materialized as known `pdmatrix` data and plotted. `pdmatrix` is a decision-expression class and does not itself provide `plot`; applying `value` to its local YALMIP payloads after a successful solve supplies the numeric Bernstein coefficients for `pdmatrix`.

```
yalmp('clear')
grid = {[0 1]};
A = pdmatrix(grid, @(rho) (1 - rho) * [-1 -1; 1 -1] + ...
    rho * [-1 -10; 0.1 -1], Degree=1);
solver = 'lmilab';
if exist('mosekopt', 'file') ~= 0
    solver = 'mosek';
end
opts = sdpsettings('solver', solver, 'verbose', 0);

Pc = pdmatrix(2, grid, Degree=0); CconstantDecay = Pc * A + A' * Pc <= -1e-10 * eye(2);
CconstantPositive = Pc >= eye(2); solConstant = optimize([CconstantDecay.toYalmip, ...
    CconstantPositive.toYalmip], [], opts);

Pd = pdmatrix(2, grid); CdependentDecay = Pd * A + A' * Pd <= -1e-10 * eye(2);
CdependentPositive = Pd >= eye(2); solDependent = optimize([CdependentDecay.toYalmip, ...
    CdependentPositive.toYalmip], [], opts); assert(solDependent.problem == 0,
    solDependent.info)

Pplot = pdmatrix(grid, ...
    {cellfun(@value, Pd.LocalValues{1}, UniformOutput=false)}, ...
    Degree=Pd.Degree);
figure(Name="Parameter-dependent Lyapunov matrix")
plot(Pplot, SamplesPerCell=40, LineWidth=1.5)
title("Solved parameter-dependent Lyapunov matrix")
```

The figure contains one curve for each entry of the solved 2-by-2 matrix $P(\rho)$, sampled on the parameter interval. It is a diagnostic visualization of the feasible solution, not a certificate beyond the coefficient-wise constraints supplied to the solver. The constant-case result is retained to exercise the comparison in the smoke test; only a MOSEK solve is used there to certify its intended infeasibility distinction.

8.3 Solve a Rate-Dependent Block LMI

This example is adapted from `+tests/+pdlmi/test_solver_smoke.m`. The goal is to find a parameter-dependent $P(\rho)$ and scalar γ such that, on every Bernstein coefficient and rate vertex,

$$\begin{bmatrix} \dot{P} + PA + A^\top P & PB & C^\top \\ B^\top P & -\gamma I & D^\top \\ C & D & -\gamma I \end{bmatrix} \preceq 0, \quad P(\rho) \succeq 0.$$

The derivative term $\dot{P}$ is built with `rhodiff(P, [-1 1])`. The comparison operators create `pdlmi` wrappers, and `toYalmip` hands the coefficient-wise constraints to YALMIP. The second listing assigns the solver- and tolerance-dependent numeric value of γ to `gammaValue`. For this model, `numel(E1.Constraints)` returns 6, and `numel(E2.Constraints)` returns 2.

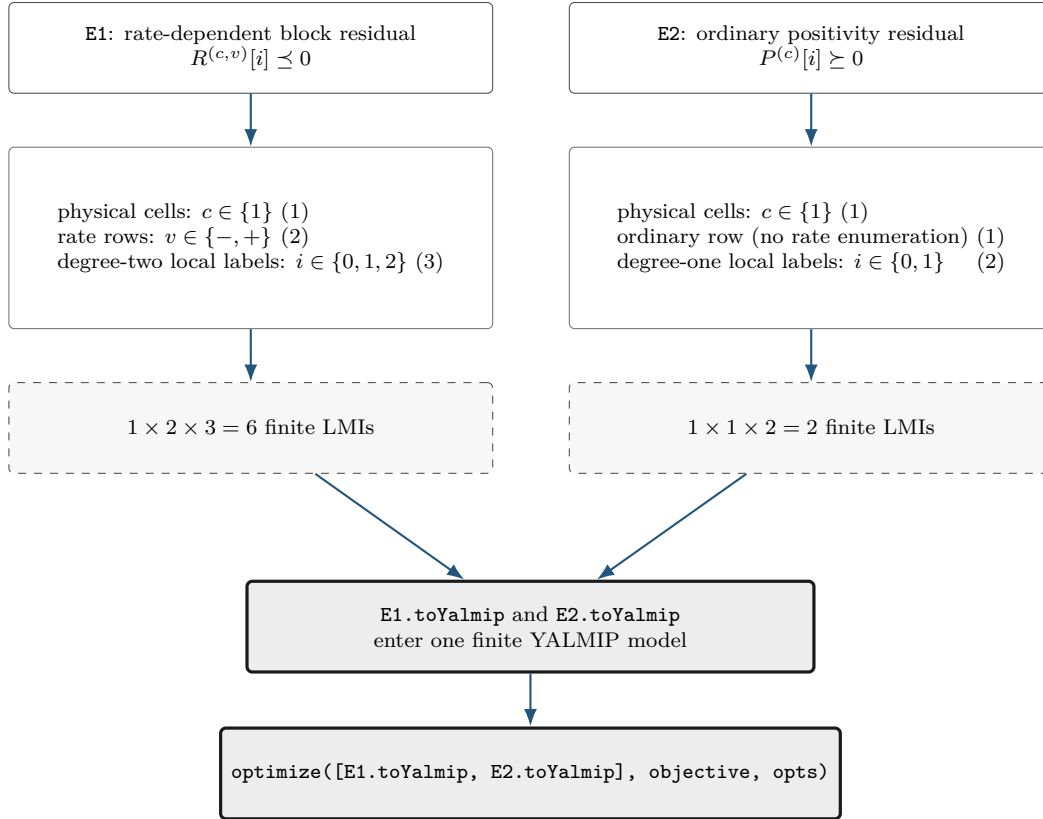
```
yalmip('clear')
grid = linspace(0, 1, 2);
A = pdmat(grid, @(rho) [-1, 0.5; -1, -2] + ...
    rho * [-1.3, -20; 2, -10], Degree=1);
B = pdmat(grid, @(rho) [1, -4; -1, -1] + ...
    rho * [2.2, 0.5; -6, -5], Degree=1);
Cmat = eye(2);
Dmat = zeros(2);

P = pdvar(2, grid);
diffP = rhodiff(P, [-1 1]);
gamma = pdvar(1, grid, Degree=0);
E1 = [diffP + P * A + A' * P, P * B, Cmat';
    B' * P, -gamma * eye(2), Dmat';
    Cmat, Dmat, -gamma * eye(2)] <= 0;
E2 = P >= 0;
decayConstraintCount = numel(E1.Constraints)
positivityConstraintCount = numel(E2.Constraints)
```

```
decayConstraintCount = 6
positivityConstraintCount = 2
```

```
objective = gamma.LocalValues{1}{1};
solver = 'lmilab';
if exist('mosekopt', 'file') ~= 0
    solver = 'mosek';
end
opts = sdpsettings('solver', solver, 'verbose', 0);
sol = optimize([E1.toYalmip, E2.toYalmip], objective, opts);
assert(sol.problem == 0, sol.info)
gammaValue = value(objective);
assert(isfinite(gammaValue))
gammaValue
```

The constraint counts above follow directly from the three independent assembly axes: physical-cell identity, rate-row identity, and local Bernstein label. In this tested example the two wrappers use the same physical grid but different rate-row and degree structures.



The numbers (6) and (2) are certificate-assembly counts: they state how many finite semidefinite constraints represent E1 and E2. They are not evidence that the optimization succeeds. Feasibility is checked separately by `sol.problem`, and the finite objective is checked after the shared `optimize` call.

Appendix A

Bernstein Representation and Exact Operations

This appendix develops the representation used by the package from the one-dimensional degree-one case to cell-local tensor polynomials. The reader needs only scalar polynomials, matrices, and elementary calculus. Every conversion, elevation, product, derivative, and subdivision below is exact; none is a sampling approximation. Farouki gives a broader historical and numerical account [3].

Each main construction follows the same reading path: *Basic idea*, *Formula*, *Worked example*, and *Visual conclusion*. The extra staging is intentional: first identify what the operation means, then read its indices, and only afterward connect it to package storage.

A.1 Convex Combinations Before Polynomials

Basic idea. A convex combination of numbers or matrices has the form

$$X = \sum_{i=0}^m w_i X_i, \quad w_i \geq 0, \quad \sum_{i=0}^m w_i = 1.$$

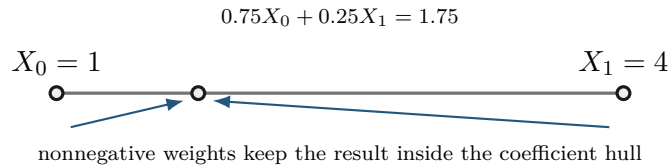
For scalars, X lies between the smallest and largest X_i . For symmetric matrices, if every $X_i \succeq 0$, then $X \succeq 0$, because

$$u^\top X u = \sum_i w_i u^\top X_i u \geq 0 \quad \text{for every } u.$$

Formula. Bernstein bases are useful because their basis values are precisely such weights at every point of a cell.

Worked example. Take $X_0 = 1$, $X_1 = 4$, and weights 0.75 and 0.25. Their combination is 1.75, which remains between the two inputs.

Visual conclusion.



The implication is one-way. A polynomial may be positive everywhere while one Bernstein coefficient is negative; Appendix B uses that fact to motivate stronger certificates.

A.2 Degree-One Interpolation On One Physical Cell

Basic idea. Let $\mathcal{H}_c = [\rho_1^{(c)}, \rho_1^{(c+1)}]$, with width $h_1^{(c)} = \rho_1^{(c+1)} - \rho_1^{(c)} > 0$. Its forward local coordinate is

$$\alpha = \frac{\rho_1 - \rho_1^{(c)}}{h_1^{(c)}} \in [0, 1].$$

Formula. The degree-one Bernstein basis is

$$B_0^1(\alpha) = 1 - \alpha, \quad B_1^1(\alpha) = \alpha.$$

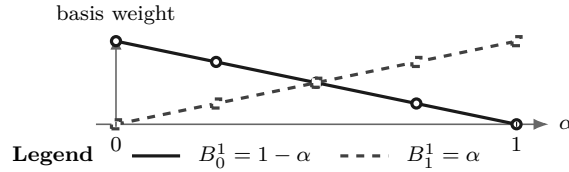
Therefore

$$C^{(c)}(\alpha) = (1 - \alpha)C^{(c)}[0] + \alpha C^{(c)}[1].$$

At the lower physical face, $\alpha = 0$ and the value is $C^{(c)}[0]$; at the upper face, $\alpha = 1$ and the value is $C^{(c)}[1]$. Between them the matrix follows the straight segment joining those two coefficient matrices.

Worked example. For example, $C^{(c)}[0] = 2$ and $C^{(c)}[1] = 4$ give $C^{(c)}(0.25) = 0.75(2) + 0.25(4) = 2.5$, exactly as in section 4.5.

Visual conclusion.



A.3 Higher Degree And The Partition Of Unity

Basic idea. Higher degree adds interior shape controls while keeping the same physical cell. It does not add physical grid nodes.

Formula. For degree m , define

$$B_i^m(\alpha) = \binom{m}{i} (1 - \alpha)^{m-i} \alpha^i, \quad i = 0, \dots, m.$$

Each basis function is nonnegative on $[0, 1]$. The binomial theorem gives

$$\sum_{i=0}^m B_i^m(\alpha) = ((1 - \alpha) + \alpha)^m = 1.$$

Thus

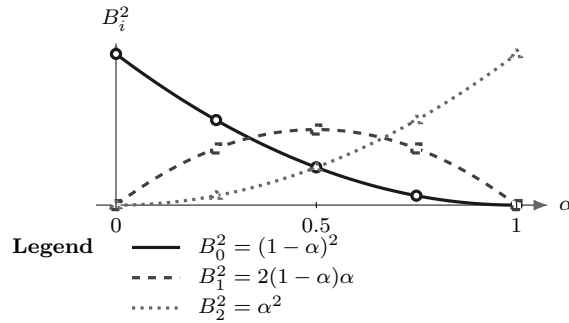
$$M_c(\alpha) = \sum_{i=0}^m B_i^m(\alpha) C^{(c)}[i]$$

is a convex combination of its local coefficient matrices.

Worked example. For degree two,

$$B_0^2 = (1 - \alpha)^2, \quad B_1^2 = 2(1 - \alpha)\alpha, \quad B_2^2 = \alpha^2.$$

Visual conclusion.



At every α , the three plotted heights are nonnegative and sum to one.

The label i identifies a basis position inside the selected physical cell. It is not a physical grid-node subscript.

A.4 Worked Conversion: $1 + 2\alpha + 3\alpha^2$

Consider

$$p(\alpha) = 1 + 2\alpha + 3\alpha^2.$$

Seek degree-two Bernstein coefficients b_0, b_1, b_2 :

$$p = b_0(1 - \alpha)^2 + 2b_1(1 - \alpha)\alpha + b_2\alpha^2.$$

Matching powers of α gives

$$b_0 = 1, \quad b_1 = 2, \quad b_2 = 6.$$

Indeed,

$$1B_0^2 + 2B_1^2 + 6B_2^2 = (1 - 2\alpha + \alpha^2) + (4\alpha - 4\alpha^2) + 6\alpha^2 = 1 + 2\alpha + 3\alpha^2.$$

The three values 1, 2, 6 are coefficients of basis functions, not samples at three equally spaced physical points. Only the two endpoint coefficients are endpoint values. At $\alpha = 1/2$,

$$p(1/2) = \frac{1}{4}(1) + \frac{1}{2}(2) + \frac{1}{4}(6) = 2.75.$$

```
p = pmat({[0 1]}, {1, 2, 6}, Degree=2);
valueAtHalf = p.evaluate(0.5)
T = bernsteinTable(p, "oneLine");
disp(T)
```

valueAtHalf = 2.7500

CellSubscript

Expression

{[1]}

"(1-alpha)^2*1 + 2(1-alpha)alpha*2 + alpha^2*6"

A.5 Exact Power–Bernstein Conversion

More generally, use brackets for algebraic coefficient labels as well:

$$p(\alpha) = \sum_{j=0}^n a[j] \alpha^j = \sum_{k=0}^n b[k] B_k^n(\alpha).$$

Power coefficients convert to Bernstein coefficients by

$$b[k] = \sum_{j=0}^k \frac{\binom{k}{j}}{\binom{n}{j}} a[j], \quad k = 0, \dots, n.$$

The inverse map is

$$a[j] = \binom{n}{j} \sum_{i=0}^j (-1)^{j-i} \binom{j}{i} b[i], \quad j = 0, \dots, n.$$

These are changes of basis inside the same degree- n polynomial space. For matrix polynomials they act entrywise. For tensor polynomials they act along one coordinate axis at a time.

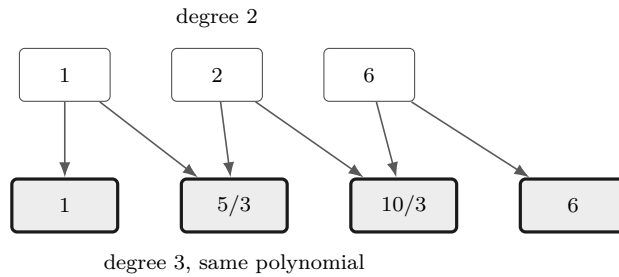
A.6 Lossless Degree Elevation

In one parameter, a polynomial on physical cell c with degree m has a Bernstein representation of every higher degree. One elevation step gives

$$\widehat{C}^{(c)}[0] = C^{(c)}[0], \quad \widehat{C}^{(c)}[k] = \frac{k}{m+1} C^{(c)}[k-1] + \left(1 - \frac{k}{m+1}\right) C^{(c)}[k], \quad \widehat{C}^{(c)}[m+1] = C^{(c)}[m].$$

For the worked coefficients $[1, 2, 6]$, elevation from degree two to degree three produces

$$\left[1, \frac{5}{3}, \frac{10}{3}, 6\right].$$



Direct elevation from m to $M \geq m$ is

$$\widehat{C}^{(c)}[k] = \sum_{i=\max(0, k-M+m)}^{\min(m, k)} C^{(c)}[i] \frac{\binom{m}{i} \binom{M-m}{k-i}}{\binom{M}{k}}.$$

The new coefficients are fixed convex combinations of the old ones. Elevation therefore changes representation but adds no decision freedom. Constructing a new higher-degree `pdvar` does enlarge the decision parameterization.

A.7 Physical Tensor Grids And Local Tensor Labels

Suppose direction s has physical nodes

$$\rho_s^{(1)} < \rho_s^{(2)} < \dots < \rho_s^{(k_s)}.$$

The counts k_s may differ. Thus $\mathcal{G} = \mathcal{G}_1 \times \dots \times \mathcal{G}_\ell$ has $\prod_s k_s$ physical nodes. A physical hypercube is identified separately by the one-based multi-index $\mathbf{c} = (c_1, \dots, c_\ell)$, with $\mathcal{H}_{\mathbf{c}} = \prod_s [\rho_s^{(c_s)}, \rho_s^{(c_s+1)}]$. Define its directional width and forward pullback by

$$h_s^{(\mathbf{c})} = \rho_s^{(c_s+1)} - \rho_s^{(c_s)}, \quad \phi_{\mathbf{c}}(\boldsymbol{\rho}) = \boldsymbol{\alpha}, \quad \alpha_s = \frac{\rho_s - \rho_s^{(c_s)}}{h_s^{(\mathbf{c})}}, \quad s = 1, \dots, \ell.$$

For the package's scalar degree m ,

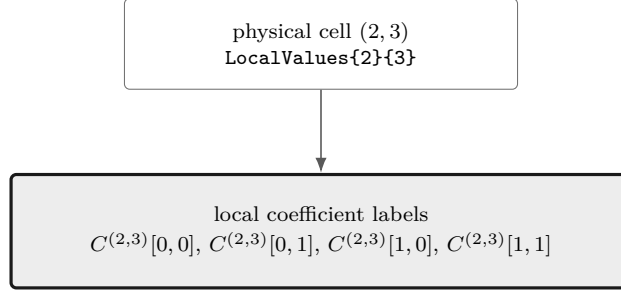
$$C^{(\mathbf{c})}(\phi_{\mathbf{c}}(\boldsymbol{\rho})) = \sum_{\mathbf{i} \in \{0, \dots, m\}^\ell} B_{\mathbf{i}}^m(\boldsymbol{\alpha}) C^{(\mathbf{c})}[\mathbf{i}], \quad B_{\mathbf{i}}^m = \prod_{s=1}^{\ell} B_{i_s}^m(\alpha_s).$$

There are $\prod_s (k_s - 1)$ physical hypercubes and $(m + 1)^\ell$ zero-based local Bernstein labels per hypercube. Physical node and hypercube indices remain one-based.

```
obj = pdbase({[0 1 2], [10 20 30 40]}, [1 1], 1);
physicalCellCount = obj.ncell()
coefficientsPerCell = obj.ncoeff()
localLabels = obj.lbls()
```

```
physicalCellCount = 6
coefficientsPerCell = 4
localLabels = 4×2 double
    0     0
    0     1
    1     0
    1     1
```

The nested tree `LocalValues{c1}{c2}...{cell}` follows physical-cell coordinates. Only after reaching a leaf does the flat coefficient cell follow the `lbls` order. Earlier dimensions vary more slowly in both `cells` and `lbls`.



Select the physical-cell leaf first; only then read the flat local coefficient labels in `lbls` order.

A.8 Products As Ordered Weighted Convolution

Basic idea. Multiplying two Bernstein polynomials creates every ordered pair of input coefficients. Pairs with the same label sum contribute to the same output. For matrix payloads, “ordered” matters: the left factor stays on the left.

Formula. First consider one parameter and fix the one-based physical cell c . Let

$$P = \sum_{i=0}^m B_i^m P^{(c)}[i], \quad Q = \sum_{j=0}^n B_j^n Q^{(c)}[j].$$

The basis-product identity

$$B_i^m B_j^n = \frac{\binom{m}{i} \binom{n}{j}}{\binom{m+n}{i+j}} B_{i+j}^{m+n}$$

gives

$$(PQ)^{(c)}[k] = \sum_{i+j=k} P^{(c)}[i] Q^{(c)}[j] \frac{\binom{m}{i} \binom{n}{j}}{\binom{m+n}{k}}.$$

This is an ordered matrix convolution. In general $P^{(c)}[i] Q^{(c)}[j] \neq Q^{(c)}[j] P^{(c)}[i]$.

Worked example. Take quadratic coefficients $P = [1, 2, 6]$ and affine coefficients $Q = [2, 4]$. The anti-diagonals give

$$\begin{aligned} R[0] &= 1(2) = 2, \\ R[1] &= (1(4) + 2(2))/3 = 4, \\ R[2] &= (2(4) + 6(2))/3 = 28/3, \\ R[3] &= 6(4) = 24. \end{aligned}$$

```
P = pdmat({[0 1]}, {1, 2, 6}, Degree=2);
Q = pdmat({[0 1]}, {2, 4}, Degree=1);
R = P * Q;
productDegree = R.Degree
productCoefficients = cell2mat(R.coeffs(1))
```

```
productDegree = 3
productCoefficients = 1x4 double
    2.0000    4.0000    9.3333   24.0000
```

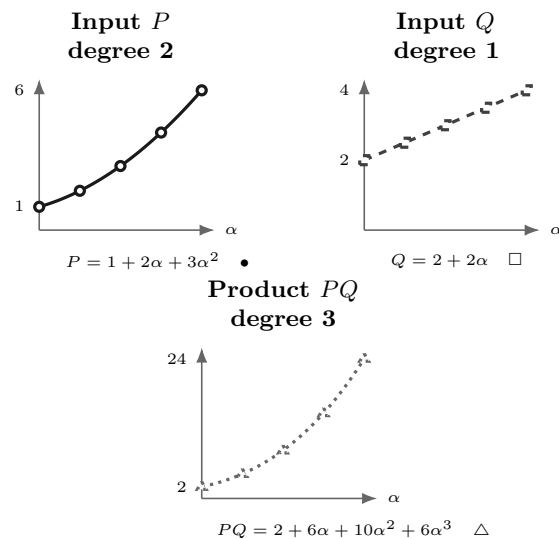
The product degree in the transcript is three: degrees add, so $2 + 1 = 3$. The four outputs are coefficients in that cubic Bernstein basis, not values sampled at four points.

The same calculation can be checked as a function identity. On $\alpha \in [0, 1]$,

$$P(\alpha) = 1 + 2\alpha + 3\alpha^2, \quad Q(\alpha) = 2 + 2\alpha,$$

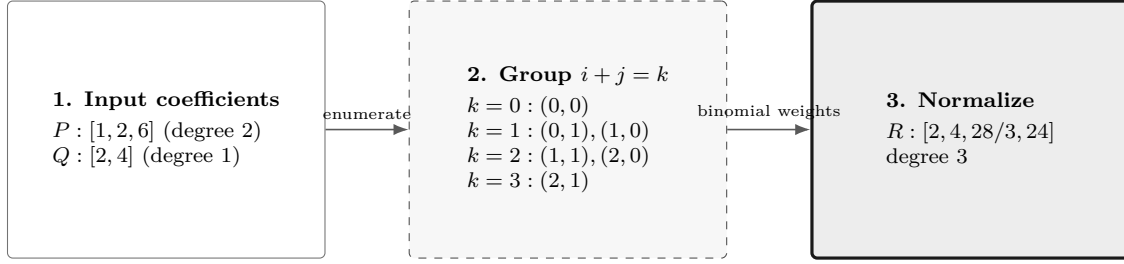
and

$$P(\alpha)Q(\alpha) = 2 + 6\alpha + 10\alpha^2 + 6\alpha^3.$$



Direct formula labels sit outside the plot regions. Solid/circle, dashed/square, and dotted/triangle treatments distinguish the three functions without relying on color.

Visual conclusion.



Only the panel-to-panel arrows encode flow. Contributions are listed inside their anti-diagonal group, so no edges cross labels or equations.

This is the algorithm behind protected `bernProd` (section 3.9.5) and public `pdmat`/`pdvar` multiplication (sections 4.8.2 and 5.7).

For tensor labels,

$$(PQ)^{(c)}[\mathbf{k}] = \sum_{\mathbf{i}+\mathbf{j}=\mathbf{k}} P^{(c)}[\mathbf{i}]Q^{(c)}[\mathbf{j}] \prod_{s=1}^{\ell} \frac{\binom{m}{i_s}\binom{n}{j_s}}{\binom{m+n}{k_s}}.$$

Labels add componentwise; physical-cell identity does not change.

A.9 Differentiation And Rate Vertices

Basic idea. Differentiate inside each physical cell. Adjacent control differences produce the derivative coefficients; multiplying by a bounded rate then creates one stored row per rate-box vertex. Value continuity of P does not force the cell-local derivatives to match across a face.

Formula. For one parameter, on physical cell $\mathbf{c}$ of width $h_1^{(c)}$,

$$\frac{\partial P}{\partial \rho} = \frac{m}{h_1^{(c)}} \sum_{i=0}^{m-1} (C^{(c)}[i+1] - C^{(c)}[i]) B_i^{m-1}(\alpha).$$

Worked example. For the worked polynomial $[1, 2, 6]$ on $[0, 2]$, $m = 2$ and $h = 2$, so the derivative coefficients are

$$\left[\frac{2}{2}(2-1), \frac{2}{2}(6-2) \right] = [1, 4].$$

This represents $dp/d\rho = 1 + 3\alpha$.

In ℓ dimensions, differentiation with respect to ρ_s replaces each coefficient by

$$\frac{m}{h_s^{(c)}} \left(C^{(c)}[\mathbf{i} + \mathbf{e}_s] - C^{(c)}[\mathbf{i}] \right),$$

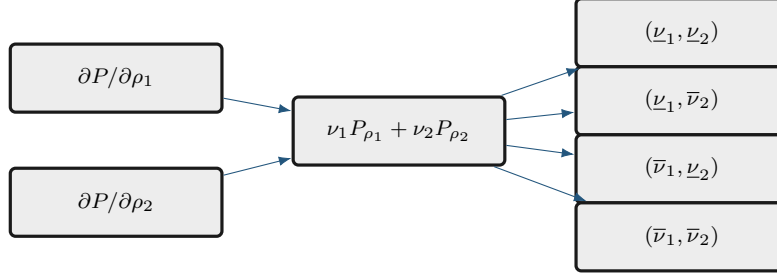
reduces the degree along direction s , and leaves the other label components unchanged. The implementation elevates partial derivatives to a common tensor degree before adding them.

Finally,

$$\dot{P} = \sum_{s=1}^{\ell} \nu_s \frac{\partial P}{\partial \rho_s}, \quad \boldsymbol{\nu} = \dot{\boldsymbol{\rho}}.$$

This matrix expression is affine in $\boldsymbol{\nu}$. Every value inside the rate box $\mathcal{R}$ is therefore a convex combination of the 2^ℓ vertex matrices. Because the positive- and negative-semidefinite cones are convex, the requested semidefinite sign holds on all of $\mathcal{R}$ whenever it holds at every vertex. `rhodiff` stores one row for each vertex.

Visual conclusion.



A.9.1 Rate-Vertex Row Storage

Let $\text{RateBounds}(s, :) = [\underline{\nu}_s \ \bar{\nu}_s]$. The rate box $\mathcal{R} = \prod_s [\underline{\nu}_s, \bar{\nu}_s]$ has the finite vertex set

$$\text{vert}(\mathcal{R}) = \prod_{s=1}^{\ell} \{\underline{\nu}_s, \bar{\nu}_s\}, \quad |\text{vert}(\mathcal{R})| = 2^\ell.$$

For a derivative object $D = \text{rhodiff}(P, \text{RateBounds})$ and one physical hypercube $\mathbf{c} = (c_1, \dots, c_\ell)$, the leaf $\mathbf{D}.\text{LocalValues}\{c_1\} \cdots \{c_\ell\}$ is a 2^ℓ -by- $(\mathbf{D}.\text{Degree} + 1)^\ell$ cell array. Its rows enumerate vertices $\boldsymbol{\nu}^{(q)} \in \text{vert}(\mathcal{R})$; its columns enumerate local Bernstein labels in `combRows` order. Each entry already contains $\sum_s \nu_s^{(q)} \partial P^{(\mathbf{c})} / \partial \rho_s$. The rows add rate alternatives inside one existing hypercube; they do not add physical cells.

For $\ell = 2$, the lower/upper tensor order is

row	ν_1	ν_2
1	$\underline{\nu}_1$	$\underline{\nu}_2$
2	$\underline{\nu}_1$	$\bar{\nu}_2$
3	$\bar{\nu}_1$	$\underline{\nu}_2$
4	$\bar{\nu}_1$	$\bar{\nu}_2$

The last parameter direction changes fastest. This is the same tensor ordering used by the package's coefficient and cell traversal, so `pdlmi` can constrain every rate row without reconstructing the rate box.

A.10 de Casteljau Evaluation And Exact Subdivision

de Casteljau recursion evaluates a Bernstein polynomial using only affine combinations. For one parameter, on a fixed physical cell c , begin with $C^{(c,0)}[i] = C^{(c)}[i]$ and define

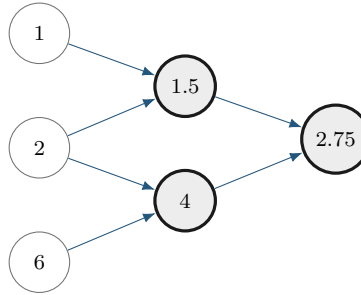
$$C^{(c,r)}[i] = (1 - \tau)C^{(c,r-1)}[i] + \tau C^{(c,r-1)}[i + 1], \quad r = 1, \dots, m.$$

The value at $\alpha = \tau$ is $C^{(c,m)}[0]$.

For $[1, 2, 6]$ and $\tau = 1/2$, the recursion is

$$[1, 2, 6] \longrightarrow [1.5, 4] \longrightarrow [2.75].$$

The left edge of this triangle gives exact coefficients $[1, 1.5, 2.75]$ on the left subinterval; the right edge gives $[2.75, 4, 6]$ on the right.



One recursion triangle supplies both the evaluation and the two child-cell coefficient lists.

Tensor evaluation or subdivision applies the same recursion along one parameter direction at a time. Subdivision changes the physical-cell partition and can tighten coefficient hulls while preserving the polynomial. The package uses exact coefficient transformations internally where documented, but it does not currently expose a public de Casteljaou, subdivision, or adaptive-refinement API.

A.11 What Changes, And What Does Not

The following distinctions prevent several common modeling mistakes.

Operation	Changes	Preserves
Basis conversion	coefficient coordinates	polynomial, degree, physical cell
Degree elevation	Bernstein degree and coefficient list	polynomial, physical grid, decision freedom
Grid subdivision	physical-cell partition and local coefficients	polynomial on the original domain
Higher-degree <code>pdvar</code>	number of decision coefficients	grid and matrix size
Multiplication	polynomial degree and coefficients	physical-cell identity
Differentiation	degree and coefficient values	physical-cell partition before rate rows

These representation facts support the finite certificates in the next appendix. They do not by themselves select a solver or prove that one certificate is necessary.

A.12 Further Reading

For a concise orientation before the primary paper by Farouki [3], see the [Bernstein polynomial overview](#) and the [de Casteljaou overview](#). They are background references for the basis and recursion; the package's cell-local storage and certificate behavior are defined by this manual and the source implementation.

Appendix B

Polynomial-Matrix Certificates on Hypercubes

This appendix starts with positive semidefinite matrices, builds SOS and Gram representations, and then explains the five certificate paths implemented by `pdlmi`: Direct, Pólya, Putinar, SparseFullBox, and FullBox. Background constructions are included so a new reader can recognize the mathematical landscape; only explicitly marked methods are package APIs.

As in the preceding appendix, each major certificate is revealed as *Basic idea* $\rightarrow$ *Formula* $\rightarrow$ *Worked example* $\rightarrow$ *Visual conclusion*. Keep two questions separate while reading: what polynomial matrix is represented, and what finite cone is being used to certify its sign?

For an original residual $\mathcal{F} \preceq 0$, define the sign-normalized positive target $S = -\mathcal{F}$. For $\mathcal{F} \succeq 0$, define $S = \mathcal{F}$. The common problem is

$$S(\boldsymbol{\alpha}) \succeq 0 \quad \text{for every } \boldsymbol{\alpha} \in [0, 1]^\ell.$$

All Gram constraints below prove non-strict inequalities. A strict model must state its own margin, for example $S - \varepsilon I \succeq 0$. The package adds no hidden margin.

B.1 Positive Semidefinite Matrices And LMIs

Basic idea. A real symmetric matrix Q is positive semidefinite when

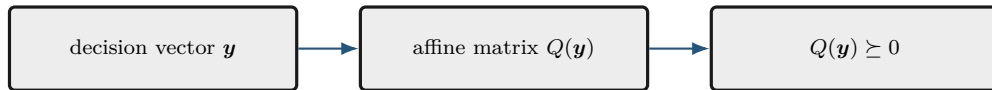
$$u^\top Q u \geq 0 \quad \text{for every vector } u.$$

Formula. Let $\mathbf{y} = (y_1, \dots, y_N)$ collect the scalar decision coordinates. An LMI is an affine matrix expression constrained to the PSD cone:

$$Q(\mathbf{y}) = Q_0 + \sum_{k=1}^N y_k Q_k \succeq 0.$$

The unknowns enter affinely, so this is a semidefinite program. A polynomial matrix sign condition appears infinite because it asks for one PSD matrix at every point. A certificate replaces those pointwise conditions by finitely many PSD matrices and affine coefficient identities.

Visual conclusion.



B.2 SOS And Gram Matrices

Basic idea. A scalar polynomial is a sum of squares (SOS) if

$$p(\xi) = q_1(\xi)^2 + \cdots + q_s(\xi)^2.$$

Every SOS polynomial is nonnegative. Choose a polynomial basis vector $v_d(\xi)$. The Gram form

$$p(\xi) = v_d(\xi)^\top Q v_d(\xi), \quad Q \succeq 0,$$

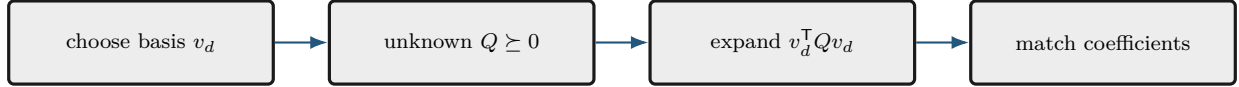
is SOS because a factorization $Q = L^\top L$ yields $p = \|Lv_d\|_2^2$. Expanding the Gram form and matching every coefficient of p gives equations linear in the entries of Q . Thus an SOS search is a finite PSD constraint plus affine equalities.

Formula. For a symmetric n -by- n polynomial matrix, set

$$Z_d(\xi) = v_d(\xi) \otimes I_n, \quad S(\xi) = Z_d(\xi)^\top Q Z_d(\xi), \quad Q \succeq 0.$$

Then $u^\top S(\xi) u = (Z_d(\xi) u)^\top Q (Z_d(\xi) u) \geq 0$ for every u , so $S(\xi) \succeq 0$. Matrix-SOS relaxations are developed in [11].

Visual conclusion.



B.3 Full-Space SOS

An unweighted Gram form proves $S(\xi) \succeq 0$ on all of $\mathbb{R}^\ell$. That can be much stronger than positivity only on a box. For example, $\alpha(1 - \alpha)$ is nonnegative on $[0, 1]$ but negative outside it. The package does not expose a general full-space SOS parser; this construction is background for understanding weighted certificates.

B.4 Direct Bernstein Coefficients

Basic idea. If every local coefficient matrix has the requested sign, every convex combination of those coefficients has the same sign. This yields a small, transparent sufficient certificate.

Formula. After any rate rows have been stacked, write the target on physical cell $\mathbf{c}$ as

$$S(\alpha) = \sum_{\mathbf{i}} B_{\mathbf{i}}^M(\alpha) C^{(\mathbf{c})}[\mathbf{i}].$$

Because the basis weights are nonnegative and sum to one,

$$C^{(\mathbf{c})}[\mathbf{i}] \succeq 0 \ \forall \mathbf{i} \implies S(\alpha) \succeq 0.$$

This is the package default. It introduces no Gram variables and creates one constraint per physical cell, rate row, and local label. The converse is false.

Worked example. Consider

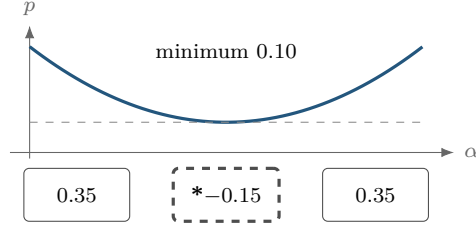
$$p(\alpha) = \alpha^2 - \alpha + 0.35 = (\alpha - 0.5)^2 + 0.10.$$

It is strictly positive, with minimum 0.10, but its degree-two Bernstein coefficients are

$$[0.35, -0.15, 0.35].$$

The negative middle coefficient defeats the direct certificate without making the polynomial negative.

Visual conclusion.



One negative Bernstein coefficient does not imply a negative curve.

The same polynomial has the PSD Bernstein Gram representation

$$\zeta_1 = \begin{bmatrix} 1 - \alpha \\ \alpha \end{bmatrix}, \quad Q = \begin{bmatrix} 0.35 & -0.15 \\ -0.15 & 0.35 \end{bmatrix} \succeq 0, \quad p = \zeta_1^\top Q \zeta_1.$$

The eigenvalues of Q are 0.20 and 0.50. A complete Gram certificate can therefore succeed when coefficient-wise signs fail.

B.5 Pólya As Lossless Elevation

Basic idea. Pólya selection does not change the residual polynomial. It rewrites that same polynomial at a higher Bernstein degree and then retries the direct coefficient test.

Formula. Classical Pólya theory concerns homogeneous polynomials strictly positive on a simplex [14]. For one box coordinate,

$$\lambda_0 = 1 - \alpha, \quad \lambda_1 = \alpha, \quad \lambda_0 + \lambda_1 = 1.$$

For ℓ coordinates, the implemented multiplier is

$$\prod_{s=1}^{\ell} (\lambda_{s,0} + \lambda_{s,1})^d = 1.$$

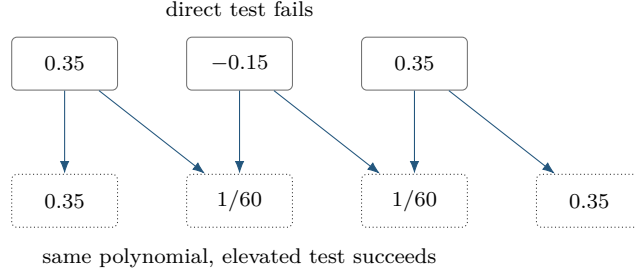
Regrouping is exactly Bernstein degree elevation: the represented residual does not change.

Worked example. For the worked coefficients,

$$[0.35, -0.15, 0.35] \longrightarrow \left[0.35, \frac{1}{60}, \frac{1}{60}, 0.35 \right]$$

after one elevation. Every new coefficient is positive, so the elevated coefficient test succeeds.

Visual conclusion.



`applyPolya(d)` applies one scalar increment in every parameter direction and then coefficient constraints to every cell and rate row. It adds neither Gram variables nor decision freedom. A fixed-level failure is inconclusive; the package makes no tensor-box completeness claim.

B.6 Exact Interval Markov–Lukács Forms

Basic idea. On one interval, endpoint factors can encode the domain exactly. Even and odd target degrees require different weighted Gram patterns, so parity is a structural choice rather than a cosmetic one.

Formula. In one variable, interval nonnegativity has exact parity-specific descriptions [8, 9]. If $\deg p \leq 2r$,

$$p \geq 0 \text{ on } [0, 1] \iff p = s_0 + \alpha(1 - \alpha)s_1,$$

where s_0, s_1 are SOS of degrees at most $2r$ and $2r - 2$. If $\deg p \leq 2r + 1$,

$$p \geq 0 \text{ on } [0, 1] \iff p = (1 - \alpha)s_L + \alpha s_U,$$

where s_L, s_U are SOS of degree at most $2r$.

For polynomial matrices, with

$$\mathcal{Z}_d(\alpha) = \begin{bmatrix} B_0^d I_n & \cdots & B_d^d I_n \end{bmatrix}^\top,$$

the even and odd forms are

$$S = \mathcal{Z}_r^\top Q_0 \mathcal{Z}_r + \alpha(1 - \alpha) \mathcal{Z}_{r-1}^\top Q_1 \mathcal{Z}_{r-1}, \quad Q_0, Q_1 \succeq 0,$$

and

$$S = (1 - \alpha) \mathcal{Z}_r^\top Q_L \mathcal{Z}_r + \alpha \mathcal{Z}_r^\top Q_U \mathcal{Z}_r, \quad Q_L, Q_U \succeq 0.$$

The second even block is absent at $r = 0$. See [10, 11] for polynomial-matrix context.

Worked example. A cubic residual has degree $2r + 1$ with $r = 1$. It therefore uses two degree-one Gram bases: one multiplied by $1 - \alpha$ and one by α . This is the one-dimensional full-box pattern used for the cubic residual in the introductory DPD-LMI example.

Visual conclusion.

even $2r$ unweighted Gram + $\alpha(1 - \alpha)$ -weighted Gram	odd $2r + 1$ $(1 - \alpha)$ lower-face Gram + α upper-face Gram
--	---

Partition Q into n -by- n blocks Q_{ij} . The Bernstein coefficient at label k of $\alpha^a(1 - \alpha)^b \mathcal{Z}_d^\top Q \mathcal{Z}_d$, expressed at total degree $2d + a + b$, is

$$\mathcal{G}_k^{d,a,b}(Q) = \frac{1}{\binom{2d+a+b}{k}} \sum_{\substack{0 \leq i,j \leq d \\ i+j=k-a}} \binom{d}{i} \binom{d}{j} Q_{ij}.$$

Equating this affine map to each target coefficient gives exact linear identities; $Q \succeq 0$ supplies the LMIs. For one parameter, `applyFullBoxPreorder` implements these exact parity forms, while `applySparseFullBoxPreorder` uses the same parity terms and identities with band-limited Gram support at intermediate widths. Their minimum absolute order is $\lfloor M/2 \rfloor$, where M is the assembled residual degree.

B.7 Putinar Modules And Schmüdgen Preorderings

For

$$K = \{\xi : g_1(\xi) \geq 0, \dots, g_q(\xi) \geq 0\},$$

a Putinar quadratic module has the form

$$\sigma_0 + \sum_{j=1}^q \sigma_j g_j, \quad \sigma_j \text{ SOS}.$$

Under the Archimedean hypothesis, every polynomial strictly positive on K has such a representation at some degree [12]. Compactness alone is not that theorem statement. The package implements one box-specific, fixed-order selector, not a general-domain or general-generator parser.

For one parameter, `applyPutinar(r)` dispatches to the same exact even/odd Markov–Lukács forms as `applyFullBoxPreorder(r)`. Its absolute default and minimum are $r = \lfloor M/2 \rfloor$. Even targets use the unweighted and $\alpha(1 - \alpha)$ -weighted Gram blocks and match at degree $2r$; odd targets use the two endpoint-weighted Gram blocks and match at degree $2r + 1$.

For $\ell \geq 2$, use the local box generators

$$g_s(\alpha) = \alpha_s(1 - \alpha_s),$$

and `applyPutinar(r)` uses

$$S = S_0 + \sum_{s=1}^{\ell} g_s S_s, \quad S_0 = \mathcal{Z}_{r1}^\top Q_0 \mathcal{Z}_{r1}, \quad S_s = \mathcal{Z}_{r1-e_s}^\top Q_s \mathcal{Z}_{r1-e_s},$$

where every present $Q_s \succeq 0$. In this multivariate branch the order is absolute and satisfies $r \geq \lceil M/2 \rceil$. Exact identities match the sign-normalized target, elevated without changing it, at tensor degree $2r$. Negative-degree multiplier blocks are omitted at $r = 0$. Each physical cell and stored rate row owns

independent Gram and equality blocks; no generator products, implicit margin, or solver call are introduced.

A Schmüdgen full preordering includes every square-free generator product:

$$p = \sum_{\beta \in \{0,1\}^q} \sigma_\beta \prod_{j=1}^q g_j^{\beta_j}, \quad \sigma_\beta \text{ SOS.}$$

Strict positivity on a compact basic closed semialgebraic set has such a representation [13]. A finite truncation is still a sufficient certificate, and up to 2^q multiplier blocks may be required.

B.8 The Implemented Multivariate FullBox State

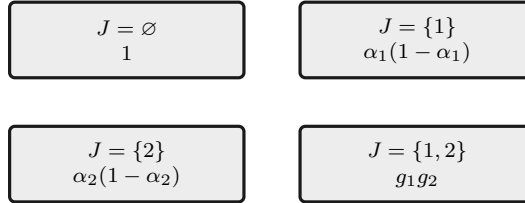
Basic idea. The mathematical construction is a full-box preordering: for several parameters, use every square-free product of the box generators. Each subset selects one weighted Gram block, and exact coefficient identities make the sum equal the sign-normalized target S on one physical cell and stored rate row. The software state is `FullBox`.

Formula. For the hypercube, use

$$g_s(\boldsymbol{\alpha}) = \alpha_s(1 - \alpha_s), \quad s = 1, \dots, \ell.$$

Worked example. For two parameters, the four subset products are $1, g_1, g_2, g_1g_2$.

Visual conclusion.



For $\ell \geq 2$ and absolute order r , define

$$\mathcal{Z}_d = \left(\bigotimes_{s=1}^{\ell} \begin{bmatrix} B_0^{d_s} & \dots & B_{d_s}^{d_s} \end{bmatrix}^T \right) \otimes I_n.$$

The implemented form is

$$S = \sum_{J \subseteq \{1, \dots, \ell\}} \left(\prod_{s \in J} g_s \right) \mathcal{Z}_{r\mathbf{1} - \chi_J}^T Q_J \mathcal{Z}_{r\mathbf{1} - \chi_J}, \quad Q_J \succeq 0.$$

Terms with negative tensor degree are omitted. A present block has dimension

$$n \prod_{s=1}^{\ell} (r + 1 - \mathbf{1}_{s \in J}).$$

Every physical cell and stored rate row gets independent dense Gram blocks and exact coefficient identities. The minimum order is $\lceil M/2 \rceil$ for $\ell \geq 2$. This is a specific dense Bernstein realization of the full-box preordering, not a full-space SOS, Putinar-only, sparse, or general-domain parser. A fixed multivariate order is sufficient rather than exact.

Finite SDP example: $\ell = 2$, $M = 3$. Let $(\alpha_1, \alpha_2) \in [0, 1]^2$ be the local coordinates of one cell, and suppose the sign-normalized target is

$$S^{(\mathbf{c})}(\boldsymbol{\alpha}) = \sum_{i_1=0}^3 \sum_{i_2=0}^3 B_{i_1}^3(\alpha_1) B_{i_2}^3(\alpha_2) C^{(\mathbf{c})}[i_1, i_2] \succeq 0.$$

The default order is $r = \lceil 3/2 \rceil = 2$. After exact elevation to degree $(4, 4)$, the implementation introduces four independent PSD Gram matrices and enforces the identity

$$\begin{aligned} S^{(\mathbf{c})}(\boldsymbol{\alpha}) &= \mathcal{Z}_{(2,2)}^T Q_{\emptyset} \mathcal{Z}_{(2,2)} + \alpha_1(1 - \alpha_1) \mathcal{Z}_{(1,2)}^T Q_1 \mathcal{Z}_{(1,2)} \\ &\quad + \alpha_2(1 - \alpha_2) \mathcal{Z}_{(2,1)}^T Q_2 \mathcal{Z}_{(2,1)} \\ &\quad + \alpha_1(1 - \alpha_1) \alpha_2(1 - \alpha_2) \mathcal{Z}_{(1,1)}^T Q_{12} \mathcal{Z}_{(1,1)}, \\ Q_{\emptyset} &\succeq 0, \quad Q_1 \succeq 0, \quad Q_2 \succeq 0, \quad Q_{12} \succeq 0. \end{aligned}$$

For an n -by- n residual, these blocks have dimensions $9n$, $6n$, $6n$, and $4n$. If

$$S^{(\mathbf{c})}(\boldsymbol{\alpha}) = \sum_{i_1=0}^4 \sum_{i_2=0}^4 B_{i_1}^4(\alpha_1) B_{i_2}^4(\alpha_2) \tilde{C}^{(\mathbf{c})}[i_1, i_2],$$

then $\tilde{C}^{(\mathbf{c})}[0, 0] = C^{(\mathbf{c})}[0, 0]$ and $\tilde{C}^{(\mathbf{c})}[1, 0] = C^{(\mathbf{c})}[0, 0]/4 + 3C^{(\mathbf{c})}[1, 0]/4$. The finite SDP has the four PSD-LMI constraints above and one exact matrix equality for each of the $5 \times 5 = 25$ elevated Bernstein labels. With $[Q]_{\mathbf{i}, \mathbf{j}}$ denoting the n -by- n Gram block indexed by labels $\mathbf{i}, \mathbf{j}$, two illustrative equalities are

$$\begin{aligned} C^{(\mathbf{c})}[0, 0] &= [Q_{\emptyset}]_{(0,0),(0,0)}, \\ \frac{1}{4}C^{(\mathbf{c})}[0, 0] + \frac{3}{4}C^{(\mathbf{c})}[1, 0] &= \frac{1}{2}[Q_{\emptyset}]_{(0,0),(1,0)} + \frac{1}{2}[Q_{\emptyset}]_{(1,0),(0,0)} + \frac{1}{4}[Q_1]_{(0,0),(0,0)}. \end{aligned}$$

The remaining 23 are generated by the same weighted Bernstein coefficient map. Thus the identities are affine equalities, not 25 additional LMIs. A $\preceq 0$ comparison instead matches the negatives of the elevated target coefficients.

```
yalmp('clear')
P = pdvar(1, {[0 1]}, Degree=2);
direct = P >= 0;
polya = direct.applyPolya(1);
putinar = direct.applyPutinar();
box = direct.applyFullBoxPreorder();
directConstraintCount = numel(direct.Constraints)
polyaState = [polya.PolyaDegree, numel(polya.Constraints)]
putinarState = [putinar.PutinarOrder, numel(putinar.Constraints)]
fullBoxState = [box.FullBoxOrder, numel(box.Constraints)]
```

```
directConstraintCount = 3
polyaState = 1×2 double
    1     4
putinarState = 1×2 double
    1     5
fullBoxState = 1×2 double
    1     5
```

Selections rebuild from the stored original **Residual** and **Relation**. Reapplying Pólya, Putinar, SparseFullBox, or FullBox replaces the previous selection; the certificates never compound, and the four opt-in families are mutually exclusive.

B.9 The Implemented SparseFullBox Band-Limited State

Basic idea. SparseFullBox is the FullBox certificate with band-limited Gram support: it retains every FullBox parity or generator-mask term and every exact target-coefficient identity. The implementation realizes the support by replacing one dense Gram matrix with a sum of free PSD clique matrices on overlapping axis-aligned windows of the tensor Bernstein basis. Harris and Parrilo provide a close univariate Bernstein precedent, while factor-width and chordal matrix-SOS work provide broader sparse-Gram context [15, 16, 17]; none of those sources defines this package’s band map or window realization.

Formula. Let one dense FullBox term use tensor basis labels

$$\mathcal{I}_d = \prod_{s=1}^{\ell} \{0, \dots, d_s\}.$$

For positive integer width b , set $w_s = \min(b, d_s + 1)$. For every start label $\mathbf{t}$ with $0 \leq t_s \leq d_s - w_s + 1$, define

$$W_{\mathbf{t}} = \prod_{s=1}^{\ell} \{t_s, \dots, t_s + w_s - 1\}.$$

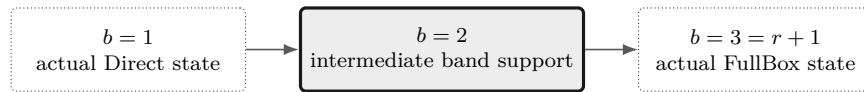
If $\mathcal{Z}_{W_{\mathbf{t}}}$ contains exactly the Bernstein basis functions whose tensor labels lie in $W_{\mathbf{t}}$, SparseFullBox replaces the dense term by

$$\sum_{\mathbf{t}} h_J(\boldsymbol{\alpha}) \mathcal{Z}_{W_{\mathbf{t}}}(\boldsymbol{\alpha})^T Q_{J,\mathbf{t}} \mathcal{Z}_{W_{\mathbf{t}}}(\boldsymbol{\alpha}), \quad Q_{J,\mathbf{t}} \succeq 0,$$

where h_J is the same endpoint or box-generator weight used by the corresponding FullBox term. The implementation sums these weighted Bernstein maps and matches every coefficient of the sign-normalized, degree-elevated target exactly.

Worked example. For one parameter, even assembled residual degree $M = 4$, and absolute order $r = 2$, the unweighted basis has labels $\{0, 1, 2\}$ and the $\alpha(1 - \alpha)$ -weighted basis has labels $\{0, 1\}$. Width $b = 2$ therefore creates unweighted windows $\{0, 1\}$ and $\{1, 2\}$, plus one complete weighted window $\{0, 1\}$. The finite certificate has three PSD blocks followed by five exact coefficient identities. The unweighted support is block tridiagonal. Width $b = 3$ includes every basis label in the unweighted block and reaches the dense FullBox endpoint.

Visual conclusion.

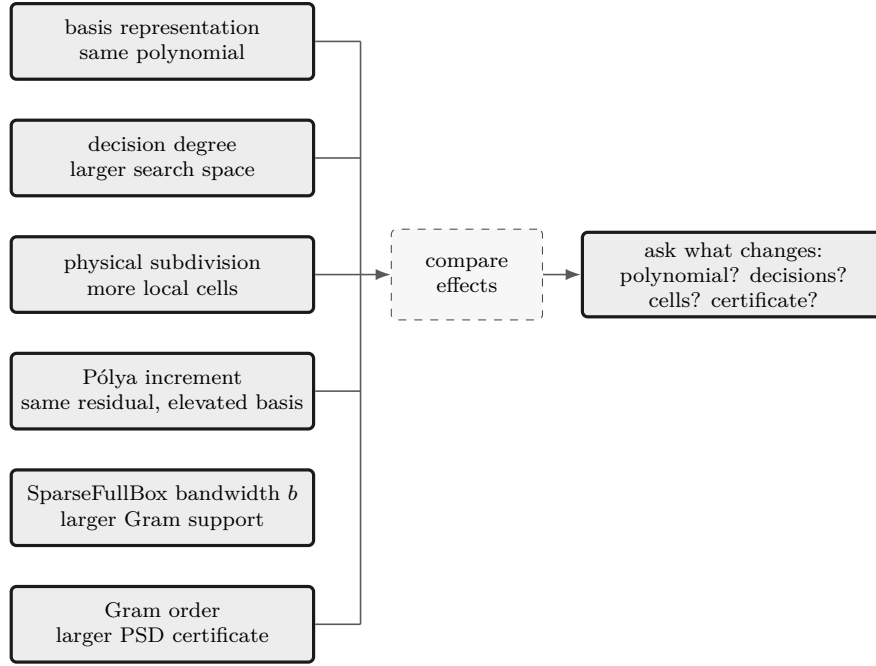


The intermediate cones are nested sufficient certificates; a wider cone need not yield a strict improvement on a particular residual.

`applySparseFullBoxPreorder(b,r)` uses the same dimension-dependent absolute-order minimum as FullBox: $\lfloor M/2 \rfloor$ in one parameter and $\lceil M/2 \rceil$ otherwise. Width one canonicalizes to actual Direct state after order validation. Width at least $r + 1$ canonicalizes to actual FullBox state. Intermediate widths retain SparseFullBox state. Every physical cell, stored rate row, and entry-wise column-major matrix entry owns an independent certificate; the method adds no hidden margin and makes no solver call.

B.10 Six Different Modeling Choices

These choices act on different parts of the problem.



Choice	Changes	Does not mean
basis conversion	coefficient coordinates	more decision freedom
decision degree	independent decision coefficients	certificate order
subdivision	physical cells and local hulls	higher polynomial degree
Pólya increment d	elevated residual coefficients	Gram variables
Putinar/full-box order	Gram bases and PSD block sizes	decision degree
SparseFullBox band-width b	band-limited Gram support and PSD clique sizes	decision degree or physical grid

When a certificate fails, do not infer continuous infeasibility. Decide separately whether the model needs a richer decision degree m , a finer physical grid, a larger Pólya increment d , a wider SparseFullBox window b , or a higher absolute Gram order r .

B.11 Implemented Boundary

Implemented public behavior	Background only
direct coefficient assembly	general full-space SOS parser
uniform <code>applyPolya([d])</code>	direction-wise or automatic Pólya search
<code>applyPutinar([r])</code>	general-domain/general-generator Putinar interface
<code>applySparseFullBoxPreorder([b[,r]])</code>	automatic or adaptive sparse-pattern selection
<code>applyFullBoxPreorder([r])</code>	general-domain Schmüdgen parser
exact one-dimensional parity forms	automatic or adaptive Gram-order search
all cells, active rate rows, and column-major entries	cross-cell/rate/entry Gram sharing
explicit user margins	package-owned strictness or solver policy

B.12 Further Reading

The primary papers by Putinar [12] and Schmüdgen [13] remain the mathematical authority for their respective theorems. For implementation-oriented background, YALMIP maintains an [SOS tutorial](#), and the [Positivstellensatz overview](#) gives a compact map of the surrounding result family. Neither link changes the fixed-order, box-specific package boundary described above.

Bibliography

- [1] C. W. Scherer, “Mixed H_2/H_∞ control for time-varying and linear parametrically-varying systems,” *International Journal of Robust and Nonlinear Control*, vol. 6, nos. 9–10, pp. 929–952, 1996. [publisher DOI record](#).
- [2] F. H. Clarke, *Optimization and Nonsmooth Analysis*. Philadelphia, PA, USA: SIAM, 1990. [doi:10.1137/1.9781611971309](#).
- [3] R. T. Farouki, “The Bernstein polynomial basis: A centennial retrospective,” *Computer Aided Geometric Design*, vol. 29, no. 6, pp. 379–419, 2012. [doi:10.1016/j.cagd.2012.03.001](#).
- [4] M. Zettler and J. Garloff, “Robustness analysis of polynomials with polynomial parameter dependency using Bernstein expansion,” *IEEE Transactions on Automatic Control*, vol. 43, no. 3, pp. 425–431, 1998. [doi:10.1109/9.661615](#).
- [5] I. Masubuchi, A. Kume, and E. Shimemura, “Spline-type solution to parameter-dependent LMIs,” in *Proceedings of the 37th IEEE Conference on Decision and Control*, vol. 2, 1998. [doi:10.1109/CDC.1998.758549](#).
- [6] Y. Xu and F. Jabbari, “Spline-based parameter varying output feedback synthesis with improved L_2 gain,” in *2024 IEEE 63rd Conference on Decision and Control*, pp. 1455–1460, 2024. [doi:10.1109/CDC56724.2024.10886461](#).
- [7] G. Chesi, “LMI techniques for optimization over polynomials in control: a survey,” *IEEE Transactions on Automatic Control*, vol. 55, no. 11, pp. 2500–2510, 2010. [doi:10.1109/TAC.2010.2046926](#).
- [8] F. Lukács, “Verschärfung des ersten Mittelwertsatzes der Integralrechnung für rationale Polynome,” *Mathematische Zeitschrift*, vol. 2, pp. 295–305, 1918. [EuDML record](#).
- [9] E. de Klerk, R. Hess, and M. Laurent, Improved convergence rates for Lasserre-type hierarchies of upper bounds for box-constrained polynomial optimization, *SIAM Journal on Optimization*, vol. 27, no. 1, pp. 347–367, 2017. [doi:10.1137/16M1065264](#).
- [10] E. M. Aylward, S. M. Itani, and P. A. Parrilo, Explicit SOS decompositions of univariate polynomial matrices and the Kalman–Yakubovich–Popov lemma, in *Proceedings of the 46th IEEE Conference on Decision and Control*, pp. 5660–5665, 2007. [author manuscript](#).
- [11] C. W. Scherer and C. W. J. Hol, Matrix sum-of-squares relaxations for robust semi-definite programs, *Mathematical Programming*, vol. 107, pp. 189–211, 2006. [doi:10.1007/s10107-005-0684-2](#).
- [12] M. Putinar, “Positive polynomials on compact semi-algebraic sets,” *Indiana University Mathematics Journal*, vol. 42, no. 3, pp. 969–984, 1993. [JSTOR 24897130](#).
- [13] K. Schmüdgen, “The K -moment problem for compact semi-algebraic sets,” *Mathematische Annalen*, vol. 289, pp. 203–206, 1991. [doi:10.1007/BF01446568](#).

- [14] V. Powers and B. Reznick, “A new bound for Pólya’s theorem with applications to polynomials positive on polyhedra,” *Journal of Pure and Applied Algebra*, vol. 164, nos. 1–2, pp. 221–229, 2001. [doi:10.1016/S0022-4049\(00\)00155-9](https://doi.org/10.1016/S0022-4049(00)00155-9).
- [15] M. T. Harris and P. A. Parrilo, “Improved nonnegativity testing in the Bernstein basis via geometric means,” arXiv:2309.10675, 2023. [doi:10.48550/arXiv.2309.10675](https://doi.org/10.48550/arXiv.2309.10675).
- [16] J. Gouveia, A. Kovačec, and M. Saeed, “On sums of squares of k -nomials,” *Journal of Pure and Applied Algebra*, vol. 226, no. 1, art. 106820, 2022. [doi:10.1016/j.jpaa.2021.106820](https://doi.org/10.1016/j.jpaa.2021.106820).
- [17] Y. Zheng and G. Fantuzzi, “Sum-of-squares chordal decomposition of polynomial matrix inequalities,” *Mathematical Programming*, vol. 197, pp. 71–108, 2023. [doi:10.1007/s10107-021-01728-w](https://doi.org/10.1007/s10107-021-01728-w).

Index

- affine decision expression, 66
- applyFullBoxPreorder, 103
- applyPolya, 98
- applyPutinar, 99
- applySparseFullBoxPreorder, 101

- band-limited Gram support, 101
- BandWidth, 92, 101
- bandwidth, 135
- bernElev, 33
- bernProd, 34
- Bernstein basis, 119
- Bernstein coefficient, 119
- Bernstein product, 54
- bernsteinTable, 46
- bernTbl, 34
- block assembly, 62

- cell-local storage, 7
- cells, 20
- chain rule, 4
- coefficient matching, 129
- coeffs, 20
- concatenation, 62
- conjugate transpose, 57
- ContainsDecision, 19
- continuity, 6

- de Casteljau evaluation, 126
- decision matrix, 66
- Degree, 19
- degree elevation, 121
- degree reduction, 119
- derivative data, 6
- diagonal, 59
- direct assembly, 96
- direct coefficient assembly, 96
- disp, 48
- display, 48

- elevate, 30
- elevLocalValues, 32
- elevVals, 31
- end, 64
- equality, 55
- evaluate, 21, 44

- full-box preordering, 103
- FullBox, 103
- FullBoxOrder, 92

- Gram matrix, 129
- GridInfo, 19

- inherited methods, 25
- installation, 13
- IsContinuous, 19

- known data, 54
- known-data certificate, 55

- lbls, 20
- Lipschitz regularity, 6
- local Bernstein label, 7
- LocalValues, 19

- mapVals, 34
- Markov–Lukács form, 131
- Markov–Lukács, 99
- matrix operations, 25
- matSubs, 34
- mergeGrid, 35

- ncell, 23
- ncoeff, 23
- npar, 23
- numArgumentsFromSubscript, 64
- numel, 56

- ordered matrix multiplication, 54

- Pólya certificate, 98
- Pólya elevation, 98
- parameter rate, 4
- path conflict, 13
- pdbase, 16
- pdlmi, 91
- pdmatrix, 9
- pdvar, 9
- physical grid, 5
- physical hypercube, 5
- physical node, 5
- plot, 49
- Pólya, 98
- positive semidefinite, 128

Positivstellensatz, 132
Putinar, 99
Putinar certificate, 99
Putinar order, 99
`PutinarOrder`, 92

quadratic module, 132

rate row, 4, 22, 45
rate vertex, 4
reshape, 58
residual, 11
`rhodiff`, 22, 45, 72

Schmüdgen preordering, 132
shared face, 6
solver diagnostics, 12
solver probe, 14
`SparseFullBox`, 101, 135
`SparseFullBoxOrder`, 92
strictness margin, 12
subdivision, 126
`subsasgn`, 64
`subsref`, 64
sum of squares, 129

tensor grid, 5
tensor window, 135
`toYalmip`, 105
trace, 59
transpose, 57
triangular part, 60

validation, 14
`value`, 87
vectorization, 58

YALMIP, 105
YALMIP dependency, 13